

**ĐẠI HỌC TRÀ VINH
TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ**



ISO 9001:2015

NGUYỄN THANH HIẾU

**XÂY DỰNG AI AGENT
GIỚI THIỆU ĐẠI HỌC TRÀ VINH
VÀ CÁC NGÀNH ĐÀO TẠO**

**ĐỒ ÁN TỐT NGHIỆP
NGÀNH CÔNG NGHỆ THÔNG TIN**

VĨNH LONG, NĂM 2026

**ĐẠI HỌC TRÀ VINH
TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ**

**XÂY DỰNG AI AGENT
GIỚI THIỆU ĐẠI HỌC TRÀ VINH
VÀ CÁC NGÀNH ĐÀO TẠO**

**ĐỒ ÁN TỐT NGHIỆP
NGÀNH CÔNG NGHỆ THÔNG TIN**

Sinh viên: **Nguyễn Thanh Hiếu**

Lớp: **DA22TTA**

MSSV: **110122221**

GVHD: **TS. Nguyễn Nhứt Lam**

VĨNH LONG, NĂM 2026

BẢN NHẬN XÉT ĐỒ ÁN TỐT NGHIỆP
(Dành cho người hướng dẫn)

Ngành: Công nghệ thông tin

Họ và tên người hướng dẫn: Nguyễn Nhứt Lam

Đơn vị công tác: Khoa Công nghệ Thông tin

Họ và tên sinh viên: Nguyễn Thanh Hiếu

MSSV: 110122221

Tên đề tài: Xây dựng AI Agent giới thiệu Đại học Trà Vinh và các ngành đào tạo.

1. Tinh thần, thái độ học tập, nghiên cứu của sinh viên:

.....
.....

2. Khả năng nghiên cứu khoa học:

.....
.....

3. Hình thức và nội dung của đồ án:

.....
.....

4. Kết luận:

.....
.....

Vĩnh Long, ngày tháng năm 2026
Người hướng dẫn
(Ký và ghi rõ họ tên)

LỜI CAM ĐOAN

Tôi xin cam đoan đồ án tốt nghiệp với đề tài “Xây dựng AI Agent giới thiệu Đại học Trà Vinh và các ngành đào tạo” là kết quả nghiên cứu và thực hiện của cá nhân tôi dưới sự hướng dẫn của TS. Nguyễn Nhứt Lam.

Toàn bộ nội dung, số liệu, kết quả kiểm thử và sản phẩm được trình bày trong đồ án phản ánh trung thực quá trình nghiên cứu, phân tích, thiết kế, xây dựng và triển khai hệ thống. Trong quá trình thực hiện, tôi có tham khảo các tài liệu, công trình nghiên cứu, thư viện và nguồn thông tin liên quan. Những nội dung tham khảo đều được trình bày hoặc trích dẫn theo quy định.

Tôi xin chịu trách nhiệm về tính trung thực và nội dung của đồ án tốt nghiệp này.

Sinh viên thực hiện

Nguyễn Thanh Hiếu

LỜI CẢM ƠN

Trước hết, em xin gửi lời cảm ơn chân thành đến quý thầy cô Trường Kỹ thuật và Công nghệ, Đại học Trà Vinh đã tận tình giảng dạy, truyền đạt kiến thức và tạo điều kiện thuận lợi cho em trong suốt quá trình học tập và thực hiện đồ án tốt nghiệp.

Em xin bày tỏ lòng biết ơn sâu sắc đến TS. Nguyễn Nhứt Lam, giảng viên hướng dẫn đồ án tốt nghiệp, đã dành thời gian hướng dẫn, góp ý và định hướng cho em trong quá trình thực hiện đề tài “Xây dựng AI Agent giới thiệu Đại học Trà Vinh và các ngành đào tạo”. Những ý kiến đóng góp của thầy đã giúp em nhìn nhận vấn đề rõ ràng hơn, hoàn thiện nội dung nghiên cứu và xây dựng hệ thống theo hướng thực tế hơn.

Em cũng xin cảm ơn quý thầy cô khoa Công nghệ thông tin đã trang bị cho em nền tảng kiến thức chuyên môn về lập trình, cơ sở dữ liệu, phát triển ứng dụng web và trí tuệ nhân tạo trong suốt quá trình học tập. Những kiến thức này là cơ sở quan trọng giúp em thực hiện các bước phân tích, thiết kế và xây dựng hệ thống trong đồ án.

Bên cạnh đó, em xin gửi lời cảm ơn đến gia đình và bạn bè đã luôn động viên, hỗ trợ về tinh thần và tạo điều kiện để em hoàn thành quá trình học tập cũng như thực hiện đề tài. Sự quan tâm và khích lệ của mọi người trong thời gian qua là nguồn động lực lớn giúp em cố gắng hoàn thiện đồ án một cách tốt nhất trong khả năng của mình.

Do kiến thức chuyên môn và kinh nghiệm thực tế còn hạn chế, đồ án khó tránh khỏi những thiếu sót trong quá trình thực hiện và trình bày. Vì vậy, em rất mong nhận được sự góp ý của quý thầy cô để có thể tiếp tục hoàn thiện kiến thức, rèn luyện kỹ năng và phát triển hệ thống theo hướng tốt hơn trong tương lai.

Em xin chân thành cảm ơn!

Sinh viên thực hiện

Nguyễn Thanh Hiếu

MỤC LỤC

Nội dung	Trang
MỤC LỤC	iii
CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI	1
1.1 Bối cảnh và vấn đề đặt ra	1
1.1.1 Thực trạng truyền thông, tuyển sinh và tham quan trường đại học	1
1.1.2 Nhu cầu trải nghiệm tham quan ảo trong giáo dục	1
1.1.3 Vai trò của trí tuệ nhân tạo trong tương tác và tư vấn thông tin	2
1.2 Giới thiệu bài toán xây dựng AI Agent giới thiệu Đại học Trà Vinh	3
1.2.1 Đặc điểm bài toán	3
1.2.2 Người dùng mục tiêu	3
1.2.3 Các chức năng chính cần xây dựng	4
1.3 Khảo sát các giải pháp liên quan	4
1.3.1 Google Street View / Google Earth	4
1.3.2 Hệ thống virtual tour của các trường đại học	5
1.3.3 Chatbot tư vấn tuyển sinh và trợ lý AI	5
1.3.4 So sánh và rút ra hướng tiếp cận	6
1.4 Mục tiêu, phạm vi và kết quả mong đợi	7
1.4.1 Mục tiêu tổng quát	7
1.4.2 Mục tiêu cụ thể	7
1.4.3 Phạm vi thực hiện	8
1.4.4 Kết quả dự kiến	8
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG	9
2.1 Kiến trúc ứng dụng web hiện đại	9
2.1.1 Mô hình Client-Server	10
2.1.2 RESTful API	10
2.1.3 Server-Sent Events (SSE)	10
2.1.4 Kiến trúc phân lớp Router – Service – Repository	11
2.2 Công nghệ Frontend	11
2.2.1 Next.js, React và TypeScript	11
2.2.2 Zustand trong quản lý trạng thái	12
2.2.3 Framer Motion trong xử lý hiệu ứng giao diện	12
2.2.4 Pannellum và hiển thị ảnh panorama 360 độ	13
2.2.5 Three.js / React Three Fiber cho nhân vật hướng dẫn 3D	13
2.3 Công nghệ Backend	13
2.3.1 FastAPI	14
2.3.2 SQLAlchemy ORM	14
2.3.3 Supabase PostgreSQL	14
2.3.4 pgvector và lưu trữ vector embedding	15
2.3.5 Cloudflare R2 Object Storage	15
2.3.6 Supabase Auth cho xác thực quản trị	16
2.4 Trí tuệ nhân tạo và xử lý ngôn ngữ tự nhiên	16
2.4.1 Mô hình ngôn ngữ lớn Google Gemini	17
2.4.2 Vector Embedding và tìm kiếm ngữ nghĩa	17
2.4.3 Retrieval-Augmented Generation (RAG)	18
2.4.4 Mô hình AI Agent và cơ chế Function Calling	18

2.5 Công nghệ giọng nói.....	19
2.5.1 Speech-to-Text bằng Web Speech API	20
2.5.2 Text-to-Speech (TTS)	20
2.5.3 Cơ chế cache và fallback âm thanh.....	21
2.6 Ảnh 360 độ và panorama.....	21
2.6.1 Ảnh equirectangular	22
2.6.2 Hiển thị panorama trên trình duyệt	22
2.7 Thuật toán tìm đường A^*	23
2.7.1 Khái niệm graph và waypoint.....	24
2.7.2 Nguyên lý thuật toán A^*	25
2.7.3 Ứng dụng A^* trong điều hướng bản đồ	25
CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG.....	26
3.1 Phân tích yêu cầu	26
3.1.1 Đối tượng sử dụng và phạm vi hệ thống.....	26
3.1.2 Yêu cầu chức năng.....	27
3.1.3 Yêu cầu phi chức năng.....	29
3.1.4 Đặc tả Use Case	30
3.2 Thiết kế kiến trúc tổng thể	33
3.2.1 Sơ đồ kiến trúc hệ thống	33
3.2.2 Luồng tương tác giữa người dùng, frontend, backend và AI	33
3.2.3 Kiến trúc Backend theo mô hình phân lớp	35
3.2.4 Kiến trúc Frontend theo hướng feature-based	36
3.3 Thiết kế cơ sở dữ liệu	36
3.3.1 Sơ đồ ERD (Entity Relationship Diagram)	37
3.3.2 Mô tả các bảng chính	40
3.3.3 Thiết kế vector store bằng pgvector.....	40
3.3.4 Thiết kế lưu trữ media trên Cloudflare R2.....	41
3.4 Thiết kế hệ thống RAG và AI Agent	42
3.4.1 Pipeline nhập liệu tài liệu.....	42
3.4.2 Pipeline truy vấn và sinh câu trả lời.....	43
3.4.3 Thiết kế AI Agent với Function Calling	44
3.4.4 Cơ chế phối hợp giữa AI Agent và giao diện	45
3.5 Thiết kế bản đồ và dẫn đường	47
3.5.1 Mô hình waypoint graph.....	47
3.5.2 Tìm đường bằng thuật toán A^*	48
3.5.3 Lưu trữ dữ liệu đường đi và waypoint.....	48
3.5.4 Hiển thị đường đi trên giao diện	49
3.5.5 Tích hợp điều hướng với AI Agent.....	49
3.6 Thiết kế giao diện người dùng.....	49
3.6.1 Giao diện Kiosk Tour	49
3.6.2 Giao diện Admin Panel.....	51
3.6.3 Luồng trải nghiệm người dùng	51
3.7 Thiết kế vận hành kiosk và quản trị hệ thống.....	52
3.7.1 Tự động reset khi không hoạt động	52
3.7.2 Khóa thao tác ngoài kiosk.....	52
3.7.3 Xử lý lỗi mạng và phục hồi giao diện.....	53
3.7.4 Xác thực và bảo vệ trang quản trị	53
CHƯƠNG 4: XÂY DỰNG HỆ THỐNG	55

4.1 Môi trường và cấu trúc triển khai dự án	55
4.1.1 Công cụ và phiên bản sử dụng.....	55
4.1.2 Cấu trúc thư mục dự án.....	56
4.1.3 Cấu hình biến môi trường	57
4.2 Xây dựng Backend API	58
4.2.1 Khởi tạo FastAPI, CORS và kết nối Supabase PostgreSQL	58
4.2.2 Tổ chức repository, service layer và schema.....	59
4.2.3 Xây dựng API địa điểm, media và bản đồ.....	60
4.2.4 Xây dựng API chat, streaming và quản trị.....	61
4.2.5 Xác thực admin bằng Supabase Auth	62
4.3 Xây dựng hệ thống AI, RAG và giọng nói	63
4.3.1 Tích hợp Gemini Client và cấu hình mô hình	63
4.3.2 Xây dựng pipeline nhập liệu tài liệu RAG	63
4.3.3 Truy vấn vector bằng pgvector và sinh câu trả lời	65
4.3.4 Xây dựng Function Calling và các công cụ của AI Agent	65
4.3.5 Tích hợp TTS, STT và đồng bộ trạng thái nhân vật 3D	66
4.4 Xây dựng giao diện Kiosk Tour.....	67
4.4.1 Trang tour chính, Start Overlay và tải tài nguyên	67
4.4.2 Hiển thị ảnh 360 độ và nhân vật hướng dẫn 3D	68
4.4.3 Chat Overlay, giọng nói và phản hồi của AI Agent	69
4.4.4 Bản đồ khuôn viên, minimap và dẫn đường bằng SVG route animation....	70
4.4.5 Info Panel hiển thị hình ảnh, video và thông tin liên quan	72
4.4.6 Xử lý trạng thái giao diện: loading, chuyển cảnh và lỗi kết nối.....	72
4.5 Xây dựng giao diện Admin Panel	73
4.5.1 Bố cục và bảo vệ trang quản trị	73
4.5.2 Quản lý nội dung tham quan: địa điểm, bản đồ, media và nhân vật 3D.....	74
4.5.3 Quản lý tài liệu RAG và cấu hình hỗ trợ	76
4.6 Ổn định hóa hệ thống và triển khai hệ thống.....	77
4.6.1 Xử lý phiên kiosk, Start Overlay và tự reset khi không hoạt động.....	77
4.6.2 Hạn chế thao tác ngoài luồng và xử lý lỗi giao diện.....	78
4.6.3 Cấu hình triển khai frontend, backend, database và storage.....	79
CHƯƠNG 5: KẾT QUẢ VÀ ĐÁNH GIÁ	80
5.1 Phạm vi và phương pháp đánh giá.....	80
5.2 Kết quả triển khai hệ thống.....	80
5.3 Kết quả kiểm thử trên môi trường production	81
5.4 Nhận xét chung	84
KẾT LUẬN	86
PHỤ LỤC	88
DANH MỤC TÀI LIỆU THAM KHẢO	92

DANH MỤC HÌNH ẢNH

Hình 2.1. Kiến trúc tổng quan ứng dụng web và luồng xử lý trong hệ thống.....	9
Hình 2.2. Quy trình tổng quan AI Agent kết hợp RAG trong hệ thống.....	16
Hình 2.3. Minh họa ảnh panorama 360 độ và cách hiển thị trên trình duyệt.....	22
Hình 2.4. Minh họa thuật toán A^* trên đồ thị waypoint của bản đồ khuôn viên.....	24
Hình 3.1. Sơ đồ Use Case tổng quát của hệ thống	32
Hình 3.2. Sơ đồ kiến trúc tổng thể của hệ thống	33
Hình 3.3. Luồng tương tác giữa người dùng, frontend, backend và AI Agent	34
Hình 3.4. Mô hình dữ liệu mức quan niệm CDM	37
Hình 3.5. Mô hình dữ liệu mức logic LDM	38
Hình 3.6. Mô hình dữ liệu mức vật lý PDM	39
Hình 3.7. Minh họa lưu trữ và truy vấn vector bằng pgvector.....	41
Hình 3.8. Pipeline nhập liệu tài liệu vào kho tri thức RAG	43
Hình 3.9. Cơ chế phối hợp giữa AI Agent và giao diện thông qua Function Calling...	46
Hình 3.10. Mô hình waypoint graph và đường đi trên bản đồ khuôn viên	47
Hình 3.11. Bố cục các thành phần chính trong giao diện Kiosk Tour	50
Hình 4.1. Cấu trúc thư mục chính của dự án.....	56
Hình 4.2. Giao diện Start Overlay trước khi bắt đầu tham quan.....	68
Hình 4.3. Giao diện chính Kiosk Tour với ảnh 360 độ và nhân vật hướng dẫn 3D.....	69
Hình 4.4. Chat Overlay và phản hồi của AI Agent trong giao diện tham quan	70
Hình 4.5. Bản đồ khuôn viên và tuyến đường dẫn đến địa điểm	71
Hình 4.6. Info Panel hiển thị media liên quan đến địa điểm	72
Hình 4.7. Giao diện Dashboard và bố cục chính của Admin Panel	74
Hình 4.8. Giao diện quản lý địa điểm và trạng thái bản đồ trong Admin Panel	75
Hình 4.9. Giao diện quản lý media trong Admin Panel	75
Hình 4.10. Giao diện quản lý nhân vật hướng dẫn 3D trong Admin Panel	76
Hình 4.11. Giao diện quản lý tài liệu RAG và trạng thái xử lý tài liệu.....	77
Hình 5.1. Quy mô dữ liệu triển khai trên production	81
Hình 5.2. Thời gian phản hồi API public qua website	82
Hình 5.3. Kết quả kiểm thử AI Agent và RAG trên production	83
Hình 5.4. Kết quả kiểm thử dẫn đường A^* trên production	83

DANH MỤC BẢNG BIỂU

Bảng 1.1. So sánh các nhóm giải pháp liên quan.....	6
Bảng 3.1. Đối tượng sử dụng của hệ thống.....	26
Bảng 3.2. Yêu cầu chức năng của hệ thống	27
Bảng 3.4. Tóm tắt các Use Case chính	30
Bảng 3.5. Các lớp chính trong kiến trúc backend.....	35
Bảng 3.6. Các nhóm chức năng chính ở frontend.....	36
Bảng 3.7. Mô tả các bảng chính trong cơ sở dữ liệu.....	40
Bảng 3.8. Các công cụ Function Calling của AI Agent.....	44
Bảng 3.9. Các thành phần chính của giao diện Kiosk Tour.....	50
Bảng 3.10. Các nhóm chức năng trong giao diện Admin Panel	51
Bảng 3.11. Tóm tắt các cơ chế vận hành và bảo vệ hệ thống	54
Bảng 4.1. Công cụ và phiên bản sử dụng trong dự án	55
Bảng 4.2. Cấu trúc thư mục chính của dự án.....	57
Bảng 4.3. Các nhóm biến môi trường chính của hệ thống.....	58
Bảng 4.4. Tổ chức các lớp xử lý trong backend	60
Bảng 4.5. Các API công khai phục vụ giao diện tham quan	61
Bảng 4.6. Các nhóm API chính của backend.....	62
Bảng 4.7. Pipeline nhập liệu tài liệu RAG	64
Bảng 4.8. Các công cụ của AI Agent.....	66
Bảng 5.1. Tổng hợp kết quả kiểm thử hệ thống.....	84

KÍ HIỆU CÁC CỤM TỪ VIẾT TẮT

Từ viết tắt	Cụm từ đầy đủ	Diễn giải
AI	Artificial Intelligence	Trí tuệ nhân tạo
AI Agent	Artificial Intelligence Agent	Tác nhân trí tuệ nhân tạo
API	Application Programming Interface	Giao diện lập trình ứng dụng
A*	A-star	Thuật toán tìm đường
ERD	Entity Relationship Diagram	Sơ đồ thực thể - quan hệ
HTTP	HyperText Transfer Protocol	Giao thức truyền tải dữ liệu
HTTPS	HyperText Transfer Protocol Secure	Phiên bản bảo mật của HTTP
JSON	JavaScript Object Notation	Định dạng dữ liệu dùng để trao đổi giữa client và server
JWT	JSON Web Token	Chuẩn token thường dùng trong xác thực
ORM	Object-Relational Mapping	Cơ chế ánh xạ giữa đối tượng trong mã nguồn và bảng dữ liệu
RAG	Retrieval-Augmented Generation	Phương pháp truy xuất tài liệu kết hợp sinh câu trả lời
REST	Representational State Transfer	Kiểu kiến trúc thường dùng khi thiết kế API cho ứng dụng web
SQL	Structured Query Language	Ngôn ngữ truy vấn cơ sở dữ liệu quan hệ
SSE	Server-Sent Events	Cơ chế gửi dữ liệu từ server đến client theo thời gian thực
STT	Speech-to-Text	Chuyển giọng nói thành văn bản
SVG	Scalable Vector Graphics	Định dạng ảnh vector
TTS	Text-to-Speech	Chuyển văn bản thành giọng nói
3D	Three-Dimensional	Không gian hoặc mô hình ba chiều

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI

Chương này trình bày bối cảnh và vấn đề đặt ra trong việc ứng dụng công nghệ số vào hoạt động giới thiệu, tư vấn và tham quan trường đại học. Trên cơ sở đó, chương xác định đặc điểm bài toán xây dựng Artificial Intelligence Agent (AI Agent) giới thiệu Đại học Trà Vinh, đối tượng người dùng, các chức năng chính và khảo sát một số giải pháp liên quan. Cuối cùng, chương làm rõ mục tiêu, phạm vi và kết quả mong đợi của đề tài, tạo cơ sở cho nội dung lý thuyết, thiết kế và triển khai ở những chương tiếp theo.

1.1 Bối cảnh và vấn đề đặt ra

1.1.1 Thực trạng truyền thông, tuyển sinh và tham quan trường đại học

Trong bối cảnh chuyển đổi số diễn ra mạnh mẽ trong giáo dục, các trường đại học ngày càng chú trọng ứng dụng công nghệ nhằm nâng cao hiệu quả truyền thông, tuyển sinh và hỗ trợ người học tiếp cận thông tin. Đối với thí sinh, phụ huynh và sinh viên mới, việc tìm hiểu về trường, ngành đào tạo, học phí, học bổng, cơ sở vật chất và môi trường học tập là nhu cầu quan trọng trước khi đưa ra quyết định học tập.

Tuy nhiên, các hình thức giới thiệu truyền thống như website, tài liệu giới thiệu, bài viết tuyển sinh hoặc tư vấn trực tiếp vẫn còn một số hạn chế. Người dùng thường phải tự tìm kiếm trên nhiều trang khác nhau, nội dung có thể dài, khó tra cứu nhanh và chưa tạo được trải nghiệm tương tác trực quan. Bên cạnh đó, hoạt động tham quan trực tiếp không phải lúc nào cũng thuận tiện, đặc biệt với người học ở xa hoặc cần tìm hiểu sơ bộ trước khi đến trường.

Sự phát triển của trí tuệ nhân tạo, đặc biệt là các mô hình ngôn ngữ lớn, mở ra hướng tiếp cận mới trong xây dựng hệ thống tư vấn thông tin thông minh. Thay vì chỉ hiển thị nội dung tĩnh, hệ thống có thể tiếp nhận câu hỏi tự nhiên, truy xuất thông tin từ kho tài liệu, sinh câu trả lời phù hợp với ngữ cảnh và thậm chí điều khiển giao diện.

Từ thực tế đó, đề tài “Xây dựng AI Agent giới thiệu Đại học Trà Vinh và các ngành đào tạo” được thực hiện nhằm xây dựng một hệ thống tương tác thông minh, có khả năng giới thiệu thông tin về Đại học Trà Vinh, hỗ trợ hỏi đáp về trường và ngành đào tạo, đồng thời kết hợp giao diện trực quan như tham quan ảo 360 độ, nhân vật hướng dẫn Three-Dimensional (3D), bản đồ khuôn viên và hệ thống quản trị nội dung.

1.1.2 Nhu cầu trải nghiệm tham quan ảo trong giáo dục

Bên cạnh nhu cầu tra cứu thông tin tuyển sinh và ngành đào tạo, người học ngày càng quan tâm đến việc trải nghiệm môi trường học tập trước khi lựa chọn trường. Đối

với các trường đại học, cơ sở vật chất, không gian học tập, thư viện, giảng đường, khu thực hành và các khu vực sinh hoạt sinh viên là những yếu tố quan trọng góp phần tạo nên ấn tượng ban đầu. Tuy nhiên, không phải thí sinh hoặc phụ huynh nào cũng có điều kiện đến trường trực tiếp để tham quan, đặc biệt là những người ở xa hoặc cần tìm hiểu thông tin trong thời gian ngắn.

Trải nghiệm tham quan ảo giúp giải quyết phần nào hạn chế này bằng cách cung cấp môi trường quan sát trực quan thông qua hình ảnh 360 độ, bản đồ tương tác, hình ảnh, video và các điểm thông tin gắn với từng vị trí cụ thể. Thay vì chỉ đọc mô tả bằng văn bản, người dùng có thể chủ động quan sát không gian, chuyển đổi giữa các địa điểm và hình dung rõ hơn về khuôn viên trường. Điều này giúp quá trình tìm hiểu thông tin trở nên sinh động, dễ tiếp cận và gần gũi với trải nghiệm thực tế hơn.

Trong lĩnh vực giáo dục, tham quan ảo không chỉ hỗ trợ truyền thông tuyển sinh mà còn có thể được sử dụng để giới thiệu cơ sở vật chất, hướng dẫn sinh viên mới làm quen với khuôn viên, hỗ trợ khách tham quan và cung cấp thông tin theo từng ngữ cảnh địa điểm. Khi kết hợp với AI Agent, hệ thống không chỉ hiển thị không gian ảo mà còn có thể giải thích, tư vấn và trả lời câu hỏi của người dùng trong quá trình tham quan. Nhờ đó, trải nghiệm tham quan ảo trở thành một kênh tương tác thông minh, hỗ trợ người dùng tìm hiểu Đại học Trà Vinh và các ngành đào tạo trực quan và hiệu quả hơn.

1.1.3 Vai trò của trí tuệ nhân tạo trong tương tác và tư vấn thông tin

Trí tuệ nhân tạo đang ngày càng được ứng dụng rộng rãi trong các hệ thống hỗ trợ người dùng tra cứu, tư vấn và tương tác với thông tin. Đặc biệt, sự phát triển của các mô hình ngôn ngữ lớn cho phép hệ thống tiếp nhận câu hỏi bằng ngôn ngữ tự nhiên, phân tích ý định của người dùng và sinh câu trả lời phù hợp với ngữ cảnh. Nhờ đó, người dùng không cần phải tìm kiếm thủ công qua nhiều trang nội dung, mà có thể đặt câu hỏi trực tiếp theo cách giao tiếp quen thuộc.

Trong bài toán giới thiệu trường đại học và ngành đào tạo, trí tuệ nhân tạo có thể hỗ trợ trả lời các câu hỏi liên quan đến thông tin tổng quan về trường, chương trình đào tạo, tuyển sinh, học phí, học bổng, cơ sở vật chất và hoạt động học tập. Khi kết hợp với kỹ thuật truy xuất thông tin từ kho tài liệu, hệ thống có thể sử dụng nguồn dữ liệu được chuẩn bị để tạo ra câu trả lời phù hợp, hạn chế phụ thuộc vào tri thức sẵn có của mô hình ngôn ngữ. Điều này giúp nội dung tư vấn bám sát hơn với dữ liệu của nhà trường.

Bên cạnh khả năng hỏi đáp, AI Agent còn đóng vai trò điều phối trải nghiệm tương tác. Thông qua cơ chế gọi công cụ, AI Agent có thể mở bản đồ, gợi ý di chuyển, hiển thị hình ảnh hoặc video minh họa và hỗ trợ người dùng trong quá trình tham quan. Nhờ đó, hệ thống không chỉ trả lời bằng văn bản mà còn cung cấp trải nghiệm đa phương thức, kết hợp nội dung tư vấn, giọng nói, hình ảnh, bản đồ và nhân vật hướng dẫn 3D.

1.2 Giới thiệu bài toán xây dựng AI Agent giới thiệu Đại học Trà Vinh

1.2.1 Đặc điểm bài toán

Bài toán của đề tài là xây dựng một AI Agent có khả năng hỗ trợ người dùng tìm hiểu thông tin về Đại học Trà Vinh và các ngành đào tạo thông qua hình thức tương tác tự nhiên. Người dùng có thể đặt câu hỏi bằng văn bản hoặc giọng nói, hệ thống sẽ xử lý yêu cầu, truy xuất thông tin liên quan từ kho tri thức và phản hồi bằng nội dung phù hợp với ngữ cảnh. Mục tiêu của bài toán là giúp người dùng tiếp cận thông tin nhanh hơn, đồng thời giảm sự phụ thuộc vào cách tra cứu thủ công trên các nguồn thông tin rời rạc.

Điểm đặc trưng của bài toán là AI Agent không chỉ thực hiện hỏi đáp thông tin, mà còn được đặt trong môi trường tham quan trực quan. Người dùng có thể quan sát một số địa điểm bằng ảnh 360 độ, xem hình ảnh hoặc video minh họa, theo dõi bản đồ khuôn viên và tương tác với nhân vật hướng dẫn 3D. Ngoài ra, hệ thống còn hỗ trợ điều hướng người dùng đến các khu vực quan tâm, cung cấp thông tin theo ngữ cảnh của từng vị trí cụ thể trong khuôn viên trường. Nhờ đó, việc giới thiệu trường và ngành đào tạo không chỉ dừng lại ở văn bản, mà được kết hợp với trải nghiệm trực quan, sinh động hơn, góp phần nâng cao mức độ tương tác và khả năng ghi nhớ thông tin người dùng.

1.2.2 Người dùng mục tiêu

Người dùng mục tiêu của hệ thống bao gồm thí sinh, phụ huynh, sinh viên mới, khách tham quan và những người quan tâm đến Đại học Trà Vinh. Đây là nhóm người thường có nhu cầu tìm hiểu thông tin tổng quan về trường, các ngành đào tạo, điều kiện học tập, cơ sở vật chất, học phí, học bổng và môi trường sinh viên trước khi đưa ra quyết định học tập hoặc tham quan thực tế. Do đó, hệ thống cần được thiết kế theo hướng dễ sử dụng, hỗ trợ tra cứu nhanh và phù hợp với nhiều nhóm người dùng khác nhau.

Đối với thí sinh và phụ huynh, hệ thống đóng vai trò như một kênh hỗ trợ tư vấn ban đầu, giúp tiếp cận thông tin nhanh hơn thông qua hình thức hỏi đáp một cách thuận tiện và trực quan. Đối với sinh viên mới hoặc khách tham quan, hệ thống có thể hỗ trợ

làm quen với một số khu vực trong khuôn viên trường, xem thông tin gắn với từng địa điểm và định hướng di chuyển thông qua bản đồ tương tác một cách dễ dàng.

Bên cạnh nhóm người dùng cuối, hệ thống cũng hướng đến người quản trị nội dung. Người quản trị có thể cập nhật thông tin địa điểm, tài liệu, media, câu hỏi gợi ý và cấu hình nhân vật hướng dẫn nhằm đảm bảo nội dung giới thiệu luôn phù hợp với nhu cầu truyền thông và tư vấn của nhà trường.

1.2.3 Các chức năng chính cần xây dựng

Để giải quyết bài toán trên, hệ thống cần xây dựng nhóm chức năng cốt lõi xoay quanh AI Agent và trải nghiệm tương tác của người dùng. Trước hết, hệ thống cần có chức năng hỏi đáp thông minh, cho phép người dùng đặt câu hỏi về Đại học Trà Vinh và các ngành đào tạo. AI Agent sẽ tiếp nhận câu hỏi, truy xuất thông tin từ kho tri thức và sinh câu trả lời phù hợp với ngữ cảnh.

Bên cạnh đó, hệ thống cần hỗ trợ trải nghiệm tham quan trực quan thông qua ảnh 360 độ, bản đồ khuôn viên, hình ảnh, video và nhân vật hướng dẫn 3D. Các thành phần này giúp người dùng không chỉ đọc thông tin mà còn có thể quan sát không gian, xem minh họa và di chuyển giữa các địa điểm liên quan trong quá trình tương tác.

Hệ thống cũng cần có chức năng điều hướng và hỗ trợ hành động thông qua AI Agent. Khi người dùng yêu cầu đến một địa điểm, xem bản đồ hoặc xem hình ảnh/video minh họa, AI Agent có thể gọi các công cụ tương ứng để điều khiển giao diện. Đây là điểm khác biệt so với chatbot thông thường, vì hệ thống có khả năng kết hợp giữa tư vấn nội dung và thao tác trực quan trên giao diện.

Cuối cùng, hệ thống cần có khu vực quản trị để cập nhật và vận hành nội dung. Người quản trị có thể quản lý địa điểm, tài liệu phục vụ Retrieval-Augmented Generation (RAG), hình ảnh, video, nhân vật hướng dẫn và các cấu hình liên quan. Chức năng này giúp hệ thống có khả năng mở rộng, cập nhật dữ liệu và duy trì chất lượng thông tin trong quá trình sử dụng.

1.3 Khảo sát các giải pháp liên quan

1.3.1 Google Street View / Google Earth

Google Street View và Google Earth là các công cụ phổ biến giúp người dùng quan sát vị trí địa lý, tuyến đường di chuyển và một phần không gian bên ngoài của các địa điểm. Đối với trường đại học, các công cụ này có thể hỗ trợ người dùng xác định vị trí trường, xem đường đến trường và hình dung sơ bộ khu vực xung quanh.

Tuy nhiên, nhu cầu tham quan từ xa không chỉ dừng lại ở việc xác định đường đi hoặc quan sát bên ngoài. Người dùng còn có nhu cầu tìm hiểu các không gian như giảng đường, thư viện, khu thực hành và khu vực sinh hoạt của sinh viên. Những nội dung này thường chưa được Google Street View và Google Earth thể hiện đầy đủ do giới hạn về phạm vi ghi hình và mức độ cập nhật dữ liệu. Do đó, các công cụ trên phù hợp với mục đích định vị và quan sát tổng quan, nhưng chưa đáp ứng trọn vẹn nhu cầu tham quan trường đại học từ xa.

1.3.2 Hệ thống virtual tour của các trường đại học

Một số trường đại học đã triển khai hệ thống virtual tour nhằm hỗ trợ người dùng tham quan khuôn viên từ xa. Các hệ thống này thường sử dụng ảnh 360 độ, điểm chuyển cảnh và các điểm thông tin trên giao diện để giúp người dùng quan sát không gian học tập, cơ sở vật chất và một số khu vực chức năng trong trường. So với hình ảnh tĩnh hoặc bài viết giới thiệu, virtual tour mang lại trải nghiệm trực quan và sinh động hơn.

Tuy nhiên, nhiều hệ thống virtual tour hiện nay chủ yếu tập trung vào việc trình diễn không gian. Người dùng có thể di chuyển giữa các điểm nhìn và xem một số thông tin được gắn sẵn, nhưng mức độ tương tác thường bị giới hạn trong phạm vi đã thiết kế trước. Khi người dùng muốn đặt câu hỏi cụ thể về ngành đào tạo, tuyển sinh, học phí, học bổng hoặc hỏi theo ngữ cảnh của địa điểm đang xem, họ vẫn phải phụ thuộc vào các kênh thông tin khác như website, bài viết hoặc tư vấn trực tiếp.

Bên cạnh đó, một số hệ thống virtual tour được triển khai bằng các nền tảng dựng tour có sẵn. Cách tiếp cận này phù hợp để xây dựng nhanh trải nghiệm tham quan 360 độ, nhưng chưa thật sự thuận lợi khi cần mở rộng sang các chức năng tương tác thông minh như hỏi đáp bằng AI Agent, truy xuất kho tri thức hoặc điều khiển giao diện theo ngữ cảnh. Vì vậy, virtual tour là một hướng tiếp cận hữu ích cho nhu cầu quan sát, nhưng vẫn còn khoảng trống nếu muốn kết hợp tham quan với tư vấn thông tin thông minh.

1.3.3 Chatbot tư vấn tuyển sinh và trợ lý AI

Chatbot tư vấn tuyển sinh và trợ lý Artificial Intelligence (AI) là các giải pháp được sử dụng để hỗ trợ người dùng tra cứu thông tin nhanh hơn thông qua hình thức hỏi đáp. Thay vì tự tìm kiếm trên nhiều trang nội dung, người dùng có thể đặt câu hỏi trực tiếp về ngành học, phương thức tuyển sinh, học phí, học bổng hoặc các thông tin thường gặp. Cách tiếp cận này giúp quá trình tiếp cận thông tin trở nên thuận tiện hơn, đặc biệt đối với các câu hỏi phổ biến và có cấu trúc lặp lại.

Tuy nhiên, nhiều chatbot tư vấn hiện nay chủ yếu hoạt động ở dạng văn bản hoặc theo các kịch bản trả lời được thiết kế sẵn. Điều này có thể đáp ứng tốt các câu hỏi đơn giản, nhưng còn hạn chế khi người dùng đặt câu hỏi theo ngữ cảnh cụ thể hoặc cần kết hợp nhiều loại thông tin khác nhau. Bên cạnh đó, chatbot truyền thống thường chưa gắn với trải nghiệm không gian của trường, nên khó hỗ trợ người dùng khi họ muốn vừa hỏi thông tin, vừa xem địa điểm liên quan, hình ảnh minh họa hoặc bản đồ khuôn viên.

Sự xuất hiện của các mô hình ngôn ngữ lớn giúp trợ lý AI có khả năng hiểu câu hỏi tự nhiên và phản hồi linh hoạt hơn so với chatbot kịch bản. Tuy vậy, nếu không được kết hợp với kho dữ liệu đáng tin cậy và các công cụ điều khiển giao diện phù hợp, trợ lý AI vẫn có thể gặp hạn chế về độ chính xác, phạm vi thông tin và khả năng hỗ trợ trải nghiệm tham quan. Vì vậy, cần có hướng tiếp cận kết hợp giữa AI Agent, kho tri thức của nhà trường và giao diện trực quan để phục vụ tốt hơn cho bài toán giới thiệu Đại học Trà Vinh và các ngành đào tạo.

1.3.4 So sánh và rút ra hướng tiếp cận

Các giải pháp đã khảo sát đáp ứng những nhu cầu khác nhau trong quá trình tìm hiểu trường đại học. Công cụ bản đồ hỗ trợ định vị, hệ thống tham quan ảo tăng tính trực quan, còn chatbot tạo thuận lợi cho việc hỏi đáp. Tuy nhiên, mỗi giải pháp vẫn có những giới hạn riêng khi được sử dụng độc lập. Sự khác biệt giữa các nhóm giải pháp được tổng hợp trong Bảng 1.1.

Bảng 1.1. So sánh các nhóm giải pháp liên quan

Nhóm giải pháp	Ưu điểm	Hạn chế
Google Street View / Google Earth	Hỗ trợ định vị, xem tuyến đường và không gian đường đi bên ngoài.	Chưa hỗ trợ tham quan chi tiết bên trong khuôn viên trường.
Website, bài viết và tài liệu giới thiệu	Cung cấp thông tin chính thức về trường, tuyển sinh và ngành đào tạo.	Thông tin có thể phân tán, người dùng phải tự chủ động tìm kiếm.
Hệ thống virtual tour	Trực quan, hỗ trợ quan sát không gian trường từ xa.	Chủ yếu trình diễn không gian, hạn chế về tư vấn thông tin linh hoạt.
Chatbot tư vấn tuyển sinh / trợ lý AI	Hỗ trợ hỏi đáp nhanh cho các câu hỏi thường gặp.	Thường thiếu trải nghiệm trực quan như ảnh 360 độ, bản đồ và media.
Hướng tiếp cận của đề tài	Kết hợp AI Agent, kho tri thức RAG và giao diện tham quan trực quan.	Phạm vi ban đầu phụ thuộc vào dữ liệu, hình ảnh và nội dung thu thập.

Từ Bảng 1.1 có thể thấy hạn chế chung của các giải pháp hiện có là sự tách rời giữa nội dung tư vấn và trải nghiệm tham quan. Người dùng thường phải chuyển đổi giữa nhiều công cụ để vừa tìm hiểu thông tin, vừa quan sát địa điểm và xác định đường đi. Đây là khoảng trống mà đề tài hướng đến giải quyết. Vì vậy, hệ thống được xây dựng theo hướng kết hợp AI Agent, kho tri thức RAG và giao diện tham quan trực quan.

1.4 Mục tiêu, phạm vi và kết quả mong đợi

1.4.1 Mục tiêu tổng quát

Mục tiêu tổng quát của đề tài là xây dựng một hệ thống AI Agent hỗ trợ giới thiệu Đại học Trà Vinh và các ngành đào tạo thông qua hình thức tương tác tự nhiên, trực quan và dễ tiếp cận. Hệ thống cho phép người dùng tra cứu thông tin về trường, tuyển sinh, ngành đào tạo, cơ sở vật chất và môi trường học tập bằng cách đặt câu hỏi trực tiếp, đồng thời kết hợp các thành phần tham quan ảo như ảnh 360 độ, bản đồ khuôn viên, media và nhân vật hướng dẫn 3D.

Thông qua việc kết hợp AI Agent, kho tri thức RAG và giao diện tham quan ảo, đề tài hướng đến xây dựng một kênh giới thiệu có tính tương tác cao. Hệ thống không chỉ hỗ trợ trả lời câu hỏi mà còn có thể thực hiện các hành động như mở bản đồ, hiển thị media và gợi ý di chuyển đến địa điểm liên quan trong quá trình tham quan.

1.4.2 Mục tiêu cụ thể

Từ mục tiêu tổng quát trên, đề tài xác định các mục tiêu cụ thể như sau:

- Xây dựng kho tri thức phục vụ hỏi đáp từ các tài liệu liên quan đến Đại học Trà Vinh, thông tin tuyển sinh, ngành đào tạo và các nội dung giới thiệu nhà trường.
- Xây dựng AI Agent có khả năng tiếp nhận câu hỏi bằng ngôn ngữ tự nhiên, truy xuất thông tin từ kho tri thức và sinh câu trả lời phù hợp với ngữ cảnh người dùng.
- Tích hợp cơ chế Function Calling để AI Agent có thể thực hiện một số hành động trên giao diện như mở bản đồ, hiển thị hình ảnh, video hoặc hỗ trợ điều hướng đến địa điểm liên quan.
- Xây dựng giao diện tham quan trực quan gồm ảnh 360 độ, bản đồ khuôn viên, hình ảnh, video và nhân vật hướng dẫn 3D nhằm hỗ trợ quá trình giới thiệu thông tin sinh động hơn.
- Tích hợp chức năng tương tác bằng giọng nói thông qua chuyển giọng nói thành văn bản và chuyển văn bản thành giọng nói.

- Xây dựng giao diện quản trị để cập nhật địa điểm, tài liệu, hình ảnh, video, nhân vật hướng dẫn và các cấu hình liên quan.

- Kiểm thử và đánh giá mức độ đáp ứng của hệ thống thông qua các kịch bản sử dụng thực tế như hỏi đáp thông tin, tham quan địa điểm, điều hướng trên bản đồ, hiển thị nội dung trực quan và quản trị dữ liệu.

1.4.3 Phạm vi thực hiện

Phạm vi của đề tài tập trung vào việc xây dựng hệ thống AI Agent giới thiệu tổng quan về Đại học Trà Vinh và các ngành đào tạo trên nền tảng web. Hệ thống gồm hai nhóm giao diện chính, bao gồm giao diện người dùng phục vụ hỏi đáp, tìm kiếm thông tin và tham quan trực quan, cùng với giao diện quản trị phục vụ cập nhật nội dung, tài liệu, địa điểm, hình ảnh, video và các cấu hình liên quan.

Về nội dung dữ liệu, đề tài tập trung vào các thông tin liên quan đến Đại học Trà Vinh như giới thiệu trường, lịch sử phát triển, cơ sở vật chất, chính sách tuyển sinh, học phí, học bổng, ngành đào tạo và một số địa điểm tiêu biểu trong khuôn viên. Các tài liệu này được xử lý nhằm phục vụ cơ chế truy xuất thông tin và sinh câu trả lời cho AI Agent.

Về chức năng, hệ thống hỗ trợ hỏi đáp bằng AI, tham quan một số địa điểm bằng ảnh 360 độ, xem hình ảnh hoặc video minh họa, điều hướng trên bản đồ khuôn viên, tương tác với nhân vật hướng dẫn 3D và quản trị nội dung. Phạm vi triển khai ban đầu chưa bao phủ toàn bộ khuôn viên trường hoặc thay thế các kênh tư vấn tuyển sinh chính thức, mà tập trung xây dựng mô hình hệ thống có khả năng mở rộng trong tương lai.

1.4.4 Kết quả dự kiến

Sau khi hoàn thành, đề tài dự kiến xây dựng được một hệ thống web hỗ trợ giới thiệu Đại học Trà Vinh và các ngành đào tạo thông qua AI Agent. Người dùng có thể đặt câu hỏi bằng ngôn ngữ tự nhiên, nhận câu trả lời dựa trên kho tri thức và tương tác với các thành phần tham quan trực quan.

Hệ thống cung cấp trải nghiệm tham quan tại một số địa điểm tiêu biểu thông qua ảnh 360 độ, bản đồ khuôn viên, hình ảnh, video và nhân vật hướng dẫn 3D, giúp người dùng có cái nhìn trực quan hơn về môi trường học tập. Giao diện quản trị linh hoạt, cho phép cập nhật và quản lý nội dung thuận tiện, hỗ trợ hệ thống duy trì và mở rộng trong quá trình sử dụng. Kết quả của đề tài có thể làm cơ sở phát triển các hệ thống giới thiệu trường học thông minh, kết hợp tư vấn thông tin, tham quan trực quan và tương tác đa phương thức trong tương lai gần.

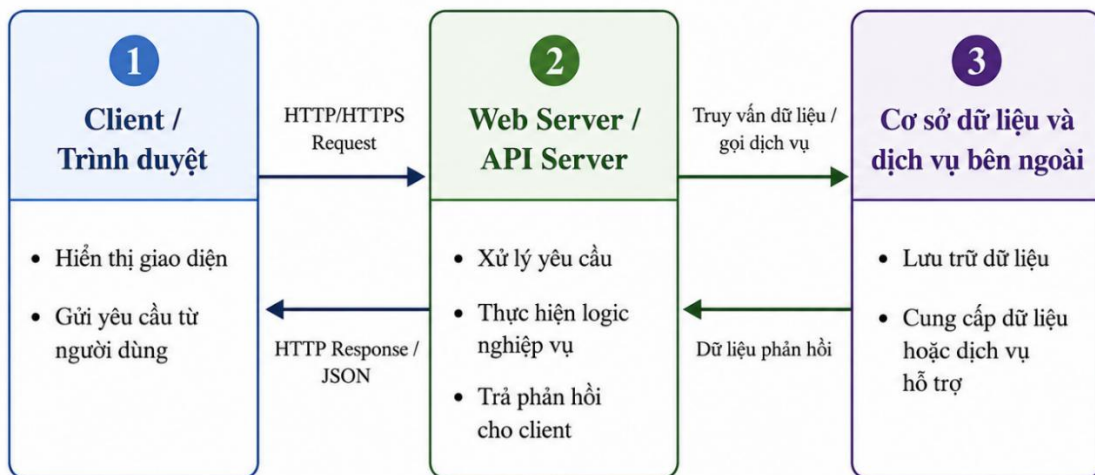
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

Chương này trình bày các cơ sở lý thuyết và công nghệ được sử dụng trong quá trình xây dựng hệ thống AI Agent giới thiệu Đại học Trà Vinh. Nội dung chương tập trung làm rõ kiến trúc ứng dụng web, các công nghệ frontend, backend, trí tuệ nhân tạo, xử lý giọng nói, ảnh 360 độ và thuật toán tìm đường A^* . Bên cạnh việc giới thiệu khái niệm, chương còn phân tích vai trò của từng công nghệ trong hệ thống nhằm làm cơ sở cho quá trình phân tích, thiết kế và triển khai ở các chương sau.

2.1 Kiến trúc ứng dụng web hiện đại

Ứng dụng web hiện đại thường được xây dựng theo mô hình tách biệt giữa frontend và backend. Trong đó, frontend đảm nhiệm việc hiển thị nội dung, tiếp nhận thao tác của người dùng và gửi yêu cầu đến backend thông qua Application Programming Interface (API). Backend chịu trách nhiệm xử lý nghiệp vụ, truy xuất dữ liệu, tích hợp dịch vụ bên ngoài và trả kết quả về giao diện.

Trong đề tài này, hệ thống gồm các thành phần chính như frontend, backend API, cơ sở dữ liệu, hệ thống lưu trữ và dịch vụ AI. Các thành phần này phối hợp với nhau để phục vụ các chức năng chính như hỏi đáp bằng AI, hiển thị ảnh 360 độ, bản đồ, media và nhân vật hướng dẫn 3D. Kiến trúc tổng quan và luồng xử lý chính giữa các thành phần của hệ thống được minh họa ở Hình 2.1.



Hình 2.1. Kiến trúc tổng quan ứng dụng web và luồng xử lý trong hệ thống

Từ Hình 2.1 có thể thấy backend API đóng vai trò trung gian giữa giao diện người dùng và các thành phần phía sau như cơ sở dữ liệu, hệ thống lưu trữ và dịch vụ AI. Kết quả xử lý được trả về frontend để hiển thị dưới dạng câu trả lời, ảnh 360 độ, media, bản đồ hoặc tuyến đường tương ứng.

2.1.1 Mô hình Client-Server

Mô hình Client-Server là mô hình phổ biến trong phát triển ứng dụng web, trong đó client chịu trách nhiệm tương tác với người dùng, còn server đảm nhiệm việc xử lý dữ liệu và nghiệp vụ. Client thường là trình duyệt web, gửi yêu cầu đến server thông qua HyperText Transfer Protocol (HTTP) hoặc HyperText Transfer Protocol Secure (HTTPS). Server tiếp nhận yêu cầu, truy xuất dữ liệu cần thiết, xử lý nghiệp vụ và trả kết quả về cho client.

Đối với hệ thống AI Agent giới thiệu Đại học Trà Vinh, mô hình Client-Server giúp phân tách rõ phần giao diện tương tác và phần xử lý phía máy chủ. Frontend tập trung vào các chức năng như trò chuyện, tham quan ảo, bản đồ và media, còn backend xử lý câu hỏi, truy xuất kho tri thức, tìm đường, quản lý dữ liệu và kết nối với các dịch vụ AI. Cách tổ chức này giúp hệ thống dễ bảo trì, đồng thời thuận lợi khi mở rộng thêm các giao diện như trang quản trị, kiosk hoặc thiết bị di động.

2.1.2 RESTful API

Representational State Transfer (REST) là một phong cách kiến trúc được sử dụng rộng rãi trong thiết kế hệ thống web, trong đó tài nguyên của hệ thống được truy cập thông qua các định danh và thao tác chuẩn của HTTP như GET, POST, PUT, PATCH và DELETE [1]. Trong thực tế phát triển ứng dụng, RESTful API thường được dùng để tổ chức các endpoint phục vụ trao đổi dữ liệu giữa client và server.

Trong hệ thống đề tài, RESTful API được sử dụng để giao tiếp giữa frontend và backend. Các chức năng như quản lý địa điểm, tài liệu, media, hội thoại AI và dẫn đường được tổ chức dưới dạng các endpoint API. Dữ liệu trao đổi chủ yếu ở dạng JavaScript Object Notation (JSON), phù hợp với môi trường web và dễ xử lý ở cả hai phía.

2.1.3 Server-Sent Events (SSE)

Bên cạnh các API trả về dữ liệu một lần, một số chức năng trong ứng dụng web cần phản hồi theo dòng để người dùng nhận được kết quả từng phần trong quá trình xử lý. Server-Sent Events (SSE) là một cơ chế cho phép server gửi dữ liệu liên tục về client thông qua một kết nối HTTP một chiều. Theo tài liệu MDN, SSE phù hợp với các trường hợp server cần đẩy sự kiện hoặc dữ liệu cập nhật về trình duyệt, trong khi client không cần gửi dữ liệu ngược lại trên cùng kết nối [2].

Trong đề tài này, SSE được sử dụng cho chế độ trả lời dạng streaming của AI Agent. Khi người dùng gửi câu hỏi, backend có thể gửi từng phần nội dung trả lời về

frontend thay vì chờ toàn bộ quá trình xử lý hoàn tất. Cách này giúp cải thiện cảm nhận về tốc độ phản hồi và tạo trải nghiệm trò chuyện tự nhiên hơn.

2.1.4 Kiến trúc phân lớp Router – Service – Repository

Để tổ chức backend rõ ràng và dễ bảo trì, hệ thống sử dụng kiến trúc phân lớp Router - Service - Repository. Tầng Router tiếp nhận yêu cầu HTTP và gọi đến các service tương ứng. Tầng Service chứa logic nghiệp vụ như xử lý câu hỏi bằng RAG, gọi mô hình AI, quản lý media, xử lý âm thanh hoặc tính toán đường đi. Tầng Repository chịu trách nhiệm làm việc trực tiếp với cơ sở dữ liệu.

Cách tổ chức này giúp tách biệt giữa xử lý API, xử lý nghiệp vụ và truy xuất dữ liệu. Nhờ đó, mã nguồn backend dễ bảo trì hơn, đồng thời thuận lợi khi mở rộng các chức năng liên quan đến AI Agent, kho tri thức, dữ liệu địa điểm, media, bản đồ và hệ thống quản trị nội dung.

2.2 Công nghệ Frontend

Frontend là phần giao diện trực tiếp tương tác với người dùng, chịu trách nhiệm hiển thị nội dung, tiếp nhận thao tác và gửi yêu cầu đến backend. Trong đề tài này, frontend tích hợp các thành phần như AI Agent, ảnh 360 độ, bản đồ khuôn viên, media và nhân vật hướng dẫn 3D.

Các công nghệ chính được sử dụng gồm Next.js, React, TypeScript, Zustand, Framer Motion, Pannellum, Three.js và React Three Fiber. Những công nghệ này hỗ trợ xây dựng giao diện theo hướng component hóa, quản lý trạng thái và hiển thị các thành phần trực quan trên trình duyệt.

2.2.1 Next.js, React và TypeScript

React là thư viện JavaScript dùng để xây dựng giao diện người dùng theo hướng thành phần, cho phép chia giao diện thành các component độc lập, dễ quản lý và có thể tái sử dụng [3]. Cách tiếp cận này phù hợp với các ứng dụng web có nhiều phần giao diện khác nhau, vì mỗi thành phần như khung trò chuyện, bản đồ, trình xem ảnh 360 độ hoặc khu vực hiển thị media có thể được xây dựng và quản lý riêng.

Next.js là framework phát triển ứng dụng web dựa trên React, hỗ trợ tổ chức route, tối ưu quá trình build và triển khai ứng dụng web hiện đại [4]. Trong đề tài này, Next.js được sử dụng để xây dựng giao diện người dùng và các trang quản trị theo cấu trúc rõ ràng, giúp việc phát triển, bảo trì và mở rộng hệ thống thuận lợi hơn.

TypeScript bổ sung hệ thống kiểu dữ liệu tĩnh cho JavaScript, giúp mô tả rõ cấu trúc dữ liệu và giảm lỗi trong quá trình phát triển [5]. Đối với hệ thống có nhiều dữ liệu như địa điểm, media, tài liệu, hội thoại và phản hồi từ AI Agent, TypeScript giúp mã nguồn frontend dễ kiểm soát hơn, đặc biệt khi trao đổi dữ liệu với backend API.

Trong đề tài, ba công nghệ này được kết hợp để xây dựng giao diện tham quan và các trang quản trị nội dung. Việc tổ chức giao diện thành các component riêng giúp hệ thống dễ mở rộng, đồng thời thuận lợi khi cần bổ sung chức năng hoặc điều chỉnh từng thành phần trong quá trình phát triển.

2.2.2 Zustand trong quản lý trạng thái

Trong ứng dụng web có nhiều thành phần tương tác, quản lý trạng thái đóng vai trò quan trọng. Trạng thái có thể bao gồm địa điểm hiện tại, danh sách địa điểm, trạng thái bản đồ, ảnh 360 độ, nội dung trò chuyện, phiên chat hoặc trạng thái bật/tắt giọng nói. Nếu không có cơ chế quản lý phù hợp, các component dễ bị phụ thuộc lẫn nhau và khó kiểm soát khi hệ thống mở rộng.

Zustand là thư viện quản lý trạng thái cho React, cho phép tạo các store để chia sẻ trạng thái giữa nhiều component [6]. Trong đề tài, Zustand được sử dụng để quản lý các trạng thái chính của giao diện tham quan và trò chuyện. Nhờ đó, thao tác của người dùng hoặc phản hồi của AI Agent có thể được phản ánh đồng bộ lên nhiều thành phần như bản đồ, panorama, khung chat và nhân vật hướng dẫn 3D.

2.2.3 Framer Motion trong xử lý hiệu ứng giao diện

Framer Motion là thư viện hỗ trợ xây dựng hiệu ứng chuyển động cho React, cho phép định nghĩa các animation khi component xuất hiện, biến mất hoặc thay đổi trạng thái trên giao diện. Thư viện này cũng hỗ trợ các trạng thái tương tác như di chuột, nhấn chọn và chuyển đổi giữa các thành phần giao diện [7].

Trong hệ thống của đề tài, Framer Motion được sử dụng để xử lý các hiệu ứng như chuyển cảnh, hiển thị khung thông tin, khung chat, bản đồ nhỏ, thư viện ảnh và trình phát video. Mặc dù các hiệu ứng này không trực tiếp xử lý nghiệp vụ chính của hệ thống, chúng giúp giao diện phản hồi mượt hơn, đồng thời hỗ trợ người dùng dễ theo dõi sự thay đổi nội dung trong quá trình tương tác.

2.2.4 Pannellum và hiển thị ảnh panorama 360 độ

Ảnh panorama 360 độ là thành phần quan trọng trong trải nghiệm tham quan trực quan, giúp người dùng quan sát không gian xung quanh tại một địa điểm thay vì chỉ xem ảnh tĩnh theo một góc cố định. Thông qua thao tác xoay góc nhìn, phóng to hoặc thu nhỏ, người dùng có thể hình dung rõ hơn về không gian thực tế trong khuôn viên.

Pannellum là thư viện hiển thị panorama trên trình duyệt, hỗ trợ ảnh dạng equirectangular và các thao tác tương tác trong không gian 360 độ [8]. Trong hệ thống của đề tài, Pannellum được tích hợp vào frontend thông qua một component riêng để hiển thị ảnh 360 độ của từng địa điểm. Khi người dùng chuyển sang địa điểm khác, component sẽ tải ảnh tương ứng và cập nhật lại khung nhìn, giúp quá trình tham quan diễn ra liền mạch hơn.

2.2.5 Three.js / React Three Fiber cho nhân vật hướng dẫn 3D

Hệ thống sử dụng nhân vật hướng dẫn 3D để tăng tính trực quan trong quá trình tương tác. Nhân vật này đóng vai trò như một thành phần giao diện hỗ trợ, thể hiện các trạng thái như chờ, suy nghĩ hoặc đang nói, từ đó giúp trải nghiệm hỏi đáp với AI Agent trực quan và tự nhiên hơn.

Three.js là thư viện JavaScript hỗ trợ xây dựng và hiển thị đồ họa 3D trên trình duyệt thông qua WebGL [9]. React Three Fiber giúp sử dụng Three.js trong môi trường React, cho phép mô tả cảnh 3D theo cách gần với component React [10]. Trong đề tài, hai công nghệ này được dùng để hiển thị nhân vật hướng dẫn 3D và điều khiển animation theo trạng thái tương tác của hệ thống, chẳng hạn chuyển sang trạng thái suy nghĩ khi AI Agent xử lý câu hỏi hoặc trạng thái nói khi phát âm thanh trả lời.

2.3 Công nghệ Backend

Backend là phần đảm nhiệm xử lý nghiệp vụ, cung cấp API cho frontend, quản lý dữ liệu và tích hợp với các dịch vụ bên ngoài. Trong hệ thống của đề tài, backend không chỉ phục vụ các yêu cầu thông thường như lấy danh sách địa điểm, quản lý tài liệu hoặc media, mà còn điều phối các chức năng liên quan đến AI Agent, truy xuất kho tri thức, tạo phản hồi, xử lý âm thanh và điều hướng bản đồ.

Các công nghệ chính được sử dụng ở phía backend gồm FastAPI, SQLAlchemy, Supabase PostgreSQL, pgvector, Cloudflare R2 và Supabase Auth. Những công nghệ này hỗ trợ xây dựng API, truy cập cơ sở dữ liệu, tìm kiếm vector, lưu trữ tệp và bảo vệ các chức năng quản trị của hệ thống.

2.3.1 FastAPI

FastAPI là framework dùng để xây dựng API với Python, hỗ trợ khai báo endpoint, xử lý request/response, dependency injection và tạo tài liệu API tự động [11]. Với cú pháp rõ ràng và khả năng hỗ trợ bất đồng bộ, FastAPI phù hợp cho các backend cần xử lý nhiều nhóm chức năng và tích hợp với các dịch vụ bên ngoài.

Trong đề tài, FastAPI được sử dụng làm nền tảng chính cho backend. Các chức năng như địa điểm, trò chuyện với AI Agent, điều hướng bản đồ, quản trị nội dung, quản lý media và cache được tổ chức thành các router riêng. Cách tổ chức này giúp API có cấu trúc rõ ràng, dễ mở rộng và thuận lợi trong quá trình kiểm thử.

FastAPI cũng hỗ trợ tốt nhiều kiểu phản hồi khác nhau. Đối với các API thông thường, backend trả dữ liệu dạng JSON, còn đối với chức năng trò chuyện streaming, backend có thể trả dữ liệu theo luồng thông qua SSE. Nhờ đó, hệ thống có thể phục vụ cả các thao tác quản trị, truy vấn dữ liệu và tương tác thời gian thực với AI Agent.

2.3.2 SQLAlchemy ORM

SQLAlchemy là thư viện hỗ trợ làm việc với cơ sở dữ liệu quan hệ trong Python, cung cấp cơ chế Object-Relational Mapping (ORM) và các công cụ truy vấn Structured Query Language (SQL) [12]. ORM giúp ánh xạ dữ liệu trong bảng thành các đối tượng trong chương trình, giúp việc xử lý dữ liệu trong mã nguồn rõ ràng và dễ quản lý hơn.

Trong hệ thống của đề tài, SQLAlchemy được sử dụng để kết nối PostgreSQL và thực hiện các thao tác đọc, ghi, cập nhật, xóa dữ liệu. Các nhóm dữ liệu như địa điểm, tài liệu, media, phiên trò chuyện, nhân vật hướng dẫn 3D và thông tin điều hướng đều được backend truy cập thông qua các lớp xử lý tương ứng.

Việc sử dụng SQLAlchemy giúp tách phần truy cập dữ liệu khỏi logic nghiệp vụ. Nhờ đó, các service có thể tập trung vào xử lý chức năng, còn các thao tác với cơ sở dữ liệu được đóng gói riêng, phù hợp với kiến trúc phân lớp của backend.

2.3.3 Supabase PostgreSQL

PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở, được sử dụng rộng rãi trong các ứng dụng web nhờ khả năng lưu trữ dữ liệu có cấu trúc và hỗ trợ truy vấn mạnh. Supabase cung cấp nền tảng backend dựa trên PostgreSQL, hỗ trợ cơ sở dữ liệu, xác thực và một số dịch vụ liên quan trong quá trình phát triển ứng dụng [13].

Trong đề tài, Supabase PostgreSQL được sử dụng làm cơ sở dữ liệu chính của hệ thống. Các dữ liệu như thông tin địa điểm, tài liệu, đoạn văn bản đã xử lý, media, phiên

trò chuyện, cấu hình nhân vật hướng dẫn 3D và dữ liệu điều hướng được lưu trữ trong cơ sở dữ liệu. Việc sử dụng PostgreSQL giúp dữ liệu có cấu trúc rõ ràng và phù hợp với các quan hệ giữa nhiều nhóm thông tin trong hệ thống.

Bên cạnh việc lưu trữ dữ liệu thông thường, PostgreSQL còn đóng vai trò quan trọng trong kho tri thức của AI Agent. Các tài liệu sau khi được xử lý có thể được chia thành các đoạn nhỏ, lưu cùng embedding và metadata. Những dữ liệu này là cơ sở cho quá trình truy xuất thông tin khi người dùng đặt câu hỏi.

2.3.4 pgvector và lưu trữ vector embedding

pgvector là phần mở rộng cho PostgreSQL, cho phép lưu trữ vector và thực hiện tìm kiếm độ tương đồng vector ngay trong cơ sở dữ liệu [14]. Công nghệ này thường được sử dụng trong các hệ thống tìm kiếm ngữ nghĩa, hệ thống gợi ý và các ứng dụng Retrieval-Augmented Generation.

Trong đề tài, pgvector được sử dụng để phục vụ quá trình tìm kiếm các đoạn tài liệu liên quan đến câu hỏi của người dùng. Khi tài liệu được đưa vào hệ thống, nội dung được chia thành các đoạn nhỏ và tạo embedding. Khi người dùng đặt câu hỏi, câu hỏi cũng được chuyển thành embedding, sau đó hệ thống so sánh với các embedding đã lưu để tìm ra các đoạn nội dung có độ liên quan cao.

Cách tiếp cận này giúp AI Agent không chỉ dựa vào kiến thức tổng quát của mô hình ngôn ngữ, mà còn có thể truy xuất thông tin từ kho tài liệu của nhà trường. Đây là nền tảng quan trọng để hệ thống trả lời các câu hỏi về Đại học Trà Vinh và các ngành đào tạo dựa trên dữ liệu đã thu thập.

2.3.5 Cloudflare R2 Object Storage

Cloudflare R2 là dịch vụ lưu trữ đối tượng, hỗ trợ lưu trữ và truy xuất tệp thông qua cơ chế tương thích với S3 [15]. Hình thức lưu trữ này phù hợp với các loại dữ liệu dạng tệp như hình ảnh, video, tài liệu, mô hình 3D hoặc âm thanh, đặc biệt trong các hệ thống có nhiều tài nguyên media cần được quản lý riêng.

Trong hệ thống của đề tài, Cloudflare R2 được sử dụng để lưu trữ các tệp phục vụ trải nghiệm tham quan, bao gồm ảnh 360 độ, hình ảnh, video, tài liệu RAG, mô hình nhân vật hướng dẫn 3D và audio thuyết minh hoặc phản hồi bằng giọng nói. Việc tách các tệp này ra khỏi cơ sở dữ liệu giúp hệ thống dễ quản lý hơn, đồng thời tăng khả năng mở rộng và tối ưu hiệu suất truy xuất dữ liệu, trong đó cơ sở dữ liệu chỉ cần lưu metadata và đường dẫn tệp, còn nội dung tệp được lưu trên hệ thống lưu trữ đối tượng.

2.3.6 Supabase Auth cho xác thực quản trị

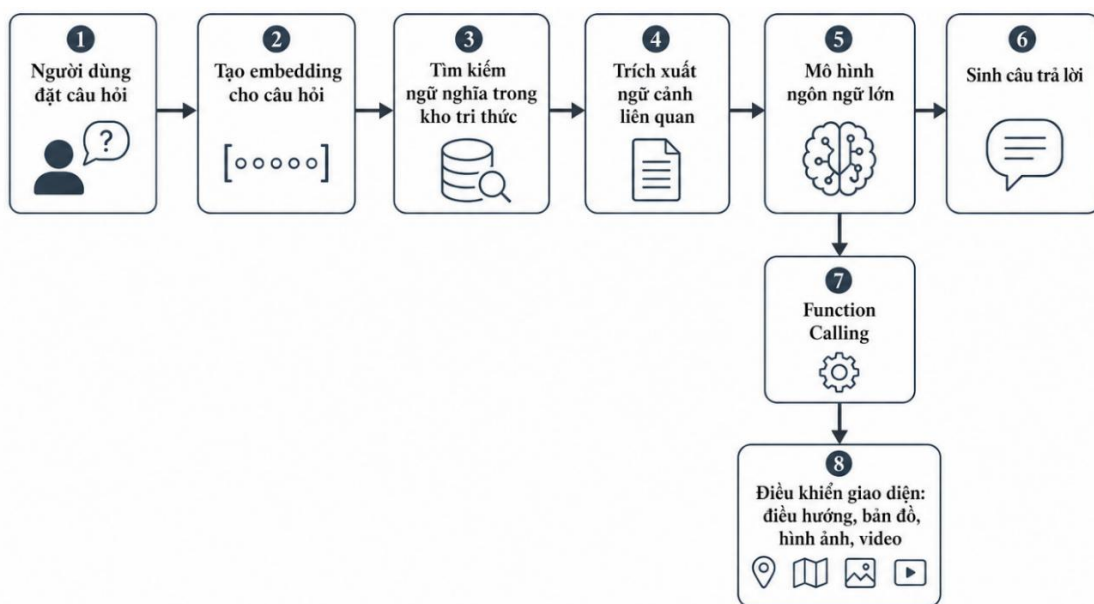
Đối với các chức năng quản trị, hệ thống cần cơ chế xác thực để hạn chế quyền truy cập trái phép. Supabase Auth cung cấp dịch vụ xác thực người dùng, hỗ trợ quản lý phiên đăng nhập và kiểm tra token khi truy cập các tài nguyên được bảo vệ [16].

Trong đề tài, Supabase Auth được sử dụng để bảo vệ các API thuộc nhóm quản trị. Khi người dùng quản trị gửi yêu cầu đến backend, token xác thực được gửi kèm trong header của request và được kiểm tra trước khi cho phép thực hiện các thao tác như quản lý địa điểm, tài liệu, media, nhân vật hướng dẫn hoặc cache. Cơ chế này giúp phân tách rõ giữa giao diện người dùng công khai và giao diện quản trị, đảm bảo các thao tác thay đổi dữ liệu hệ thống chỉ dành cho tài khoản có quyền truy cập phù hợp.

2.4 Trí tuệ nhân tạo và xử lý ngôn ngữ tự nhiên

Trí tuệ nhân tạo, đặc biệt là các mô hình ngôn ngữ lớn, được ứng dụng rộng rãi trong các hệ thống hỏi đáp, trợ lý ảo và tư vấn thông tin hiện nay. Các mô hình này có khả năng tiếp nhận câu hỏi bằng ngôn ngữ tự nhiên, phân tích ngữ cảnh và sinh câu trả lời phù hợp với yêu cầu của người dùng. Trong đề tài này, trí tuệ nhân tạo giữ vai trò trung tâm trong việc xây dựng AI Agent hỗ trợ người dùng tìm hiểu thông tin về Đại học Trà Vinh và các ngành đào tạo một cách linh hoạt hơn.

Tuy nhiên, nếu chỉ dựa vào mô hình ngôn ngữ, hệ thống gặp hạn chế khi trả lời thông tin cụ thể theo tài liệu nhà trường. Vì vậy, đề tài kết hợp mô hình ngôn ngữ lớn với cơ chế truy xuất tri thức, embedding, tìm kiếm ngữ nghĩa và function calling. Quy trình tổng quan của AI Agent kết hợp RAG trong hệ thống được minh họa ở Hình 2.2.



Hình 2.2. Quy trình tổng quan AI Agent kết hợp RAG trong hệ thống

Từ Hình 2.2 có thể thấy câu hỏi của người dùng không được gửi trực tiếp đến mô hình ngôn ngữ để trả lời ngay, mà trước hết được xử lý và truy xuất các đoạn tài liệu liên quan từ kho tri thức. Các thông tin này sau đó được kết hợp với câu hỏi và ngữ cảnh hội thoại để làm đầu vào cho mô hình AI sinh câu trả lời. Cách tiếp cận này giúp AI Agent vừa có khả năng giao tiếp tự nhiên, vừa có thể khai thác dữ liệu riêng của hệ thống và thực hiện một số hành động giao diện khi cần thiết.

2.4.1 Mô hình ngôn ngữ lớn Google Gemini

Mô hình ngôn ngữ lớn là các mô hình trí tuệ nhân tạo được huấn luyện trên lượng dữ liệu lớn nhằm xử lý và sinh ngôn ngữ tự nhiên. Các mô hình này có thể thực hiện nhiều tác vụ như trả lời câu hỏi, tóm tắt văn bản, phân tích ý định người dùng, sinh nội dung và hỗ trợ hội thoại. Trong hệ thống hỏi đáp, mô hình ngôn ngữ lớn đóng vai trò tiếp nhận câu hỏi từ người dùng và sinh phản hồi dưới dạng văn bản tự nhiên.

Gemini là nhóm mô hình AI của Google, cung cấp các khả năng xử lý ngôn ngữ, sinh nội dung và tích hợp với các công cụ trong ứng dụng [17]. Trong đề tài, Gemini được sử dụng để sinh câu trả lời cho AI Agent dựa trên câu hỏi của người dùng, lịch sử hội thoại và ngữ cảnh được truy xuất từ kho tri thức.

Việc sử dụng mô hình ngôn ngữ lớn giúp hệ thống có khả năng giao tiếp linh hoạt hơn so với các chatbot kịch bản truyền thống. Người dùng không cần chọn câu hỏi cố định, mà có thể đặt câu hỏi bằng ngôn ngữ tự nhiên về trường, ngành đào tạo, địa điểm hoặc các nội dung liên quan. Tuy nhiên, để câu trả lời gắn với dữ liệu của đề tài, mô hình cần được bổ sung ngữ cảnh từ kho tài liệu thông qua cơ chế RAG.

2.4.2 Vector Embedding và tìm kiếm ngữ nghĩa

Embedding là kỹ thuật biểu diễn văn bản dưới dạng vector số, giúp hệ thống xác định mức độ tương đồng giữa câu hỏi của người dùng và các đoạn tài liệu đã lưu trữ. Đây là nền tảng của tìm kiếm ngữ nghĩa trong các hệ thống RAG.

Trong đề tài, tài liệu sau khi được xử lý sẽ được chia thành các đoạn nhỏ. Mỗi đoạn được chuyển thành embedding và lưu vào cơ sở dữ liệu. Khi người dùng đặt câu hỏi, hệ thống cũng tạo embedding cho câu hỏi, sau đó thực hiện tìm kiếm vector để tìm các đoạn tài liệu có nội dung liên quan nhất. Các đoạn này sẽ được sử dụng làm ngữ cảnh cho mô hình ngôn ngữ khi sinh câu trả lời.

So với tìm kiếm từ khóa truyền thống, tìm kiếm ngữ nghĩa có khả năng khai thác ý nghĩa của câu hỏi tốt hơn. Người dùng có thể diễn đạt câu hỏi theo nhiều cách khác

nhau, trong khi hệ thống vẫn có thể tìm được các đoạn tài liệu liên quan nếu nội dung có ý nghĩa tương đồng. Điều này phù hợp với bài toán tư vấn thông tin, vì người dùng thường không biết chính xác từ khóa hoặc tên tài liệu cần tra cứu.

2.4.3 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) là hướng tiếp cận kết hợp giữa truy xuất thông tin và sinh ngôn ngữ. Thay vì để mô hình ngôn ngữ trả lời hoàn toàn dựa trên kiến thức đã học, hệ thống sẽ truy xuất các tài liệu liên quan từ một kho tri thức, sau đó đưa các tài liệu này vào ngữ cảnh để mô hình sinh câu trả lời [18].

Trong đề tài, RAG được sử dụng để hỗ trợ AI Agent trả lời các câu hỏi liên quan đến Đại học Trà Vinh và các ngành đào tạo trong phạm vi kho dữ liệu đã xây dựng. Quy trình tổng quát gồm các bước: tiếp nhận câu hỏi, tạo embedding cho câu hỏi, tìm kiếm các đoạn tài liệu liên quan trong kho tri thức, xây dựng ngữ cảnh từ các đoạn tài liệu tìm được, sau đó gọi mô hình ngôn ngữ để sinh câu trả lời.

Cách tiếp cận RAG giúp hệ thống khai thác được dữ liệu riêng đã thu thập, chẳng hạn thông tin tuyển sinh, ngành đào tạo, học phí, học bổng, địa điểm và các nội dung giới thiệu trường theo từng nhóm thông tin cụ thể. Điều này giúp câu trả lời có cơ sở hơn so với việc chỉ sử dụng mô hình ngôn ngữ tổng quát. Đồng thời, khi nội dung của nhà trường thay đổi, hệ thống có thể cập nhật kho tài liệu để cải thiện khả năng trả lời mà không cần huấn luyện lại mô hình ngôn ngữ.

Trong phạm vi đề tài, RAG không chỉ là một thành phần kỹ thuật hỗ trợ hỏi đáp, mà còn là nền tảng giúp AI Agent kết nối giữa câu hỏi tự nhiên của người dùng và kho tri thức của hệ thống. Nhờ đó, người dùng có thể tiếp cận thông tin theo cách hội thoại thay vì phải tự tìm kiếm trong nhiều tài liệu hoặc trang thông tin khác nhau.

2.4.4 Mô hình AI Agent và cơ chế Function Calling

Function calling là cơ chế cho phép mô hình AI gọi các hàm hoặc công cụ được định nghĩa trước trong ứng dụng. Thay vì chỉ trả về văn bản, mô hình có thể xác định khi nào cần gọi một công cụ, truyền tham số phù hợp và để ứng dụng thực thi công cụ đó và nhận về kết quả cần thiết [19].

Trong đề tài, function calling được sử dụng để mở rộng khả năng của AI Agent. AI Agent không chỉ trả lời câu hỏi, mà còn có thể tạo ra các hành động phục vụ giao diện như điều hướng đến một địa điểm, hiển thị hình ảnh hoặc video liên quan, mở bản

đồ hoặc yêu cầu tìm kiếm thêm trong kho tài liệu. Các hành động này giúp hệ thống kết hợp giữa hội thoại và trải nghiệm tham quan trực quan.

Khái niệm AI Agent thường được hiểu là một hệ thống có khả năng tiếp nhận yêu cầu, phân tích ngữ cảnh, lựa chọn hành động phù hợp và tương tác với môi trường hoặc công cụ bên ngoài để hoàn thành nhiệm vụ [20]. Trong đề tài này, AI Agent được xây dựng theo hướng thực dụng: tiếp nhận câu hỏi từ người dùng, truy xuất thông tin bằng RAG, sinh câu trả lời bằng mô hình ngôn ngữ và điều phối một số hành động giao diện thông qua function calling.

Cách tiếp cận này khác với chatbot chỉ trả lời văn bản. Khi người dùng hỏi về một địa điểm hoặc yêu cầu xem hình ảnh, AI Agent có thể đồng thời trả lời và điều khiển giao diện hiển thị nội dung phù hợp. Nhờ đó, quá trình tìm hiểu thông tin về trường và ngành đào tạo trở nên trực quan hơn.

Trong phạm vi đề tài, AI Agent đóng vai trò là thành phần trung tâm kết nối giữa người dùng, kho tri thức và giao diện tham quan của hệ thống. Khi người dùng đặt câu hỏi bằng ngôn ngữ tự nhiên, hệ thống sẽ xử lý câu hỏi, truy xuất thông tin liên quan, sinh câu trả lời và thực hiện các hành động giao diện khi cần thiết. Tuy nhiên, AI Agent không được xây dựng với mục tiêu thay thế hoàn toàn các kênh tư vấn chính thức của nhà trường, mà được xem như một công cụ hỗ trợ giới thiệu, tư vấn sơ bộ và tạo trải nghiệm tương tác cho người dùng trong quá trình tìm hiểu thông tin.

Tóm lại, việc kết hợp mô hình ngôn ngữ lớn, embedding, RAG và function calling giúp hệ thống vượt ra khỏi mô hình chatbot văn bản đơn thuần. AI Agent trong đề tài có thể vừa trả lời câu hỏi, vừa khai thác kho tri thức và phối hợp với giao diện trực quan để hỗ trợ người dùng tìm hiểu Đại học Trà Vinh và các ngành đào tạo một cách linh hoạt, sinh động và phù hợp với ngữ cảnh hơn.

2.5 Công nghệ giọng nói

Công nghệ giọng nói giúp hệ thống hỗ trợ thêm hình thức tương tác tự nhiên bên cạnh nhập văn bản truyền thống. Trong các hệ thống trợ lý ảo hoặc AI Agent, giọng nói có thể được sử dụng theo hai hướng chính: chuyển giọng nói của người dùng thành văn bản để xử lý và chuyển câu trả lời dạng văn bản thành âm thanh để phát lại cho người dùng. Nhờ đó, quá trình tương tác trở nên gần gũi và thuận tiện hơn, đặc biệt trong bối cảnh hệ thống có nhân vật hướng dẫn 3D.

Trong đề tài này, công nghệ giọng nói được sử dụng để hỗ trợ người dùng đặt câu hỏi bằng giọng nói và nghe phản hồi từ AI Agent. Phần nhận dạng giọng nói được xử lý ở phía trình duyệt, trong khi phần tổng hợp giọng nói được xử lý ở backend để tạo audio cho câu trả lời hoặc nội dung giới thiệu địa điểm.

2.5.1 Speech-to-Text bằng Web Speech API

Speech-to-Text (STT) là quá trình chuyển đổi âm thanh lời nói của người dùng thành văn bản. Trong ứng dụng web, chức năng này giúp người dùng có thể đặt câu hỏi bằng giọng nói thay vì phải nhập nội dung bằng bàn phím.

Web Speech API là API trên trình duyệt hỗ trợ các chức năng liên quan đến giọng nói, bao gồm nhận dạng giọng nói và tổng hợp giọng nói tùy theo khả năng hỗ trợ của trình duyệt [21]. Trong đề tài, phần nhận dạng giọng nói được triển khai ở frontend thông qua đối tượng SpeechRecognition hoặc webkitSpeechRecognition. Hệ thống thiết lập ngôn ngữ nhận dạng là tiếng Việt, cho phép lấy nội dung người dùng nói và chuyển thành văn bản để gửi đến AI Agent.

Việc sử dụng Web Speech API giúp hệ thống bổ sung khả năng nhập liệu bằng giọng nói mà không cần xây dựng riêng một mô hình nhận dạng âm thanh. Tuy nhiên, khả năng hoạt động của API này phụ thuộc vào trình duyệt và môi trường sử dụng. Vì vậy, chức năng giọng nói được xem là thành phần hỗ trợ, còn nhập văn bản vẫn là phương thức tương tác cơ bản và ổn định hơn.

2.5.2 Text-to-Speech (TTS)

Text-to-Speech (TTS) là quá trình chuyển đổi văn bản thành âm thanh giọng nói. Trong hệ thống AI Agent, TTS giúp câu trả lời của mô hình ngôn ngữ không chỉ hiển thị dưới dạng văn bản mà còn có thể được phát thành giọng nói.

Trong đề tài, backend sử dụng khả năng tạo giọng nói của Gemini API để tổng hợp âm thanh từ câu trả lời hoặc nội dung giới thiệu địa điểm [22]. Khi AI Agent sinh câu trả lời, hệ thống có thể gửi nội dung văn bản đến bộ xử lý TTS để tạo tệp âm thanh. Frontend sau đó phát âm thanh này và đồng bộ trạng thái của nhân vật hướng dẫn 3D, chẳng hạn chuyển sang trạng thái đang nói khi audio được phát.

Bên cạnh TTS cho câu trả lời hội thoại, hệ thống còn sử dụng audio cho các nội dung giới thiệu địa điểm. Điều này giúp người dùng vừa quan sát không gian 360 độ, vừa nghe phần giới thiệu tương ứng. Nhờ đó, trải nghiệm tham quan không chỉ phụ thuộc vào văn bản trên màn hình mà có thêm yếu tố thuyết minh bằng giọng nói.

2.5.3 Cơ chế cache và fallback âm thanh

Việc tạo âm thanh bằng TTS có thể tốn thời gian xử lý và phụ thuộc vào dịch vụ bên ngoài. Nếu mỗi câu trả lời hoặc nội dung giới thiệu đều được tạo lại từ đầu, hệ thống có thể phản hồi chậm hơn và tiêu tốn nhiều tài nguyên hơn. Vì vậy, cơ chế cache audio là cần thiết để tái sử dụng các âm thanh đã được tạo trước đó.

Trong hệ thống của đề tài, nội dung văn bản, giọng đọc và phong cách giọng được dùng để tạo khóa cache cho audio. Nếu âm thanh tương ứng đã tồn tại, hệ thống có thể sử dụng lại thay vì gọi dịch vụ TTS. Các tệp audio được lưu tạm ở runtime cache và đồng bộ lên hệ thống lưu trữ để tái sử dụng trong các lần truy cập sau.

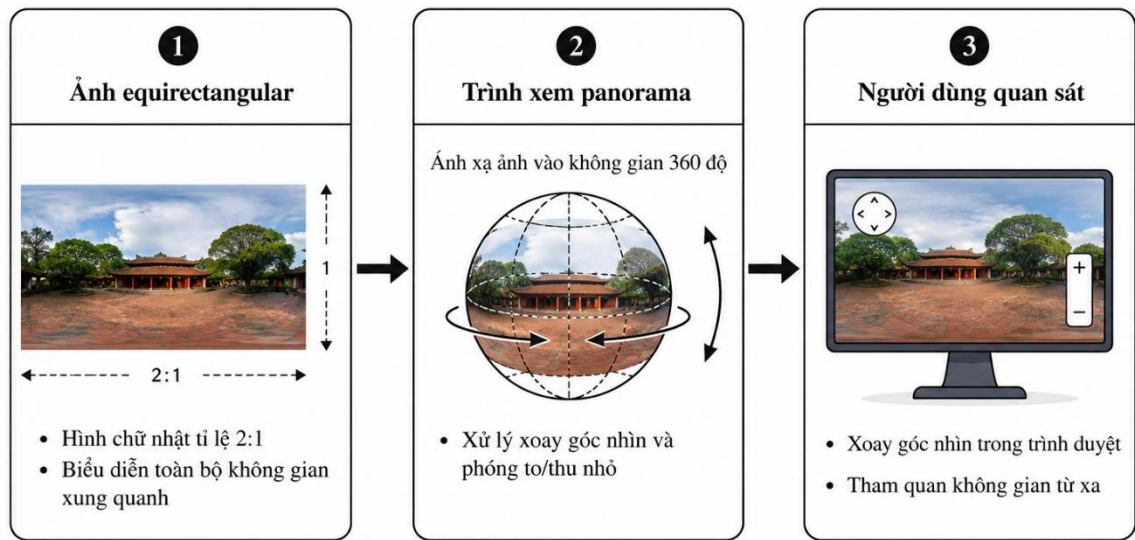
Ngoài ra, hệ thống còn có cơ chế fallback audio để xử lý các trường hợp phát sinh lỗi. Khi dịch vụ TTS chính gặp lỗi hoặc vượt giới hạn sử dụng, hệ thống có thể chuyển sang dịch vụ tạo giọng nói dự phòng để vẫn cung cấp được phản hồi âm thanh cho người dùng. Cách tiếp cận này giúp tăng tính ổn định của chức năng giọng nói, đặc biệt trong các tình huống trình diễn, kiểm thử hoặc sử dụng thực tế.

Tóm lại, công nghệ giọng nói trong đề tài được sử dụng nhằm tăng tính tự nhiên của quá trình tương tác. Speech-to-Text giúp người dùng đặt câu hỏi bằng giọng nói, Text-to-Speech giúp AI Agent phản hồi bằng âm thanh, còn cache và fallback audio giúp cải thiện tốc độ phản hồi và độ ổn định của hệ thống.

2.6 Ảnh 360 độ và panorama

Ảnh 360 độ là dạng hình ảnh đặc biệt cho phép người dùng quan sát không gian xung quanh từ một điểm nhìn cố định một cách toàn diện hơn. Khác với ảnh thông thường chỉ thể hiện một góc nhìn nhất định, ảnh 360 độ có khả năng bao quát toàn bộ không gian theo phương ngang và một phần hoặc toàn bộ phương dọc. Khi được hiển thị trên trình duyệt bằng trình xem panorama, người dùng có thể dễ dàng xoay góc nhìn, phóng to hoặc thu nhỏ để quan sát không gian theo cách trực quan và sinh động hơn.

Trong đề tài này, ảnh 360 độ được sử dụng để hỗ trợ trải nghiệm tham quan một số địa điểm trong khuôn viên Đại học Trà Vinh. Người dùng có thể quan sát không gian tại từng địa điểm, kết hợp với bản đồ, hình ảnh, video và phản hồi đáp của AI Agent. Cách hiển thị ảnh panorama 360 độ trên trình duyệt được minh họa ở Hình 2.3.



Hình 2.3. Minh họa ảnh panorama 360 độ và cách hiển thị trên trình duyệt

Từ Hình 2.3 có thể thấy ảnh panorama 360 độ cho phép người dùng quan sát không gian theo nhiều hướng khác nhau ngay trên trình duyệt. Nhờ đó, việc giới thiệu trường không chỉ dừng lại ở văn bản mà còn được bổ sung thêm yếu tố trực quan, góp phần nâng cao trải nghiệm tham quan và tìm hiểu thông tin.

2.6.1 Ảnh equirectangular

Một trong các định dạng phổ biến của ảnh panorama 360 độ là ảnh equirectangular. Đây là dạng ảnh phẳng có tỉ lệ thường gặp là 2:1, trong đó toàn bộ không gian xung quanh được trải ra trên một mặt phẳng hình chữ nhật. Khi nhìn trực tiếp, ảnh equirectangular có thể bị méo ở một số vùng, đặc biệt ở phần trên và dưới ảnh. Tuy nhiên, khi được đưa vào trình xem panorama, ảnh sẽ được ánh xạ lên một không gian ảo để người dùng có thể xoay góc nhìn tự nhiên hơn.

Trong hệ thống tham quan ảo, ảnh equirectangular phù hợp vì chỉ cần một tệp ảnh có thể biểu diễn toàn bộ không gian xung quanh tại một điểm chụp. Điều này giúp việc lưu trữ, quản lý và hiển thị ảnh đơn giản hơn so với việc phải xử lý nhiều ảnh riêng lẻ cho từng hướng nhìn.

Đối với đề tài, ảnh equirectangular được sử dụng làm ảnh nền 360 độ cho các địa điểm. Mỗi địa điểm có thể gắn với một ảnh nền tương ứng, từ đó frontend tải ảnh và hiển thị thông qua trình xem panorama. Cách tiếp cận này giúp người dùng có cảm giác đang đứng tại một vị trí trong khuôn viên và quan sát không gian xung quanh.

2.6.2 Hiển thị panorama trên trình duyệt

Để người dùng có thể tương tác với ảnh 360 độ, ảnh equirectangular cần được hiển thị thông qua một trình xem panorama trên trình duyệt. Trình xem này có nhiệm vụ

vụ dựng không gian quan sát, xử lý thao tác xoay góc nhìn, phóng to, thu nhỏ và cập nhật khung nhìn theo thao tác của người dùng.

Pannellum là thư viện hỗ trợ hiển thị ảnh panorama trên trình duyệt, trong đó có ảnh equirectangular [8]. Trong đề tài, Pannellum được tích hợp vào frontend thông qua một component riêng. Khi người dùng chuyển địa điểm, component này sẽ tải ảnh 360 độ tương ứng và cập nhật lại trình xem, giúp quá trình tham quan diễn ra liền mạch hơn.

Bên cạnh việc hiển thị ảnh, panorama còn được kết hợp với các thành phần khác như bản đồ khuôn viên, media minh họa và AI Agent. Khi người dùng hỏi về một địa điểm, hệ thống có thể điều hướng đến địa điểm liên quan, hiển thị ảnh 360 độ và cung cấp thêm thông tin bằng văn bản hoặc giọng nói. Sự kết hợp này giúp trải nghiệm tham quan trở nên trực quan hơn so với chỉ sử dụng văn bản hoặc hình ảnh tĩnh.

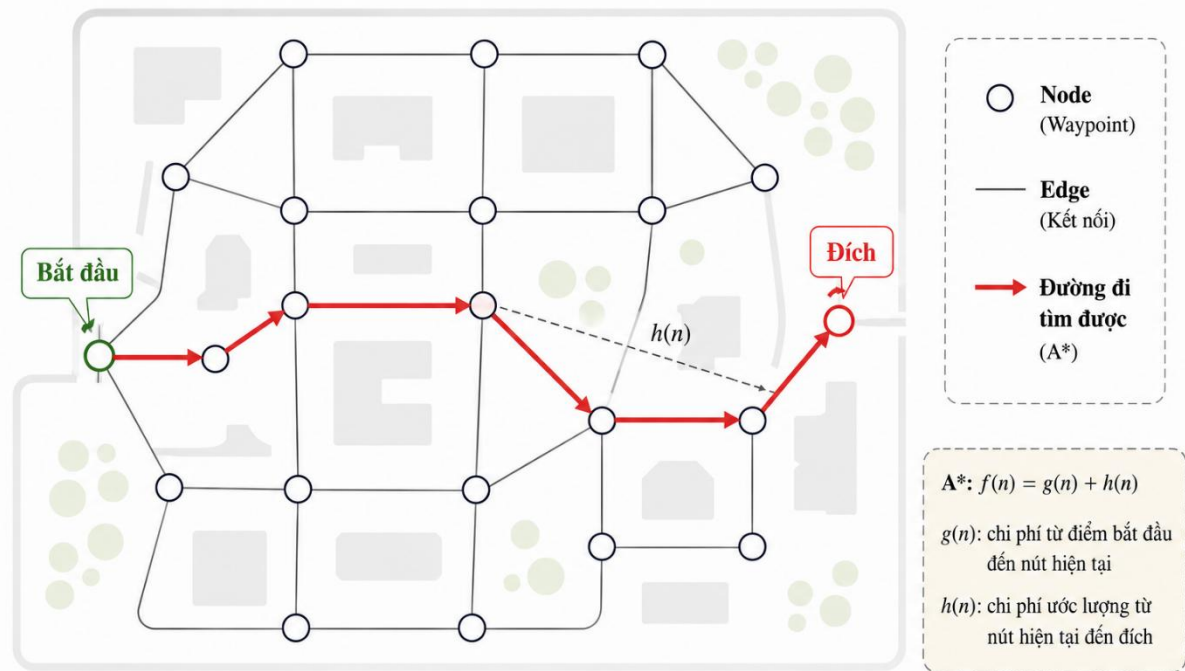
Tuy nhiên, ảnh 360 độ vẫn phụ thuộc vào chất lượng ảnh, vị trí chụp, ánh sáng và mức độ bao phủ dữ liệu. Vì vậy, trong phạm vi đề tài, ảnh panorama được sử dụng như một thành phần hỗ trợ trực quan, giúp người dùng có cái nhìn sơ bộ và sinh động hơn về một số không gian tiêu biểu trong khuôn viên trường.

2.7 Thuật toán tìm đường A^*

Trong hệ thống tham quan trực quan, bản đồ khuôn viên không chỉ dùng để hiển thị vị trí các địa điểm mà còn hỗ trợ người dùng hình dung đường đi giữa các khu vực. Để thực hiện chức năng này, hệ thống cần biểu diễn bản đồ dưới dạng có thể tính toán và áp dụng thuật toán tìm đường phù hợp.

Trong đề tài, thuật toán A^* được sử dụng để tìm đường đi giữa các địa điểm trên bản đồ khuôn viên. A^* là thuật toán tìm đường phổ biến, kết hợp giữa chi phí đã đi từ điểm bắt đầu và hàm ước lượng chi phí đến điểm đích, từ đó ưu tiên mở rộng các nút có khả năng nằm trên đường đi tối ưu [23]. Cách biểu diễn thuật toán A^* trên đồ thị waypoint của bản đồ khuôn viên được minh họa ở Hình 2.4.

Thuật toán tìm đường A* trên bản đồ khuôn viên



Hình 2.4. Minh họa thuật toán A* trên đồ thị waypoint của bản đồ khuôn viên

Từ Hình 2.4 có thể thấy các địa điểm và các điểm trung gian trên bản đồ được biểu diễn thành các waypoint, còn các đoạn đường có thể di chuyển được biểu diễn bằng các cạnh nối giữa các waypoint. Dựa trên cấu trúc này, thuật toán A* có thể xác định tuyến đường từ điểm bắt đầu đến điểm đích và trả về tuyến đường tìm được để frontend hiển thị trực quan trên bản đồ cho người dùng.

2.7.1 Khái niệm graph và waypoint

Để áp dụng thuật toán tìm đường, bản đồ khuôn viên được biểu diễn dưới dạng đồ thị. Trong đó, các điểm quan trọng được biểu diễn thành các nút, còn các đoạn đường có thể di chuyển giữa hai điểm được biểu diễn thành các cạnh. Mỗi nút có thể gắn với tọa độ trên bản đồ, tên địa điểm hoặc slug của địa điểm tương ứng. Mỗi cạnh thể hiện khả năng di chuyển giữa hai nút và có thể kèm theo chi phí di chuyển.

Cách biểu diễn này giúp hệ thống tách phần hình ảnh bản đồ khỏi phần tính toán đường đi một cách rõ ràng. Hình ảnh bản đồ được dùng để hiển thị trực quan cho người dùng, còn đồ thị waypoint được dùng để tính toán tuyến đường. Khi cần tìm đường từ địa điểm hiện tại đến một địa điểm khác, hệ thống chỉ cần xác định nút bắt đầu, nút đích và chạy thuật toán trên đồ thị.

Trong phạm vi đề tài, tọa độ các nút được chuẩn hóa theo bản đồ để frontend có thể vẽ đường đi. Sau khi tìm được tuyến đường, danh sách tọa độ tương ứng sẽ được gửi về frontend để hiển thị trên bản đồ.

2.7.2 Nguyên lý thuật toán A^*

Thuật toán A^* hoạt động bằng cách đánh giá mỗi nút dựa trên một hàm chi phí. Hàm này kết hợp giữa chi phí thực tế đã đi từ nút bắt đầu đến nút hiện tại và chi phí ước lượng từ nút hiện tại đến nút đích, được thể hiện như sau:

$$f(n) = g(n) + h(n) \quad (2.1)$$

Trong công thức (2.1), $g(n)$ là chi phí thực tế từ nút bắt đầu đến nút hiện tại, còn $h(n)$ là chi phí ước lượng từ nút hiện tại đến nút đích. Giá trị $f(n)$ được dùng để quyết định nút nào sẽ được ưu tiên mở rộng tiếp theo. Nếu hàm $h(n)$ được chọn phù hợp, A^* có thể tìm được đường đi hiệu quả hơn so với việc duyệt toàn bộ đồ thị.

Trong bài toán bản đồ khuôn viên, $h(n)$ có thể được tính dựa trên khoảng cách hình học giữa nút hiện tại và nút đích. Với mỗi bước duyệt, thuật toán cập nhật chi phí của các nút lân cận, lưu lại nút trước đó để phục hồi đường đi và tiếp tục mở rộng cho đến khi tìm được nút đích hoặc không còn nút nào để xét.

A^* phù hợp với bài toán của đề tài vì bản đồ khuôn viên có thể được biểu diễn bằng hệ thống waypoint rõ ràng, đồng thời thuật toán có khả năng tìm tuyến đường hợp lý giữa các địa điểm để hiển thị cho người dùng. Ngoài danh sách nút trên đường đi, thuật toán còn có thể trả về các thông tin như tổng chi phí, số nút đã duyệt hoặc các bước mở rộng để phục vụ minh họa quá trình tìm đường.

2.7.3 Ứng dụng A^* trong điều hướng bản đồ

Trong hệ thống của đề tài, thuật toán A^* được sử dụng để tìm đường giữa các địa điểm trên bản đồ khuôn viên. Khi người dùng chọn địa điểm hoặc khi AI Agent yêu cầu điều hướng, backend xử lý truy vấn trên dữ liệu đồ thị và trả về tuyến đường gồm các nút, tọa độ và tổng chi phí. Frontend sử dụng kết quả này để hiển thị đường đi trực quan trên bản đồ, giúp người dùng dễ hình dung hướng di chuyển đến địa điểm cần tìm. Cách triển khai này giúp bản đồ không chỉ là hình ảnh minh họa, mà còn trở thành thành phần hỗ trợ tương tác trong quá trình tham quan. Nhờ đó, chức năng hỏi đáp của AI Agent được liên kết với chức năng tham quan trực quan, hỗ trợ người dùng di chuyển giữa các địa điểm trong quá trình giới thiệu trường.

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

Chương này trình bày quá trình phân tích yêu cầu và thiết kế hệ thống AI Agent giới thiệu Đại học Trà Vinh, gồm yêu cầu hệ thống, kiến trúc tổng thể, cơ sở dữ liệu, RAG và AI Agent, bản đồ dẫn đường, giao diện người dùng và cơ chế vận hành kiosk.

3.1 Phân tích yêu cầu

Phân tích yêu cầu là bước xác định nhóm người dùng, phạm vi chức năng và các điều kiện cần đáp ứng trước khi thiết kế hệ thống. Trong đề tài này, yêu cầu được xem xét theo hai nhóm chính: người dùng tham quan và người quản trị nội dung.

3.1.1 Đối tượng sử dụng và phạm vi hệ thống

Hệ thống hướng đến hai nhóm sử dụng chính: người dùng tham quan và người quản trị. Người dùng tham quan bao gồm thí sinh, phụ huynh, sinh viên mới hoặc các cá nhân có nhu cầu tìm hiểu về Đại học Trà Vinh qua giao diện tour. Người quản trị chịu trách nhiệm cập nhật dữ liệu, tài liệu, media và các cấu hình vận hành hệ thống.

Để làm rõ vai trò và nhu cầu của từng nhóm sử dụng, các đối tượng chính của hệ thống được trình bày trong Bảng 3.1.

Bảng 3.1. Đối tượng sử dụng của hệ thống

Đối tượng	Vai trò	Nhu cầu chính
Người dùng tham quan	Sử dụng giao diện kiosk hoặc web tour để tìm hiểu trường, địa điểm, ngành đào tạo và môi trường học tập.	Xem địa điểm, ảnh 360 độ, media, bản đồ, nghe nội dung giới thiệu và đặt câu hỏi cho AI Agent bằng ngôn ngữ tự nhiên.
Người quản trị	Quản lý dữ liệu và nội dung vận hành của hệ thống.	Cập nhật địa điểm, tài liệu RAG, media, cấu hình nhân vật hướng dẫn 3D, cache và thiết lập kiosk.

Từ Bảng 3.1 có thể thấy hệ thống được thiết kế cho cả người dùng cuối và người vận hành nội dung. Trong đó, người dùng tham quan tập trung vào trải nghiệm tìm hiểu thông tin và khám phá trực quan, còn người quản trị tập trung vào việc cập nhật, duy trì và cấu hình dữ liệu phục vụ hệ thống.

Trong phạm vi đề tài, hệ thống tập trung vào giới thiệu thông tin, hỗ trợ hỏi đáp sơ bộ và tạo trải nghiệm tham quan trực quan. Hệ thống không thay thế các kênh tư vấn chính thức của nhà trường, không xử lý đăng ký tuyển sinh, xét tuyển, thanh toán hoặc thủ tục hành chính. Các câu trả lời của AI Agent dựa trên kho tài liệu đã thu thập và chỉ mang tính hỗ trợ tham khảo.

3.1.2 Yêu cầu chức năng

Yêu cầu chức năng mô tả các khả năng mà hệ thống cần cung cấp cho người dùng và người quản trị trong quá trình sử dụng. Trong đề tài này, các chức năng được chia thành các nhóm chính gồm tham quan trực quan, hỏi đáp bằng AI Agent, bản đồ dẫn đường, giọng nói và quản trị nội dung. Danh sách các yêu cầu chức năng chính của hệ thống được trình bày trong Bảng 3.2.

Bảng 3.2. Yêu cầu chức năng của hệ thống

Mã yêu cầu	Nhóm chức năng	Mô tả yêu cầu
F01	Khởi tạo trải nghiệm tham quan	Hệ thống cho phép người dùng bắt đầu phiên tham quan, tải dữ liệu địa điểm mặc định và hiển thị giao diện chính.
F02	Xem thông tin địa điểm	Người dùng có thể xem danh sách địa điểm, chuyển giữa các địa điểm và xem thông tin giới thiệu tương ứng.
F03	Hiển thị ảnh 360 độ	Hệ thống hiển thị ảnh panorama 360 độ của địa điểm, cho phép người dùng xoay góc nhìn, phóng to, thu nhỏ và đưa góc nhìn về trạng thái ban đầu.
F04	Hiển thị media	Người dùng có thể xem hình ảnh, video và các nội dung media liên quan đến địa điểm đang tham quan.
F05	Hỏi đáp với AI Agent	Người dùng có thể đặt câu hỏi bằng văn bản để AI Agent trả lời thông tin về trường, địa điểm và ngành đào tạo.
F06	Truy xuất kho tri thức RAG	Hệ thống truy xuất các đoạn tài liệu liên quan từ kho tri thức để hỗ trợ mô hình AI sinh câu trả lời bám sát dữ liệu đã thu thập.
F07	Function Calling và điều khiển giao diện	AI Agent có thể yêu cầu giao diện thực hiện một số hành động như điều hướng đến địa điểm, mở media hoặc hiển thị bản đồ.

Mã yêu cầu	Nhóm chức năng	Mô tả yêu cầu
F08	Tương tác giọng nói	Hệ thống hỗ trợ nhập câu hỏi bằng giọng nói và phát câu trả lời hoặc nội dung giới thiệu dưới dạng âm thanh.
F09	Bản đồ và dẫn đường	Người dùng có thể mở bản đồ khuôn viên, xem vị trí các địa điểm và tuyến đường giữa các địa điểm bằng thuật toán A^* .
F10	Quản trị địa điểm và media	Người quản trị có thể cập nhật thông tin địa điểm, trạng thái hiển thị, ảnh nền 360 độ, hình ảnh, video và các liên kết liên quan.
F11	Quản trị tài liệu RAG	Người quản trị có thể tải tài liệu lên hệ thống, theo dõi trạng thái xử lý, phân loại và xóa tài liệu khi cần.
F12	Quản trị nhân vật hướng dẫn 3D và kiosk	Người quản trị có thể cấu hình nhân vật hướng dẫn 3D, giọng đọc, thiết lập kiosk, cache và một số thông số vận hành.
F13	Xác thực trang quản trị	Hệ thống yêu cầu đăng nhập trước khi truy cập các chức năng thay đổi dữ liệu.

Từ Bảng 3.2 có thể thấy các yêu cầu chức năng của hệ thống bao quát cả trải nghiệm tham quan của người dùng và hoạt động quản trị nội dung. Nhóm chức năng dành cho người dùng tập trung vào xem địa điểm, hỏi đáp bằng AI, tương tác giọng nói, media và bản đồ dẫn đường, còn nhóm quản trị hỗ trợ cập nhật địa điểm, tài liệu RAG, media, nhân vật hướng dẫn 3D, cache và cấu hình kiosk.

3.1.3 Yêu cầu phi chức năng

Yêu cầu phi chức năng mô tả các tiêu chí về khả năng sử dụng, hiệu năng, độ ổn định, khả năng bảo trì và an toàn khi vận hành hệ thống. Với định hướng sử dụng trong môi trường kiosk và tích hợp nhiều thành phần như AI Agent, RAG, media, bản đồ và giọng nói, các yêu cầu này giúp hệ thống hoạt động ổn định, dễ mở rộng và phù hợp hơn với quá trình sử dụng thực tế trong nhiều tình huống khác nhau. Danh sách các yêu cầu phi chức năng chính được trình bày trong Bảng 3.3.

Bảng 3.3. Yêu cầu phi chức năng của hệ thống

Mã yêu cầu	Nhóm yêu cầu	Mô tả yêu cầu
NF01	Dễ sử dụng	Giao diện cần trực quan, dễ thao tác, phù hợp với người dùng không có kiến thức kỹ thuật.
NF02	Tính tương tác	Hệ thống cần phản hồi rõ ràng khi người dùng chuyển địa điểm, đặt câu hỏi, mở media hoặc xem bản đồ.
NF03	Hiệu năng phản hồi	Các thao tác thông thường như chuyển địa điểm, mở bản đồ, xem media cần phản hồi trong thời gian hợp lý, phần AI có thể hiển thị trạng thái chờ hoặc streaming để giảm cảm giác chậm.
NF04	Tính ổn định	Hệ thống cần xử lý được một số tình huống lỗi như mất kết nối, tải media thất bại hoặc dịch vụ AI phản hồi chậm.
NF05	Khả năng bảo trì	Mã nguồn cần được tổ chức theo module, tách biệt frontend, backend, service, repository và các thành phần AI để thuận lợi cho việc mở rộng.
NF06	Khả năng mở rộng	Hệ thống cần hỗ trợ bổ sung thêm địa điểm, tài liệu, media và cấu hình nhân vật hướng dẫn 3D thông qua trang quản trị.
NF07	An toàn dữ liệu quản trị	Các chức năng thay đổi dữ liệu cần được bảo vệ bằng xác thực, tránh truy cập trái phép vào trang quản trị.

Mã yêu cầu	Nhóm yêu cầu	Mô tả yêu cầu
NF08	Phù hợp môi trường vận hành kiosk	Hệ thống cần có cơ chế hạn chế thao tác ngoài luồng, tự reset khi không hoạt động và phục hồi giao diện khi gặp sự cố phổ biến.
NF09	Tính nhất quán dữ liệu	Dữ liệu địa điểm, media, tài liệu và vector embedding cần được liên kết rõ ràng để AI Agent và giao diện sử dụng đúng ngữ cảnh.

Từ Bảng 3.3 có thể thấy các yêu cầu phi chức năng tập trung vào việc bảo đảm hệ thống dễ sử dụng, phản hồi ổn định và phù hợp với môi trường kiosk thực tế. Bên cạnh trải nghiệm người dùng, hệ thống cũng cần chú trọng khả năng bảo trì, mở rộng dữ liệu linh hoạt, bảo vệ chức năng quản trị và duy trì tính nhất quán giữa dữ liệu địa điểm, media, tài liệu và AI Agent.

3.1.4 Đặc tả Use Case

Dựa trên các yêu cầu chức năng đã xác định, hệ thống được đặc tả bằng các Use Case nhằm thể hiện mối quan hệ giữa tác nhân và các chức năng chính. Trong phạm vi đề tài, hệ thống có hai nhóm tác nhân chính là người dùng tham quan và người quản trị. Người dùng tham quan tương tác với giao diện tour để tìm hiểu thông tin, quan sát không gian, đặt câu hỏi và sử dụng các chức năng hỗ trợ tham quan, còn người quản trị tương tác với Admin Panel để cập nhật dữ liệu và cấu hình vận hành hệ thống. Các thành phần như AI Agent, cơ sở dữ liệu, dịch vụ AI và hệ thống lưu trữ được xem là thành phần xử lý hoặc dịch vụ hỗ trợ bên trong hệ thống, không phải tác nhân trực tiếp.

Các Use Case theo từng nhóm tác nhân được tóm tắt trong Bảng 3.4.

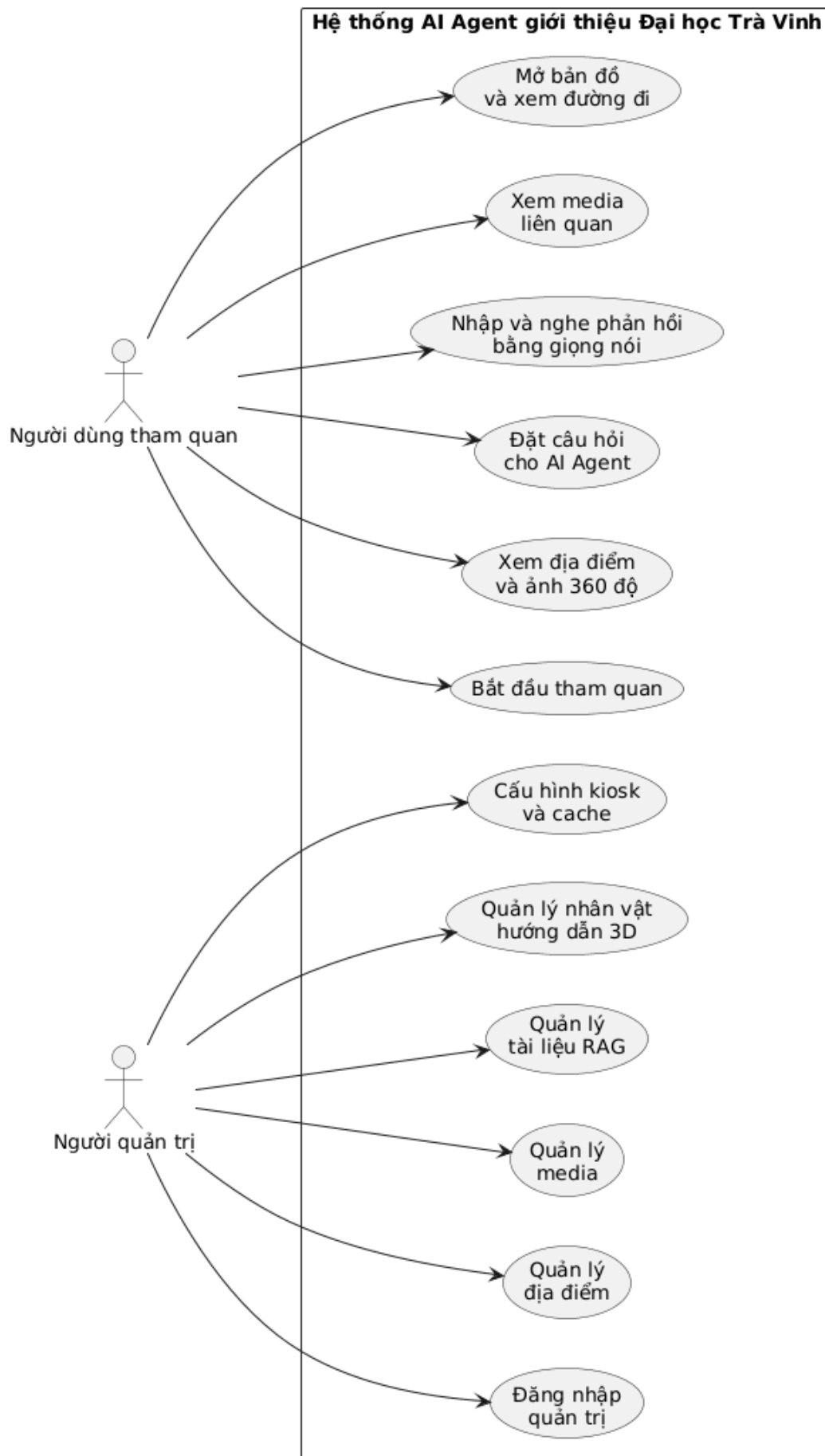
Bảng 3.4. Tóm tắt các Use Case chính

Mã Use Case	Tác nhân	Tên Use Case
UC01	Người dùng tham quan	Bắt đầu tham quan
UC02	Người dùng tham quan	Xem địa điểm và ảnh 360 độ
UC03	Người dùng tham quan	Đặt câu hỏi cho AI Agent
UC04	Người dùng tham quan	Tương tác bằng giọng nói
UC05	Người dùng tham quan	Xem media liên quan
UC06	Người dùng tham quan	Mở bản đồ và xem đường đi
UC07	Người quản trị	Đăng nhập quản trị
UC08	Người quản trị	Quản lý địa điểm

Mã Use Case	Tác nhân	Tên Use Case
UC09	Người quản trị	Quản lý media
UC10	Người quản trị	Quản lý tài liệu RAG
UC11	Người quản trị	Quản lý nhân vật hướng dẫn 3D
UC12	Người quản trị	Cấu hình kiosk và cache

Từ Bảng 3.4 có thể thấy các Use Case của người dùng tham quan tập trung vào quá trình trải nghiệm hệ thống, gồm bắt đầu tham quan, xem địa điểm, quan sát ảnh 360 độ, đặt câu hỏi cho AI Agent, tương tác bằng giọng nói, xem media và sử dụng bản đồ dẫn đường. Trong khi đó, các Use Case của người quản trị tập trung vào việc đăng nhập, cập nhật và duy trì dữ liệu hệ thống, bao gồm quản lý địa điểm, media, tài liệu RAG, nhân vật hướng dẫn 3D, kiosk và cache.

Dựa trên các Use Case đã xác định, sơ đồ Use Case tổng quát được xây dựng nhằm thể hiện trực quan mối quan hệ giữa hai nhóm tác nhân và các chức năng chính của hệ thống, như trình bày ở Hình 3.1.



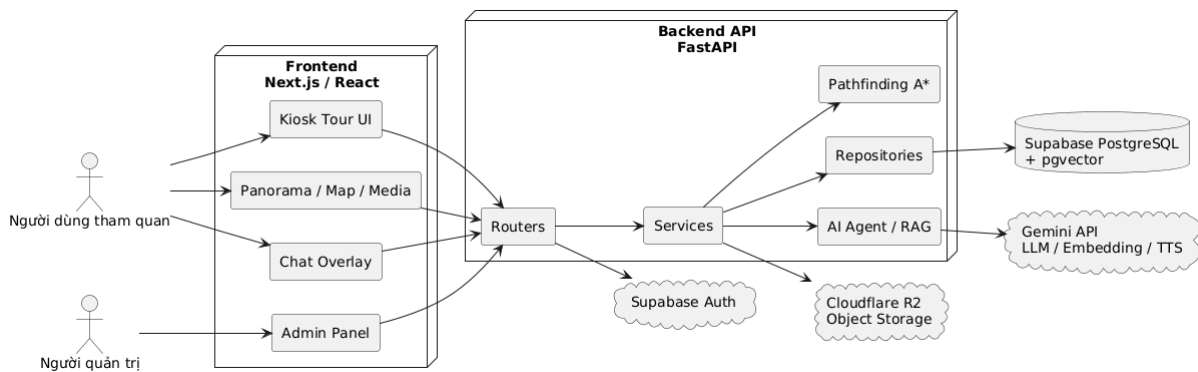
Hình 3.1. Sơ đồ Use Case tổng quát của hệ thống

3.2 Thiết kế kiến trúc tổng thể

Sau khi xác định các yêu cầu, hệ thống được thiết kế theo hướng tách biệt giữa frontend, backend API, cơ sở dữ liệu, hệ thống lưu trữ và các dịch vụ AI nhằm tạo điều kiện thuận lợi cho quá trình phát triển, bảo trì và mở rộng trong tương lai.

3.2.1 Sơ đồ kiến trúc hệ thống

Kiến trúc tổng thể của hệ thống gồm giao diện tham quan, giao diện quản trị, backend API, cơ sở dữ liệu, hệ thống lưu trữ và các dịch vụ AI. Hai nhóm giao diện giao tiếp với backend thông qua API, còn backend chịu trách nhiệm xử lý nghiệp vụ, truy xuất dữ liệu, điều phối AI Agent, xử lý RAG, quản lý media và tính toán đường đi trên bản đồ. Supabase PostgreSQL được dùng để lưu trữ dữ liệu hệ thống, trong khi các dịch vụ AI được gọi khi cần xử lý câu hỏi, tạo embedding hoặc sinh phản hồi bằng giọng nói. Kiến trúc tổng thể của hệ thống được minh họa ở Hình 3.2.

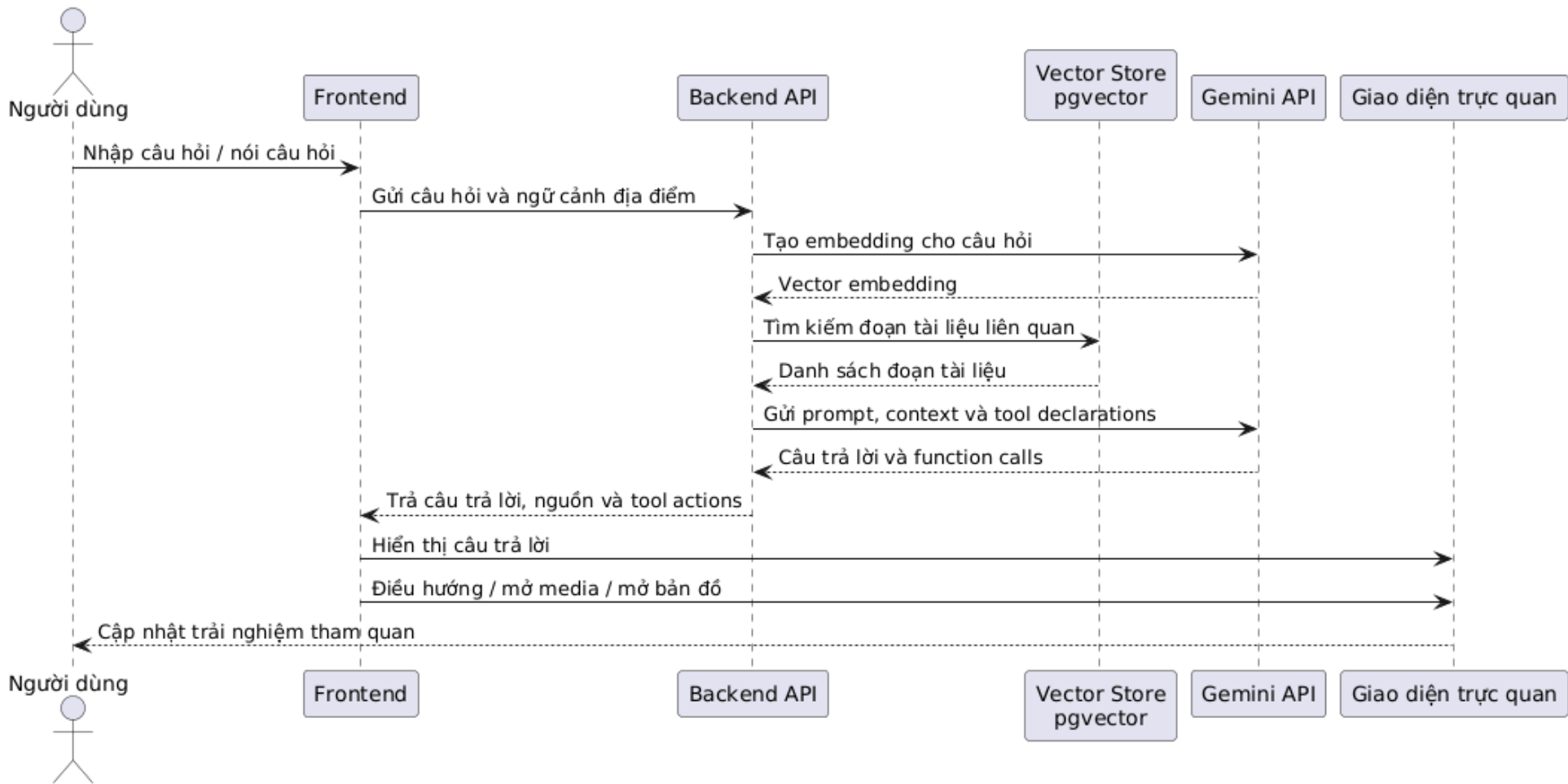


Hình 3.2. Sơ đồ kiến trúc tổng thể của hệ thống

Từ Hình 3.2 có thể thấy backend API đóng vai trò trung tâm trong việc kết nối giao diện người dùng, giao diện quản trị, cơ sở dữ liệu, hệ thống lưu trữ và các dịch vụ AI. Cách thiết kế này giúp hệ thống phân tách rõ trách nhiệm giữa các thành phần, đồng thời thuận lợi khi cần mở rộng chức năng hoặc thay đổi từng phần riêng lẻ.

3.2.2 Luồng tương tác giữa người dùng, frontend, backend và AI

Luồng tương tác tổng quát mô tả quá trình người dùng thao tác trên frontend, frontend gửi yêu cầu đến backend và backend xử lý để trả kết quả về giao diện. Với các chức năng liên quan đến AI Agent, backend không chỉ xử lý yêu cầu thông thường mà còn truy xuất kho tri thức, gọi mô hình AI và xác định các hành động giao diện cần thực hiện. Kết quả sau đó được frontend sử dụng để hiển thị câu trả lời, cập nhật nội dung tham quan hoặc thực hiện các thao tác như mở bản đồ, điều hướng và hiển thị media. Luồng xử lý này được minh họa ở Hình 3.3.



Hình 3.3. Luồng tương tác giữa người dùng, frontend, backend và AI Agent

3.2.3 Kiến trúc Backend theo mô hình phân lớp

Backend được thiết kế theo mô hình phân lớp nhằm tách biệt rõ trách nhiệm giữa tiếp nhận yêu cầu, xử lý nghiệp vụ và truy xuất dữ liệu. Cách tổ chức này giúp mã nguồn dễ theo dõi hơn, đồng thời hạn chế việc trộn lẫn logic xử lý HTTP, logic nghiệp vụ và truy vấn cơ sở dữ liệu trong cùng một nơi.

Các lớp chính trong kiến trúc backend và vai trò được trình bày trong Bảng 3.5.

Bảng 3.5. Các lớp chính trong kiến trúc backend

Lớp / thành phần	Vai trò	Ví dụ chức năng trong hệ thống
Router	Tiếp nhận request từ frontend, kiểm tra đầu vào cơ bản và gọi service tương ứng.	API địa điểm, chat, điều hướng, quản trị, quản lý cache.
Schema	Định nghĩa cấu trúc dữ liệu request/response và hỗ trợ kiểm tra dữ liệu.	Dữ liệu địa điểm, tài liệu, media, phiên chat, cấu hình kiosk.
Service	Xử lý nghiệp vụ và điều phối các bước xử lý chính	RAG, ingestion tài liệu, TTS, lưu trữ media, tìm đường A*.
Repository	Làm việc trực tiếp với cơ sở dữ liệu	Truy vấn địa điểm, tài liệu, media, vector embedding.
AI module	Đóng gói phần gọi mô hình AI, embedding, TTS và function declarations.	Gemini chat, embedding, công cụ navigate_to, show_media, toggle_map, search_documents.
Dependency và Auth	Kiểm tra xác thực và cung cấp các phụ thuộc dùng chung.	Kết nối database, kiểm tra token Supabase Auth.

Từ Bảng 3.5 có thể thấy backend được tổ chức theo từng lớp với trách nhiệm riêng. Router tập trung vào tiếp nhận request, Service xử lý nghiệp vụ chính, Repository làm việc với cơ sở dữ liệu, còn các module AI và Auth hỗ trợ những chức năng đặc thù như gọi mô hình AI, tạo embedding, kiểm tra token và cung cấp các phụ thuộc dùng chung. Cách phân lớp này giúp backend dễ bảo trì, kiểm thử và mở rộng khi bổ sung thêm chức năng mới trong tương lai.

3.2.4 Kiến trúc Frontend theo hướng feature-based

Frontend được thiết kế theo hướng feature-based, tức là mã nguồn được tổ chức quanh các nhóm chức năng chính thay vì chỉ chia theo loại tệp. Cách tổ chức này phù hợp với hệ thống có nhiều thành phần tương tác như tour, chat, bản đồ, panorama, media và trang quản trị. Mỗi nhóm chức năng có thể chứa component, hook, store, kiểu dữ liệu và các xử lý liên quan riêng.

Các nhóm chức năng chính ở frontend được trình bày trong Bảng 3.6.

Bảng 3.6. Các nhóm chức năng chính ở frontend

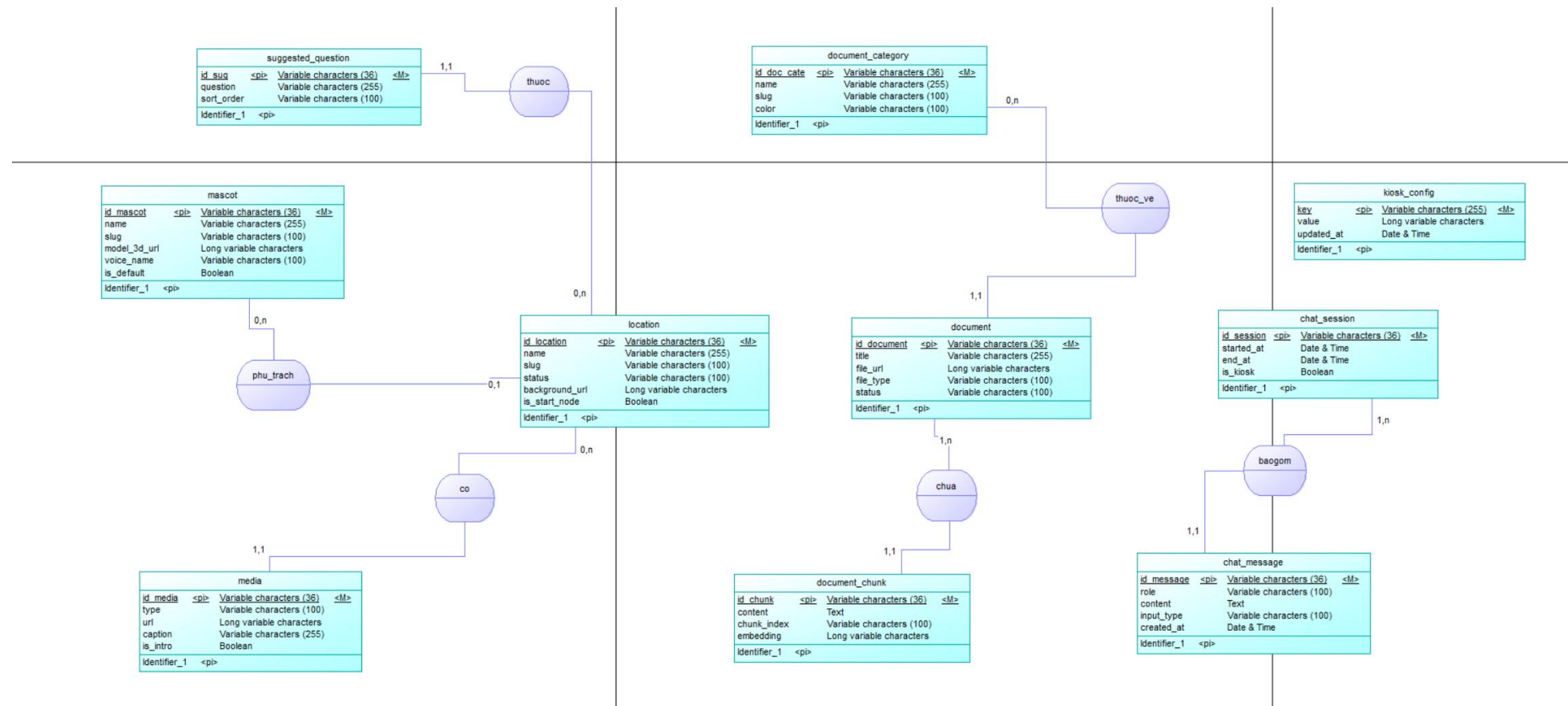
Nhóm chức năng	Vai trò	Thành phần tiêu biểu
Tour	Quản lý trải nghiệm tham quan trực quan	Panorama viewer, bản đồ, media panel, nhân vật 3D, trạng thái địa điểm.
Chat	Quản lý hội thoại với AI Agent	Khung chat, gửi câu hỏi, xử lý streaming, phát audio, thực thi tool actions.
Admin	Quản lý giao diện quản trị	Đăng nhập, dashboard, địa điểm, tài liệu, media, nhân vật hướng dẫn 3D, cache.
App routes	Tổ chức các trang chính của ứng dụng	Trang tour chính, admin panel, map demo, map editor.
Thành phần dùng chung	Chứa các xử lý dùng chung trong hệ thống	Gọi API, xử lý URL, phục hồi WebGL, cấu hình dùng chung.

Từ Bảng 3.6 có thể thấy frontend được tổ chức theo từng nhóm chức năng thay vì gom toàn bộ component, hook hoặc store vào các thư mục chung, giúp cấu trúc mã nguồn rõ ràng và dễ mở rộng hơn trong quá trình phát triển. Trong giao diện tour và chat, frontend sử dụng store trạng thái để quản lý địa điểm, panorama, bản đồ, tuyến đường, media, hội thoại và nhân vật hướng dẫn 3D, đồng thời đảm bảo việc đồng bộ dữ liệu giữa các thành phần giao diện diễn ra nhất quán. Đối với Admin Panel, giao diện được chia theo các nhóm quản trị như địa điểm, tài liệu, media, nhân vật 3D, cache và cấu hình hệ thống, từ đó hỗ trợ việc quản lý và bảo trì hệ thống một cách hiệu quả hơn.

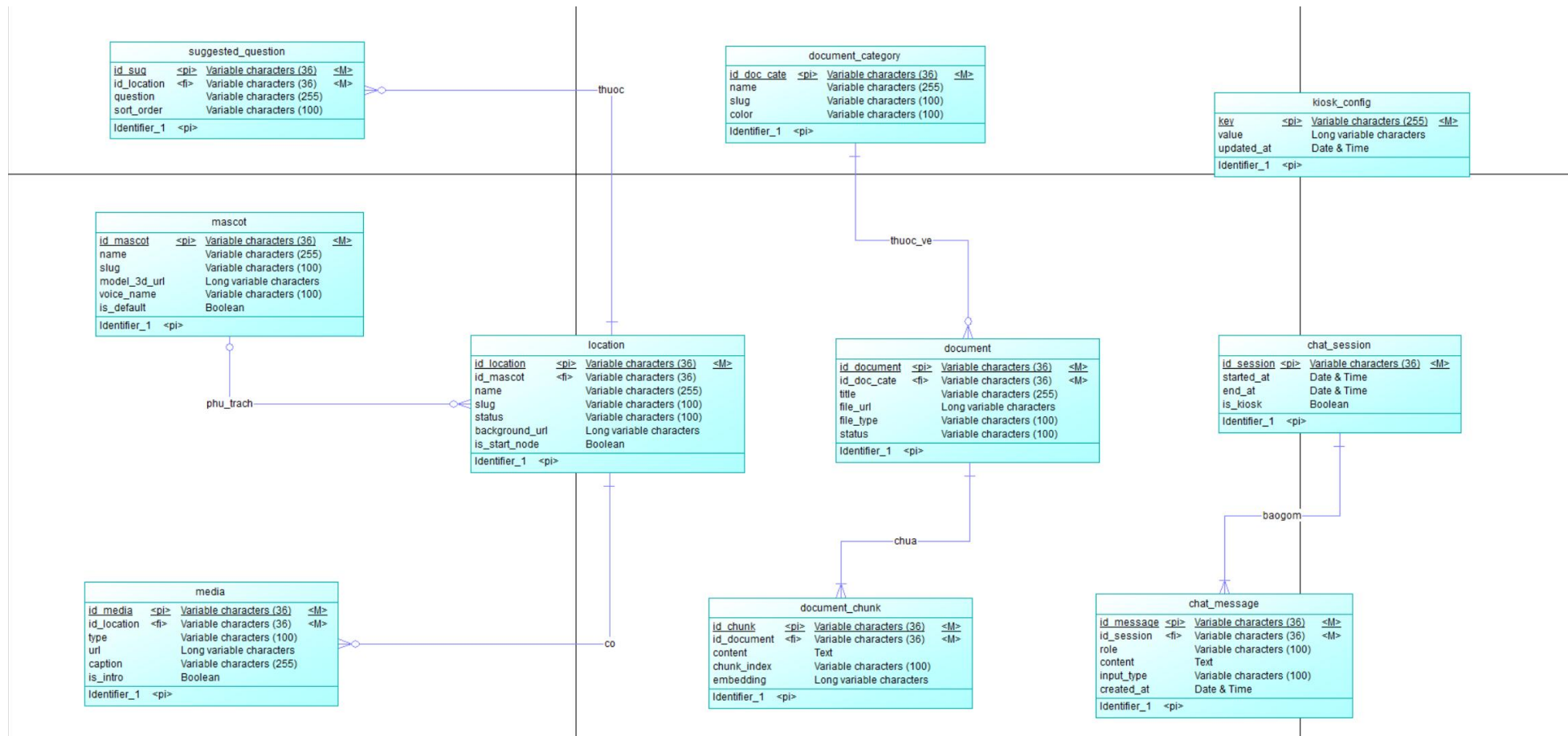
3.3 Thiết kế cơ sở dữ liệu

Cơ sở dữ liệu của hệ thống được thiết kế nhằm lưu trữ dữ liệu phục vụ tham quan, hỏi đáp bằng AI và quản trị nội dung. Các nhóm dữ liệu chính gồm địa điểm, media, nhân vật hướng dẫn 3D, tài liệu RAG, phiên trò chuyện và cấu hình kiosk. Để mô tả cấu trúc dữ liệu, đề tài trình bày ERD ở ba mức: CDM, LDM và PDM, tương ứng với mức quan niệm, mức logic và mức vật lý triển khai trong PostgreSQL/Supabase.

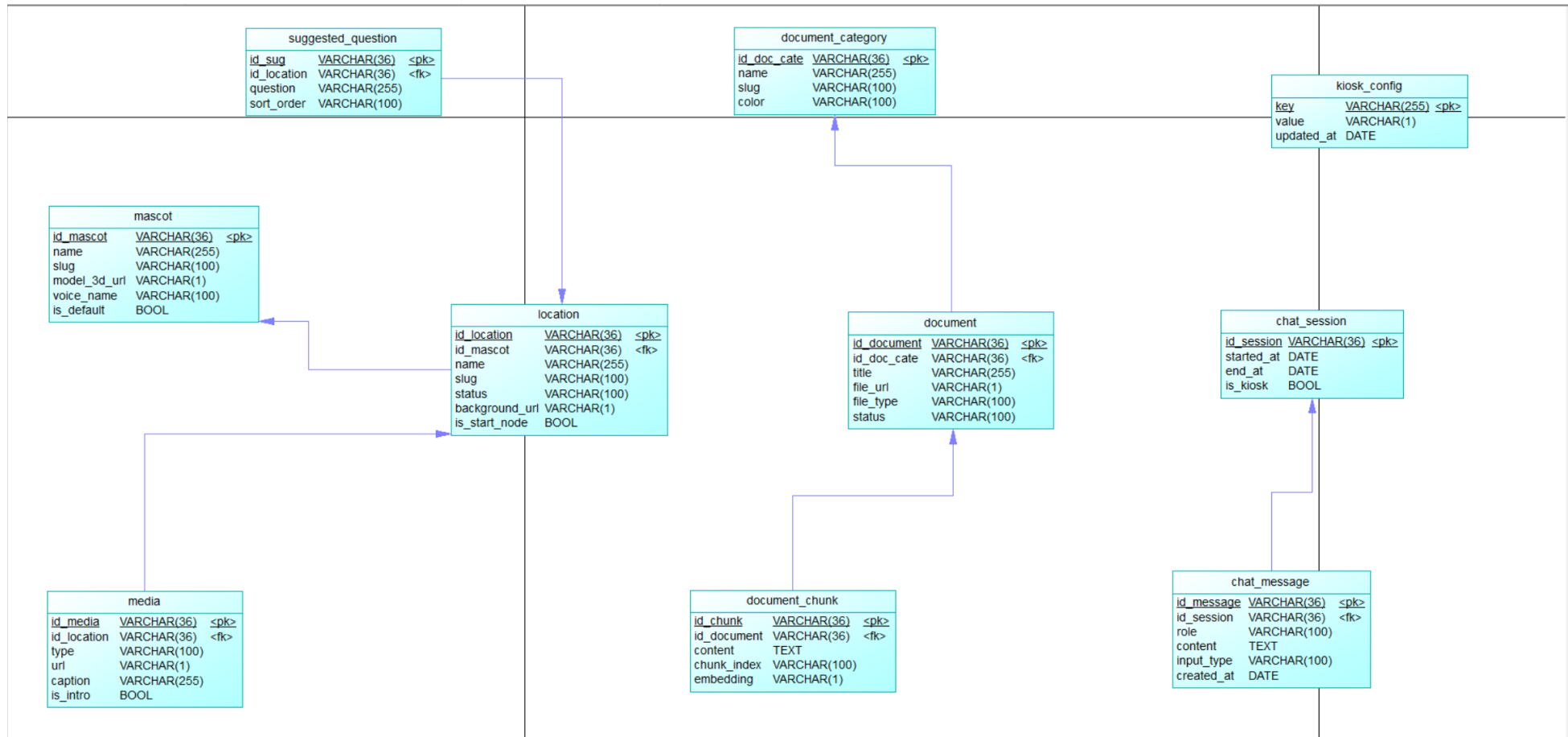
3.3.1 Sơ đồ ERD (Entity Relationship Diagram)



Hình 3.4. Mô hình dữ liệu mức quan niệm CDM



Hình 3.5. Mô hình dữ liệu mức logic LDM



Hình 3.6. Mô hình dữ liệu mức vật lý PDM

3.3.2 Mô tả các bảng chính

Dựa trên mô hình dữ liệu đã thiết kế, cơ sở dữ liệu của hệ thống gồm các bảng chính phục vụ ba nhóm chức năng: quản lý nội dung tham quan, quản lý tài liệu RAG và lưu trữ dữ liệu vận hành. Vai trò của các bảng chính được trình bày trong Bảng 3.7.

Bảng 3.7. Mô tả các bảng chính trong cơ sở dữ liệu

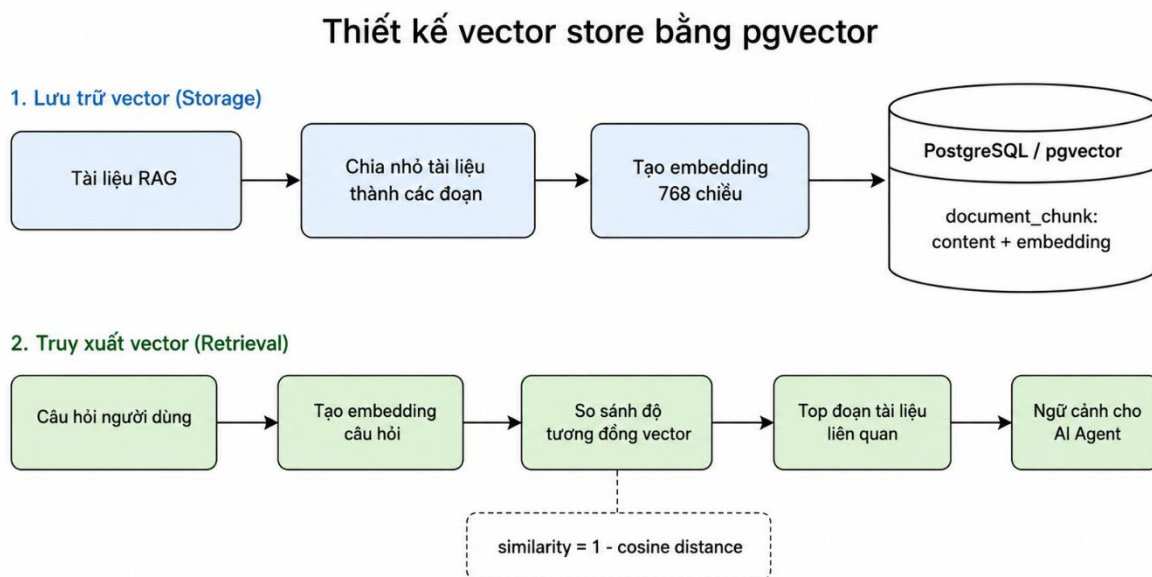
Tên bảng	Vai trò
mascot	Lưu thông tin nhân vật hướng dẫn 3D, bao gồm tên, mã định danh, đường dẫn mô hình 3D, giọng nói và trạng thái mặc định.
location	Lưu thông tin các địa điểm trong khuôn viên như tên địa điểm, mã định danh, trạng thái, ảnh nền 360 độ và nhân vật hướng dẫn được gán.
media	Lưu hình ảnh, video hoặc media liên quan đến từng địa điểm, phục vụ quá trình giới thiệu trực quan.
suggested_question	Lưu các câu hỏi gợi ý theo từng địa điểm, giúp người dùng dễ bắt đầu tương tác với AI Agent.
document_category	Lưu danh mục tài liệu, dùng để phân loại tài liệu phục vụ cho kho tri thức RAG.
document	Lưu thông tin tài liệu được tải lên hệ thống như tiêu đề, đường dẫn tệp, loại tệp, trạng thái xử lý và danh mục tương ứng.
document_chunk	Lưu các đoạn nội dung được tách từ tài liệu, kèm vector embedding để phục vụ truy vấn ngữ nghĩa bằng pgvector.
chat_session	Lưu thông tin phiên trò chuyện của người dùng trong quá trình sử dụng hệ thống.
chat_message	Lưu nội dung tin nhắn giữa người dùng và AI Agent, bao gồm vai trò, nội dung, kiểu nhập liệu và thời điểm tạo.
kiosk_config	Lưu các cấu hình vận hành kiosk như thiết lập giao diện, cache hoặc các tham số hệ thống cần điều chỉnh.

Từ Bảng 3.7 có thể thấy các bảng trong cơ sở dữ liệu được tổ chức theo từng nhóm chức năng rõ ràng. Nhóm location, media, mascot và suggested_question phục vụ dữ liệu tham quan. Nhóm document_category, document và document_chunk phục vụ kho tri thức RAG, còn chat_session, chat_message và kiosk_config hỗ trợ lưu trữ lịch sử tương tác và cấu hình vận hành. Cách tổ chức này giúp dữ liệu dễ quản lý, truy xuất và mở rộng trong quá trình phát triển hệ thống.

3.3.3 Thiết kế vector store bằng pgvector

Trong hệ thống RAG, vector store được sử dụng để lưu trữ các đoạn tài liệu dưới dạng vector embedding. Cách lưu trữ này giúp hệ thống tìm kiếm theo mức độ tương đồng ngữ nghĩa thay vì chỉ dựa trên sự trùng khớp từ khóa.

Trong đề tài, vector store được thiết kế dựa trên bảng `document_chunk`. Mỗi bản ghi tương ứng với một đoạn nội dung được tách từ tài liệu gốc, gồm nội dung văn bản, thứ tự đoạn, metadata và trường embedding. Trường embedding lưu vector 768 chiều, biểu diễn ý nghĩa của đoạn tài liệu dưới dạng số để pgvector có thể thực hiện truy vấn ngữ nghĩa. Quá trình lưu trữ và truy vấn vector được minh họa ở Hình 3.7.



Hình 3.7. Minh họa lưu trữ và truy vấn vector bằng pgvector

Từ Hình 3.7 có thể thấy tài liệu RAG được chia thành các đoạn nhỏ, sau đó tạo embedding và lưu vào PostgreSQL/pgvector. Khi người dùng đặt câu hỏi, câu hỏi cũng được chuyển thành embedding để so sánh với các vector đã lưu. Những đoạn tài liệu có độ tương đồng cao nhất sẽ được chọn làm ngữ cảnh cho AI Agent.

Trong phạm vi đề tài, kho tri thức RAG được tổ chức theo hướng dùng chung cho toàn hệ thống. Cách thiết kế này phù hợp với mục tiêu hỏi đáp tổng quát về Đại học Trà Vinh và các ngành đào tạo, đồng thời giúp việc quản lý tài liệu trong trang quản trị đơn giản hơn. Khi cần cập nhật nội dung, người quản trị chỉ cần bổ sung hoặc xử lý lại tài liệu trong một kho tri thức chung, thay vì phải quản lý riêng theo từng địa điểm.

3.3.4 Thiết kế lưu trữ media trên Cloudflare R2

Trong hệ thống, các tệp media như ảnh 360 độ, hình ảnh, video, tài liệu RAG và audio không được lưu trực tiếp trong cơ sở dữ liệu. Thay vào đó, Cloudflare R2 được sử dụng để lưu trữ nội dung tệp, còn PostgreSQL chỉ lưu các thông tin mô tả như metadata, loại tệp và đường dẫn tham chiếu.

Các tệp trên R2 được tổ chức theo cấu trúc thư mục logic để thuận tiện cho việc quản lý. Ảnh 360 độ được lưu theo từng địa điểm, media minh họa được nhóm theo nội

dung liên quan, còn tài liệu RAG được phân loại theo danh mục tài liệu. Cách tổ chức này giúp hệ thống dễ quản lý tài nguyên media mà không làm tăng dung lượng lưu trữ của cơ sở dữ liệu quan hệ.

Khi người quản trị tải lên media hoặc tài liệu từ trang admin, backend tiếp nhận tệp, tạo khóa lưu trữ phù hợp, upload tệp lên R2 và ghi lại URL vào cơ sở dữ liệu. Khi người dùng cần xem nội dung, frontend sử dụng URL đã lưu để tải tệp từ R2 và hiển thị trên giao diện. Cách thiết kế này giúp tách biệt dữ liệu tệp với dữ liệu nghiệp vụ, đồng thời thuận tiện hơn khi cần mở rộng kho media của hệ thống.

3.4 Thiết kế hệ thống RAG và AI Agent

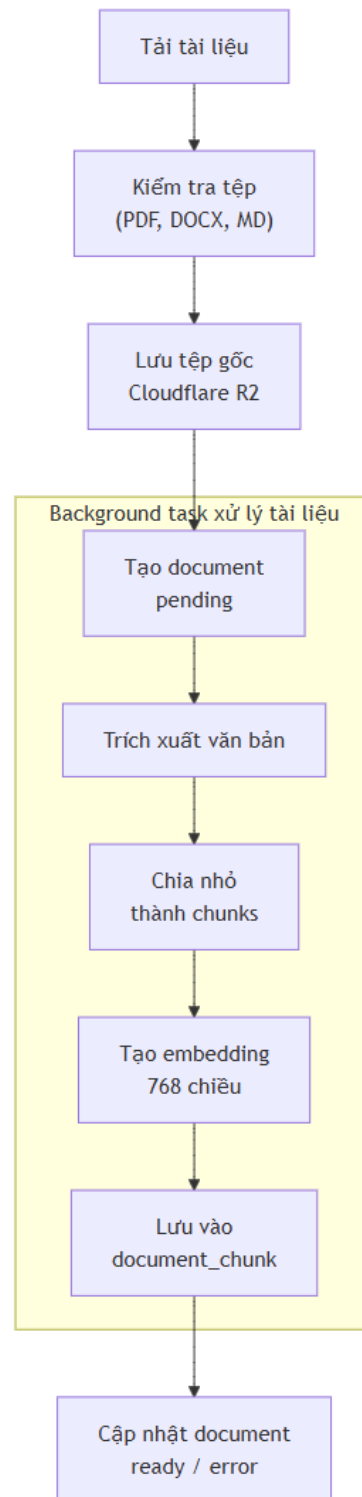
Hệ thống RAG và AI Agent là thành phần trung tâm giúp hệ thống trả lời câu hỏi dựa trên kho tri thức đã thu thập, đồng thời có thể điều khiển một số thành phần giao diện khi cần. Trong đề tài, RAG được dùng để truy xuất các đoạn tài liệu liên quan, còn AI Agent sử dụng kết quả truy xuất này kết hợp với Function Calling để tạo câu trả lời hoặc đề xuất hành động giao diện.

3.4.1 Pipeline nhập liệu tài liệu

Pipeline nhập liệu tài liệu được thiết kế để chuyển tài liệu gốc thành các đoạn nội dung có thể truy vấn bằng vector. Khi người quản trị tải tài liệu lên từ trang admin, backend kiểm tra định dạng tệp, dung lượng tệp và danh mục tài liệu. Tệp gốc được upload lên Cloudflare R2, đồng thời cơ sở dữ liệu tạo một bản ghi tài liệu với trạng thái ban đầu là đang chờ xử lý.

Sau khi bản ghi tài liệu được tạo, hệ thống sử dụng tác vụ nền để thực hiện các bước xử lý nặng như trích xuất văn bản, chia nhỏ nội dung, tạo embedding và lưu kết quả vào bảng document_chunk. Cách thiết kế này giúp API tải lên phản hồi nhanh hơn, không bắt người quản trị chờ toàn bộ quá trình xử lý hoàn tất trong cùng một request.

Các đoạn tài liệu được chia theo hướng giữ lại ngữ cảnh nội dung, chẳng hạn ưu tiên cấu trúc tiêu đề, đoạn văn và bảng. Nhờ đó, khi truy vấn RAG, mỗi đoạn tài liệu vẫn giữ được ý nghĩa tương đối đầy đủ để làm ngữ cảnh cho AI Agent. Pipeline nhập liệu tài liệu vào kho tri thức RAG được minh họa ở Hình 3.8.



Hình 3.8. Pipeline nhập liệu tài liệu vào kho tri thức RAG

3.4.2 Pipeline truy vấn và sinh câu trả lời

Khi người dùng đặt câu hỏi, backend tiếp nhận nội dung câu hỏi cùng ngữ cảnh hiện tại của giao diện, chẳng hạn địa điểm đang tham quan và lịch sử hội thoại gần nhất. Câu hỏi được chuyển thành vector embedding, sau đó hệ thống tìm kiếm các đoạn tài

liệu liên quan trong vector store bằng pgvector. Các đoạn có độ tương đồng cao nhất được chọn làm ngữ cảnh cho mô hình ngôn ngữ.

Sau bước truy xuất, backend xây dựng prompt gồm câu hỏi của người dùng, lịch sử hội thoại, ngữ cảnh địa điểm, danh sách địa điểm hợp lệ và các đoạn tài liệu liên quan. Prompt này được gửi đến mô hình ngôn ngữ để sinh câu trả lời. Kết quả trả về có thể bao gồm nội dung trả lời, nguồn tài liệu liên quan và các lời gọi hàm nếu mô hình xác định cần thực hiện hành động giao diện.

Trong hệ thống, kho tri thức được tổ chức theo hướng dùng chung cho toàn bộ quá trình hỏi đáp. Thiết kế này phù hợp với đề tài vì phần lớn câu hỏi của người dùng xoay quanh thông tin chung về Đại học Trà Vinh, ngành đào tạo, tuyển sinh, học phí, học bổng và các quy chế liên quan.

3.4.3 Thiết kế AI Agent với Function Calling

AI Agent trong đề tài không chỉ sinh câu trả lời dạng văn bản, mà còn có thể đề xuất các hành động giao diện thông qua Function Calling. Các công cụ được khai báo cho mô hình gồm hai nhóm chính: nhóm truy xuất dữ liệu và nhóm điều khiển giao diện. Nhờ đó, AI Agent có thể kết hợp giữa trả lời câu hỏi, tìm kiếm bổ sung trong kho tri thức và hỗ trợ người dùng thao tác trên giao diện tham quan.

Các công cụ Function Calling của AI Agent được trình bày trong Bảng 3.8.

Bảng 3.8. Các công cụ Function Calling của AI Agent

Công cụ	Nhóm	Vai trò
search_documents	Data tool	Tìm kiếm thêm thông tin trong kho tài liệu RAG khi ngữ cảnh ban đầu chưa đủ để trả lời.
navigate_to	UI tool	Yêu cầu frontend điều hướng đến một địa điểm trong khuôn viên dựa trên slug hợp lệ.
show_media	UI tool	Yêu cầu frontend mở hình ảnh hoặc video liên quan đến địa điểm hiện tại hoặc địa điểm được điều hướng.
toggle_map	UI tool	Yêu cầu frontend mở hoặc đóng bản đồ khuôn viên khi người dùng cần xem vị trí hoặc đường đi.

Với search_documents, backend thực hiện thêm một lượt truy xuất RAG bằng câu truy vấn do mô hình đề xuất, sau đó gọi lại mô hình để sinh câu trả lời cuối cùng. Với các UI tool như navigate_to, show_media và toggle_map, backend không trực tiếp thay đổi giao diện mà trả về các tool actions cho frontend. Frontend tiếp nhận các hành động này và cập nhật giao diện tương ứng, chẳng hạn điều hướng địa điểm, mở media hoặc hiển thị bản đồ theo yêu cầu của người dùng.

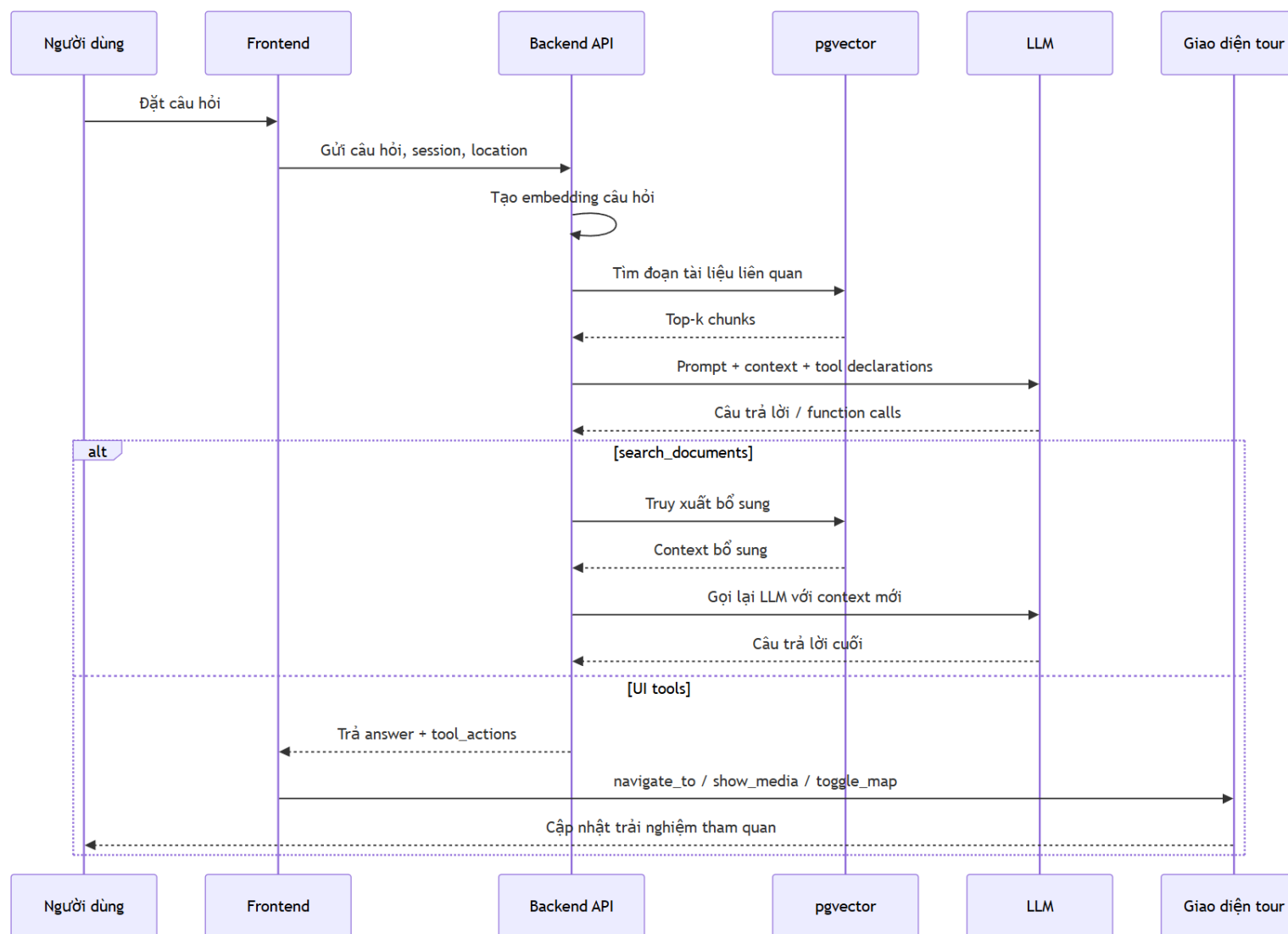
3.4.4 Cơ chế phối hợp giữa AI Agent và giao diện

Cơ chế phối hợp giữa AI Agent và giao diện được thiết kế theo hướng backend quyết định nội dung trả lời và hành động cần thực hiện, còn frontend chịu trách nhiệm thực thi các hành động đó trên giao diện. Cách tách biệt này giúp AI Agent không phụ thuộc trực tiếp vào chi tiết triển khai giao diện, đồng thời frontend vẫn kiểm soát được thứ tự điều hướng, mở media, mở bản đồ và phát âm thanh.

Đối với chế độ streaming, hệ thống xử lý theo hướng collect-then-decide. Backend nhận phản hồi đầy đủ từ mô hình ở lượt đầu để xác định có tool call hay không. Nếu mô hình gọi công cụ tìm kiếm tài liệu, backend thực hiện truy xuất bổ sung rồi mới stream câu trả lời cuối cùng. Nếu mô hình gọi công cụ giao diện, backend gửi các tool actions về frontend để giao diện cập nhật theo đúng ngữ cảnh.

Ở phía frontend, tool actions được xử lý theo thứ tự phù hợp. Nếu AI Agent vừa yêu cầu điều hướng vừa yêu cầu mở media, frontend sẽ ưu tiên điều hướng đến địa điểm trước, sau đó mới mở media. Với các phản hồi có giọng nói, hành động giao diện có thể được trì hoãn cho đến khi audio phát xong để trải nghiệm tham quan tự nhiên hơn.

Thiết kế này giúp AI Agent kết hợp được hai khả năng chính: trả lời dựa trên kho tri thức và điều khiển giao diện tham quan thông qua Function Calling. Cơ chế phối hợp giữa AI Agent và giao diện được minh họa ở Hình 3.9.



Hình 3.9. Cơ chế phối hợp giữa AI Agent và giao diện thông qua Function Calling

3.5 Thiết kế bản đồ và dẫn đường

Hệ thống bản đồ được thiết kế nhằm hỗ trợ người dùng hình dung vị trí các địa điểm trong khuôn viên và đường đi giữa các khu vực. Thay vì chỉ hiển thị bản đồ như một hình ảnh tĩnh, hệ thống tách riêng phần bản đồ trực quan và dữ liệu đường đi để có thể tính toán, hiển thị và điều hướng linh hoạt.

3.5.1 Mô hình waypoint graph

Bản đồ khuôn viên được biểu diễn dưới dạng waypoint graph. Trong mô hình này, các điểm quan trọng trên bản đồ được xem là các nút, còn các đoạn đường có thể di chuyển giữa hai nút được xem là các cạnh. Mô hình waypoint graph và đường đi trên bản đồ khuôn viên được minh họa ở Hình 3.10.



Hình 3.10. Mô hình waypoint graph và đường đi trên bản đồ khuôn viên

Từ Hình 3.10 có thể thấy các waypoint được đặt tại những vị trí quan trọng trên bản đồ, chẳng hạn công, giao điểm đường đi và các địa điểm cần tham quan. Mỗi waypoint có thể lưu các thông tin như mã nút, slug địa điểm, tọa độ, nhãn hiển thị, loại nút và trạng thái hoạt động, còn các cạnh thể hiện khả năng di chuyển giữa hai nút và có thể gắn chi phí đi kèm để phục vụ tính toán đường đi.

Tọa độ của waypoint được chuẩn hóa theo hệ tọa độ 100x100, với gốc tọa độ ở góc trên bên trái của bản đồ. Cách biểu diễn này giúp backend và frontend sử dụng cùng một hệ tọa độ khi tính toán và hiển thị đường đi. Backend sử dụng waypoint graph để chạy thuật toán tìm đường, trong khi frontend sử dụng tọa độ trả về để vẽ tuyến đường lên bản đồ trong quá trình tham quan.

3.5.2 Tìm đường bằng thuật toán A^*

Khi người dùng chọn một địa điểm đích, hệ thống cần xác định tuyến đường từ địa điểm hiện tại đến địa điểm cần đến. Backend tiếp nhận slug của điểm bắt đầu và điểm đích, sau đó ánh xạ các slug này sang nút tương ứng trong waypoint graph. Thuật toán A^* được sử dụng để tìm đường đi dựa trên chi phí di chuyển giữa các nút và khoảng cách ước lượng đến đích.

Kết quả tìm đường gồm trạng thái tìm thấy đường, danh sách các nút trên tuyến đường, danh sách tọa độ tương ứng, tổng chi phí, số nút đã duyệt và thời gian tính toán. Trong một số trường hợp cần minh họa quá trình tìm đường, hệ thống có thể trả thêm các bước duyệt của thuật toán để phục vụ giao diện mô phỏng A^* .

3.5.3 Lưu trữ dữ liệu đường đi và waypoint

Dữ liệu waypoint graph được lưu trữ dưới dạng JSON nhằm thuận tiện cho việc chỉnh sửa, đồng bộ và sử dụng lại giữa các thành phần của hệ thống. Tập dữ liệu graph chứa danh sách nút, danh sách cạnh, hệ tọa độ sử dụng và một số thông tin mô tả bản đồ. Cách lưu trữ này phù hợp với bài toán của đề tài vì số lượng địa điểm và waypoint không quá lớn, nhưng vẫn cần đủ linh hoạt để cập nhật khi bản đồ thay đổi.

Ở phía backend, dữ liệu graph được đọc và xây dựng thành danh sách kề để phục vụ thuật toán A^* . Đối với frontend, dữ liệu tọa độ giúp xác định vị trí các điểm trên bản đồ và hiển thị marker tương ứng. Một số đường đi đã tính trước có thể được lưu làm dữ liệu dự phòng, tuy nhiên luồng chính của hệ thống vẫn ưu tiên gọi API để tính đường theo waypoint graph hiện tại.

3.5.4 Hiện thị đường đi trên giao diện

Trên giao diện, bản đồ được hiển thị bằng hình ảnh nền, còn đường đi và marker được vẽ trên một lớp SVG đặt chồng lên bản đồ. SVG sử dụng cùng hệ tọa độ 100x100 với waypoint graph, nhờ đó các tọa độ do backend trả về có thể được chuyển trực tiếp thành các điểm của polyline.

Khi có kết quả tìm đường, frontend nối các tọa độ theo thứ tự để tạo thành tuyến đường. Tuyến đường được hiển thị bằng màu nổi bật, kèm hiệu ứng vẽ đường để người dùng dễ theo dõi quá trình di chuyển. Ngoài tuyến đường chính, giao diện cũng có thể hiển thị điểm hiện tại, điểm đích và các địa điểm đang hoạt động trên bản đồ.

3.5.5 Tích hợp điều hướng với AI Agent

Chức năng dẫn đường không chỉ được kích hoạt khi người dùng chọn trực tiếp trên bản đồ, mà còn có thể được kích hoạt thông qua AI Agent. Khi người dùng hỏi về một địa điểm hoặc yêu cầu đi đến một khu vực cụ thể, AI Agent có thể sinh hành động điều hướng thông qua công cụ `navigate_to`. Frontend tiếp nhận hành động này, mở bản đồ, gọi API tìm đường và hiển thị tuyến đường đến địa điểm tương ứng.

Cách thiết kế này giúp kết nối chức năng hỏi đáp với trải nghiệm tham quan trực quan. Người dùng có thể bắt đầu bằng một câu hỏi tự nhiên, sau đó hệ thống không chỉ trả lời bằng văn bản mà còn điều hướng đến địa điểm liên quan. Nhờ đó, bản đồ trở thành một thành phần tương tác trong luồng tham quan, thay vì chỉ minh họa vị trí.

3.6 Thiết kế giao diện người dùng

Giao diện người dùng của hệ thống được thiết kế theo hai nhóm chính: giao diện Kiosk Tour dành cho người dùng tham quan và giao diện Admin Panel dành cho người quản trị. Giao diện Kiosk Tour tập trung vào trải nghiệm trực quan, thao tác chạm và tương tác với AI Agent, còn Admin Panel tập trung vào quản lý dữ liệu, tài liệu, media, nhân vật hướng dẫn 3D và cấu hình vận hành hệ thống.

3.6.1 Giao diện Kiosk Tour

Giao diện Kiosk Tour là màn hình chính mà người dùng sử dụng trong quá trình tham quan. Giao diện được thiết kế theo hướng toàn màn hình, lấy ảnh 360 độ của địa điểm làm nền chính. Trên lớp nền này, hệ thống hiển thị các thành phần tương tác như nhân vật hướng dẫn 3D, khung chat, bản đồ khuôn viên, bảng thông tin địa điểm và các nút điều khiển giọng nói.

Khi người dùng bắt đầu trải nghiệm, hệ thống hiển thị màn hình bắt đầu để chờ các tài nguyên quan trọng như ảnh 360 độ và mô hình nhân vật 3D được tải xong. Sau khi người dùng chạm để bắt đầu, giao diện chuyển sang chế độ tham quan, trong đó người dùng có thể quan sát không gian, đặt câu hỏi cho AI Agent, mở bản đồ, xem media liên quan hoặc di chuyển sang địa điểm khác.

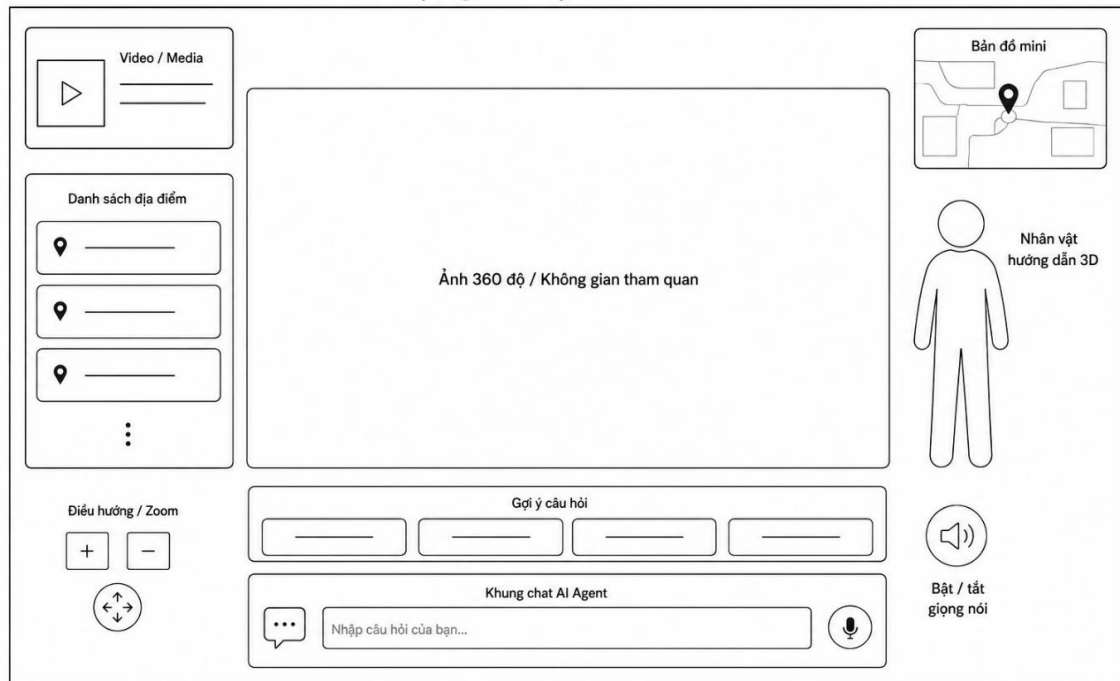
Các thành phần chính của giao diện Kiosk Tour được trình bày trong Bảng 3.9.

Bảng 3.9. Các thành phần chính của giao diện Kiosk Tour

Thành phần	Vai trò thiết kế
Ảnh 360 độ	Là nền chính của trải nghiệm tham quan, giúp người dùng quan sát không gian địa điểm.
Nhân vật hướng dẫn	Tạo cảm giác có người hướng dẫn, phản hồi theo trạng thái suy nghĩ hoặc nói của AI Agent.
Khung chat	Cho phép người dùng nhập câu hỏi, xem câu trả lời và tương tác với AI Agent.
Bản đồ khuôn viên	Hiển thị vị trí hiện tại, các địa điểm có thể đến và tuyến đường dẫn đường.
Bảng thông tin và media	Hiển thị hình ảnh, video hoặc nội dung liên quan đến địa điểm đang tham quan.
Điều khiển giọng nói	Cho phép bật hoặc tắt phát âm thanh khi AI Agent trả lời.

Bố cục tổng quát của giao diện Kiosk Tour được thể hiện ở Hình 3.11.

Bố cục giao diện Kiosk Tour



Hình 3.11. Bố cục các thành phần chính trong giao diện Kiosk Tour

Thiết kế này giúp người dùng luôn quan sát không gian tham quan ở trung tâm, trong khi các thành phần hỗ trợ được bố trí xung quanh để không che khuất nội dung chính. Các trạng thái như đang tải, chuyển địa điểm, lỗi kết nối hoặc cảnh báo không hoạt động được hiển thị bằng lớp phủ toàn màn hình để người dùng dễ nhận biết.

3.6.2 Giao diện Admin Panel

Admin Panel được thiết kế để người quản trị cập nhật, kiểm soát và vận hành nội dung của hệ thống mà không cần can thiệp trực tiếp vào mã nguồn. Giao diện quản trị sử dụng bố cục dashboard với thanh điều hướng, khu vực nội dung chính và các trang chức năng riêng biệt, giúp việc quản lý dữ liệu trở nên trực quan và thuận tiện hơn.

Các nhóm chức năng chính trong Admin Panel gồm quản lý địa điểm, quản lý tài liệu RAG, quản lý thư viện media, quản lý nhân vật hướng dẫn 3D, quản lý cache và cấu hình kiosk. Mỗi nhóm chức năng được tách thành một trang riêng, giúp người quản trị dễ tìm kiếm, cập nhật và kiểm tra trạng thái dữ liệu. Các nhóm chức năng trong giao diện Admin Panel được trình bày trong Bảng 3.10.

Bảng 3.10. Các nhóm chức năng trong giao diện Admin Panel

Nhóm giao diện	Nội dung quản lý
Tổng quan	Theo dõi số liệu vận hành, phiên tham quan và trạng thái dữ liệu.
Địa điểm	Cập nhật tên, mô tả, ảnh 360 độ, câu hỏi gợi ý, trạng thái hoạt động và bản đồ.
Tài liệu	Tải tài liệu RAG, phân loại tài liệu và theo dõi trạng thái xử lý.
Media	Quản lý hình ảnh, video và nội dung media gắn với địa điểm.
Nhân vật hướng dẫn 3D	Cấu hình mô hình 3D, giọng đọc và thông tin liên quan đến nhân vật hướng dẫn.
Cache	Theo dõi và tạo lại các dữ liệu cache phục vụ phản hồi nhanh.
Cấu hình	Thiết lập thời gian chờ, điểm bắt đầu mặc định, chế độ kiosk và âm thanh mặc định.

Thiết kế Admin Panel ưu tiên tính rõ ràng và khả năng bảo trì. Các thao tác quản trị quan trọng được thực hiện thông qua API backend có xác thực, trong khi frontend tự động gửi token phiên đăng nhập khi gọi các API quản trị. Cách tổ chức này giúp tách biệt giao diện người dùng tham quan và giao diện quản trị, đồng thời đảm bảo dữ liệu hệ thống được cập nhật một cách có kiểm soát.

3.6.3 Luồng trải nghiệm người dùng

Luồng trải nghiệm bắt đầu từ màn hình chờ, sau đó chuyển sang giao diện tham quan khi các tài nguyên chính đã sẵn sàng. Người dùng có thể quan sát địa điểm, nghe

giới thiệu, đặt câu hỏi, xem thông tin bổ sung hoặc chọn điểm đến trên bản đồ. Khi yêu cầu điều hướng, hệ thống hiển thị bản đồ, tính toán tuyến đường và chuyển sang địa điểm mới sau khi hoàn tất hiệu ứng dẫn đường.

Đối với các tương tác với AI Agent, người dùng có thể đặt câu hỏi bằng văn bản hoặc giọng nói. AI Agent trả lời bằng văn bản, có thể kèm âm thanh và các hành động giao diện như mở bản đồ, điều hướng đến địa điểm hoặc hiển thị media liên quan. Nhờ đó, trải nghiệm người dùng không chỉ là đọc thông tin, mà còn là quá trình tương tác kết hợp giữa hỏi đáp và tham quan trực quan.

3.7 Thiết kế vận hành kiosk và quản trị hệ thống

Do hệ thống được định hướng sử dụng trong môi trường kiosk, thiết kế vận hành cần đảm bảo giao diện có thể hoạt động ổn định trong thời gian dài, hạn chế thao tác ngoài ý muốn và tự phục hồi trong một số tình huống lỗi thường gặp. Bên cạnh đó, các chức năng quản trị cần được bảo vệ bằng xác thực để tránh việc người dùng không có quyền truy cập thay đổi dữ liệu hệ thống.

3.7.1 Tự động reset khi không hoạt động

Trong môi trường kiosk, phiên tham quan có thể bị bỏ dở khi người dùng rời khỏi màn hình. Để tránh người dùng tiếp theo tiếp tục phiên cũ, hệ thống theo dõi trạng thái không hoạt động. Các sự kiện như chạm màn hình, nhấp chuột, nhấn phím hoặc cuộn trang được xem là dấu hiệu người dùng vẫn đang tương tác.

Khi không có tương tác trong một khoảng thời gian nhất định, hệ thống hiển thị cảnh báo đếm ngược. Nếu người dùng tiếp tục thao tác, cảnh báo được đóng và bộ đếm được khởi động lại. Nếu hết thời gian cảnh báo, hệ thống thực hiện reset bằng cách tải lại trang, từ đó xóa trạng thái phiên hiện tại, giải phóng tài nguyên WebGL và đưa kiosk về trạng thái ban đầu.

3.7.2 Khóa thao tác ngoài kiosk

Khi chạy ở chế độ kiosk, hệ thống cần hạn chế các thao tác không phục vụ quá trình tham quan. Frontend thiết kế cơ chế Gesture Lock để chặn một số thao tác như mở menu chuột phải, làm mới trang bằng phím tắt, đóng tab bằng tổ hợp phím hoặc vuốt từ cạnh màn hình trên thiết bị cảm ứng.

Cơ chế này được bật theo cấu hình môi trường kiosk, giúp hệ thống vẫn có thể chạy linh hoạt trong chế độ phát triển hoặc kiểm thử. Khi triển khai trên thiết bị kiosk

thật, việc hạn chế các thao tác ngoài luồng giúp giảm khả năng người dùng thoát khỏi trải nghiệm tham quan hoặc làm gián đoạn trạng thái giao diện.

3.7.3 Xử lý lỗi mạng và phục hồi giao diện

Hệ thống phụ thuộc vào backend để lấy danh sách địa điểm, dữ liệu media, bản đồ và phản hồi từ AI Agent. Vì vậy, frontend được thiết kế cơ chế xử lý lỗi mạng ở các bước tải dữ liệu quan trọng. Khi không thể kết nối đến backend, giao diện hiển thị trạng thái mất kết nối hoặc đang thử kết nối lại và tự động thử lại trong một số lần nhất định.

Nếu sau nhiều lần thử lại hệ thống vẫn không kết nối được, giao diện chuyển sang trạng thái lỗi nghiêm trọng và cung cấp nút thử lại cho người dùng hoặc người vận hành. Ngoài lỗi mạng, hệ thống cũng có cơ chế an toàn khi tài nguyên như ảnh 360 độ hoặc mô hình 3D tải quá lâu: sau một khoảng thời gian chờ, giao diện có thể bỏ qua trạng thái chờ tải để tránh việc kiosk bị treo ở màn hình tải.

3.7.4 Xác thực và bảo vệ trang quản trị

Trang quản trị được tách khỏi giao diện Kiosk Tour và yêu cầu đăng nhập trước khi truy cập nhằm bảo vệ dữ liệu hệ thống. Ở frontend, middleware kiểm tra phiên đăng nhập khi người dùng truy cập các đường dẫn thuộc /admin. Nếu chưa đăng nhập, người dùng được chuyển đến trang đăng nhập. Trường hợp đã đăng nhập và truy cập lại trang đăng nhập, hệ thống chuyển về dashboard quản trị.

Khi gọi các API quản trị, frontend gửi kèm token đăng nhập trong header Authorization. Backend kiểm tra token này thông qua dịch vụ xác thực của Supabase trước khi cho phép thực hiện các thao tác quản trị. Các API quản trị như cập nhật địa điểm, tải tài liệu, quản lý media, cấu hình nhân vật hướng dẫn và cấu hình kiosk đều nằm sau lớp xác thực này.

Các cơ chế vận hành và bảo vệ hệ thống được tóm tắt trong Bảng 3.11.

Bảng 3.11. Tóm tắt các cơ chế vận hành và bảo vệ hệ thống

Cơ chế vận hành	Mục đích thiết kế
Idle reset	Đưa kiosk về trạng thái ban đầu khi người dùng không còn tương tác.
Gesture Lock	Hạn chế thao tác ngoài luồng như chuột phải, phím tắt hoặc vuốt cạnh màn hình.
Network recovery	Tự động thử lại khi backend chưa sẵn sàng hoặc kết nối tạm thời bị gián đoạn.
Loading safety timeout	Tránh việc giao diện bị kẹt ở trạng thái tải quá lâu.
Admin authentication	Bảo vệ các chức năng cập nhật dữ liệu và cấu hình hệ thống.

Tổng thể, thiết kế vận hành kiosk giúp hệ thống phù hợp hơn với môi trường sử dụng công cộng, nơi người dùng có thể tương tác liên tục và không phải lúc nào cũng thao tác theo đúng luồng dự kiến. Các cơ chế như tự động đặt lại trạng thái, hạn chế thao tác ngoài luồng, phục hồi lỗi và xác thực quản trị hỗ trợ hệ thống duy trì trạng thái ổn định trong quá trình sử dụng. Đồng thời, các cơ chế này cũng góp phần giảm rủi ro phát sinh từ thao tác không mong muốn, lỗi kết nối tạm thời hoặc truy cập trái phép vào khu vực quản trị. Nhờ đó, hệ thống không chỉ đáp ứng yêu cầu hiển thị và tương tác, mà còn có khả năng vận hành an toàn hơn trong điều kiện sử dụng thực tế.

CHƯƠNG 4: XÂY DỰNG HỆ THỐNG

Sau khi hoàn thành quá trình phân tích và thiết kế, chương này trình bày quá trình xây dựng hệ thống AI Agent giới thiệu Đại học Trà Vinh và các ngành đào tạo. Nội dung chương tập trung vào môi trường phát triển, cấu trúc mã nguồn, xây dựng backend API, tích hợp hệ thống RAG và AI Agent, phát triển giao diện Kiosk Tour, giao diện Admin Panel, cũng như các cơ chế hỗ trợ vận hành và triển khai hệ thống. Thông qua đó, chương làm rõ cách các thành phần đã thiết kế ở chương trước được hiện thực thành một hệ thống web có thể sử dụng, quản trị và mở rộng trong thực tế

4.1 Môi trường và cấu trúc triển khai dự án

4.1.1 Công cụ và phiên bản sử dụng

Trong quá trình xây dựng hệ thống, dự án sử dụng nhiều công cụ và thư viện khác nhau cho frontend, backend, cơ sở dữ liệu, AI, lưu trữ media và triển khai. Các công cụ chính được lựa chọn dựa trên yêu cầu của hệ thống như hiển thị ảnh 360 độ, tích hợp nhân vật 3D, xây dựng API, xử lý RAG, kết nối cơ sở dữ liệu và quản lý tệp media. Danh sách các công cụ và phiên bản sử dụng được trình bày trong Bảng 4.1.

Bảng 4.1. Công cụ và phiên bản sử dụng trong dự án

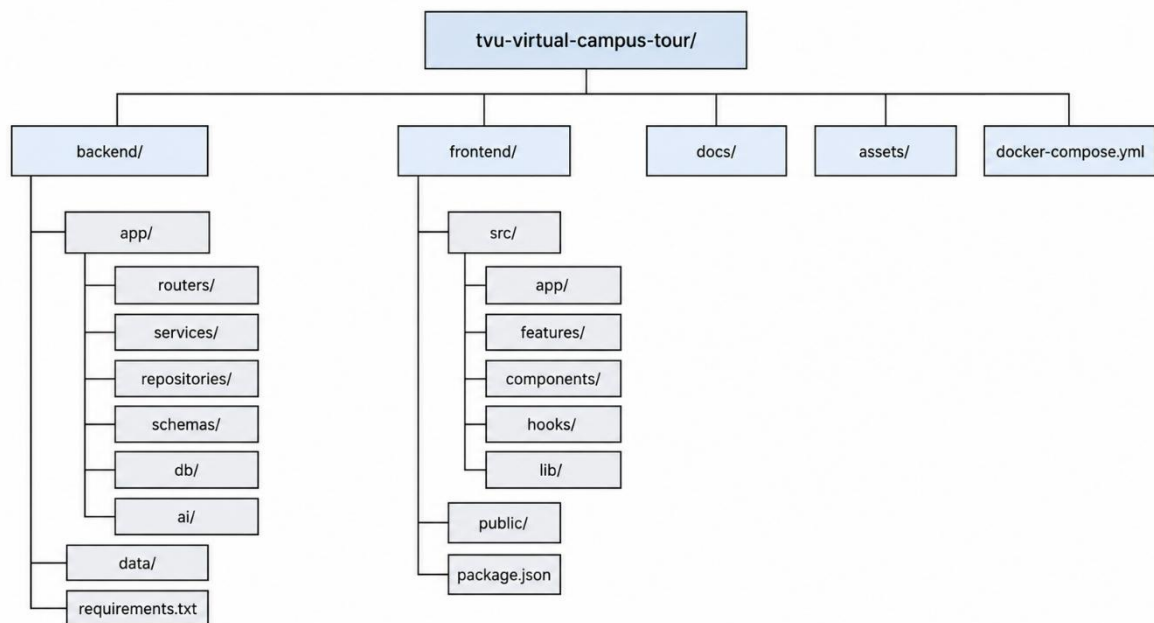
Nhóm	Công cụ / thư viện	Phiên bản / ghi chú
Ngôn ngữ frontend	TypeScript	Phiên bản 5
Framework frontend	Next.js	16.2.4
Thư viện giao diện	React, React DOM	19.2.4
Quản lý trạng thái	Zustand	5.0.12
Hiệu ứng giao diện	Framer Motion	12.38.0
Hiển thị ảnh 360 độ	Pannellum	2.5.7
Hiển thị nhân vật 3D	Three.js, React Three Fiber, Drei	Three.js 0.184.0; React Three Fiber 9.6.0
Xác thực frontend	Supabase JS, Supabase SSR	Supabase JS 2.105.4; Supabase SSR 0.10.3
Ngôn ngữ backend	Python	Từ phiên bản 3.11
Framework backend	FastAPI, Uvicorn	FastAPI 0.115.0; Uvicorn 0.30.0
Cơ sở dữ liệu	Supabase PostgreSQL	Sử dụng PostgreSQL và extension pgvector
ORM và kết nối CSDL	SQLAlchemy, asyncpg	SQLAlchemy 2.0.35; asyncpg 0.29.0
Vector store	pgvector	0.3.2
AI và embedding	Google Gemini	Sử dụng Gemini cho chat, embedding và TTS

Nhóm	Công cụ / thư viện	Phiên bản / ghi chú
Lưu trữ media	Cloudflare R2	Sử dụng thông qua API tương thích S3
Xử lý tài liệu	pdfplumber, python-docx	Trích xuất nội dung từ PDF và DOCX
Container backend	Docker	Backend có Dockerfile và docker-compose phục vụ triển khai

Các phiên bản trên được lấy từ cấu hình mã nguồn của dự án, cụ thể là package.json ở frontend và requirements.txt, pyproject.toml ở backend. Việc xác định rõ công cụ và phiên bản sử dụng giúp quá trình cài đặt, kiểm thử và triển khai lại hệ thống hạn chế lỗi phát sinh do khác biệt môi trường.

4.1.2 Cấu trúc thư mục dự án

Mã nguồn dự án được tổ chức theo hướng tách biệt frontend và backend. Cách tổ chức này giúp quá trình phát triển, kiểm thử và triển khai từng phần của hệ thống trở nên rõ ràng hơn. Cấu trúc thư mục chính của dự án được minh họa ở Hình 4.1.



Hình 4.1. Cấu trúc thư mục chính của dự án

Các thành phần chính trong cấu trúc thư mục được mô tả trong Bảng 4.2.

Bảng 4.2. Cấu trúc thư mục chính của dự án

Thư mục / tệp	Vai trò
frontend	Chứa mã nguồn giao diện web, gồm Kiosk Tour, Admin Panel và các thành phần tương tác phía trình duyệt.
frontend/src/app	Chứa các route chính của Next.js như trang tour, trang quản trị và các trang hỗ trợ bản đồ.
frontend/src/features	Chứa các module chức năng theo hướng feature-based, gồm tham quan, chat và quản trị.
frontend/src/components, frontend/src/hooks, frontend/src/lib	Chứa component dùng chung, custom hook và các hàm hỗ trợ như gọi API, kết nối Supabase, middleware xác thực.
backend	Chứa mã nguồn FastAPI backend, bao gồm API, xử lý dữ liệu, RAG, AI Agent và các service nghiệp vụ.
backend/app/routers	Định nghĩa các nhóm API như địa điểm, chat, navigation, admin và cache.
backend/app/services, backend/app/repositories, backend/app/schemas	Tổ chức backend theo lớp xử lý nghiệp vụ, lớp truy cập dữ liệu và schema request/response.
backend/app/db, backend/app/ai	Chứa cấu hình cơ sở dữ liệu, định nghĩa bảng ORM, tích hợp AI, prompt và function calling.
backend/data	Chứa dữ liệu JSON phục vụ bản đồ, waypoint graph, đường đi và một số dữ liệu cache.
assets, docs	Chứa tài nguyên thiết kế, ảnh bản đồ, tài liệu báo cáo và ghi chú triển khai.

Ở frontend, mã nguồn được tổ chức theo hướng feature-based để tách các nhóm chức năng lớn như tham quan, chat và quản trị. Ở backend, mã nguồn được tổ chức theo mô hình phân lớp gồm router, service, repository và schema. Cách tổ chức này giúp hệ thống dễ mở rộng, đồng thời giảm sự phụ thuộc trực tiếp giữa giao diện, API và lớp xử lý dữ liệu trong quá trình phát triển.

4.1.3 Cấu hình biến môi trường

Hệ thống sử dụng biến môi trường để tách các thông tin cấu hình khỏi mã nguồn. Các thông tin như khóa API, chuỗi kết nối cơ sở dữ liệu, cấu hình Supabase và

Cloudflare R2 không được ghi trực tiếp vào mã nguồn, mà được nạp thông qua tệp .env khi chạy ứng dụng.

Các nhóm biến môi trường chính của hệ thống được trình bày trong Bảng 4.3.

Bảng 4.3. Các nhóm biến môi trường chính của hệ thống

Nhóm cấu hình	Biến môi trường tiêu biểu
Kết nối frontend - backend	NEXT_PUBLIC_API_URL
Supabase frontend	NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY
Chế độ kiosk	NEXT_PUBLIC_KIOSK_MODE
Ứng dụng backend	APP_NAME, DEBUG, CORS_ORIGINS
Google Gemini	GEMINI_API_KEY, GEMINI_CHAT_MODEL, GEMINI_EMBEDDING_MODEL, GEMINI_TTS_MODEL
Cơ sở dữ liệu	DATABASE_URL
Supabase backend	SUPABASE_URL, SUPABASE_ANON_KEY, SUPABASE_SERVICE_ROLE_KEY
Cloudflare R2	R2_ENDPOINT_URL, R2_ACCESS_KEY_ID, R2_SECRET_ACCESS_KEY, R2_BUCKET_NAME, R2_PUBLIC_URL

Đối với frontend, các biến có tiền tố NEXT_PUBLIC_ có thể được sử dụng ở phía trình duyệt nên chỉ dùng cho những thông tin được phép công khai như URL API hoặc khóa anon của Supabase. Đối với backend, các khóa nhạy cảm như Gemini API key, service role key và R2 secret key chỉ được cấu hình ở môi trường server. Cách tách cấu hình này giúp hệ thống có thể chạy ở nhiều môi trường khác nhau như local, staging hoặc server triển khai, đồng thời hạn chế việc để lộ thông tin nhạy cảm trong mã nguồn.

4.2 Xây dựng Backend API

Backend của hệ thống được xây dựng bằng FastAPI, đóng vai trò cung cấp API cho giao diện Kiosk Tour, Admin Panel, hệ thống RAG, AI Agent, bản đồ dẫn đường và lưu trữ media. Trong quá trình triển khai, backend được tổ chức theo mô hình phân lớp nhằm tách phần tiếp nhận HTTP request, xử lý nghiệp vụ và truy cập cơ sở dữ liệu. Cách tổ chức này giúp mã nguồn dễ kiểm thử, dễ mở rộng và hạn chế việc đưa logic xử lý trực tiếp vào router.

4.2.1 Khởi tạo FastAPI, CORS và kết nối Supabase PostgreSQL

Ứng dụng backend được khởi tạo trong tệp main.py. Tại đây, FastAPI được cấu hình với tên ứng dụng, phiên bản, đường dẫn tài liệu API, CORS và các router chức năng. Các router chính được đăng ký gồm location, chat, navigation, admin và admin

cache. Ngoài ra, backend cũng cung cấp endpoint `/api/health` để kiểm tra trạng thái hoạt động của dịch vụ sau khi chạy hoặc triển khai.

Khi ứng dụng khởi động, backend kiểm tra cấu hình Gemini API, khởi tạo client AI nếu có khóa API hợp lệ và nạp các bộ nhớ đệm như cache câu hỏi - trả lời và cache âm thanh. Việc khởi tạo trong lifecycle của FastAPI giúp các thành phần dùng chung sẵn sàng trước khi API bắt đầu nhận request.

CORS được cấu hình dựa trên danh sách origin trong biến môi trường. Cấu hình này cho phép frontend gọi API backend trong môi trường phát triển cục bộ hoặc khi triển khai, đồng thời vẫn giới hạn các nguồn được phép truy cập. Đây là bước cần thiết vì frontend và backend của hệ thống có thể chạy ở hai origin khác nhau.

Kết nối cơ sở dữ liệu được triển khai bằng SQLAlchemy bất đồng bộ kết hợp với driver `asyncpg`. Chuỗi kết nối được lấy từ biến môi trường `DATABASE_URL`, sau đó được chuẩn hóa sang dạng `postgresql+asyncpg`. Mỗi request cần truy cập dữ liệu sẽ sử dụng một `AsyncSession` thông qua dependency `get_db`. Cách tổ chức này giúp việc truy cập cơ sở dữ liệu thống nhất và phù hợp với kiến trúc backend bất đồng bộ.

4.2.2 Tổ chức repository, service layer và schema

Backend được tổ chức theo mô hình phân lớp gồm router, service, repository và schema. Router chịu trách nhiệm tiếp nhận HTTP request, kiểm tra tham số đầu vào cơ bản và gọi service tương ứng. Service chứa logic nghiệp vụ như xử lý địa điểm, nhập liệu tài liệu, truy vấn RAG, tạo câu trả lời AI, tìm đường và lưu trữ media. Repository tập trung vào thao tác dữ liệu với cơ sở dữ liệu, giúp service không phụ thuộc trực tiếp vào câu truy vấn SQL hoặc ORM cụ thể. Schema được xây dựng bằng Pydantic để định nghĩa cấu trúc request và response.

Các lớp xử lý chính trong backend được trình bày trong Bảng 4.4.

Bảng 4.4. Tổ chức các lớp xử lý trong backend

Lớp xử lý	Thư mục / tệp tiêu biểu	Vai trò
Router	app/routers	Định nghĩa endpoint, nhận request, gọi service/repository và trả response.
Service	app/services	Chứa logic nghiệp vụ như RAG, ingest tài liệu, tìm đường, cache và lưu trữ.
Repository	app/repositories	Thực hiện truy vấn và cập nhật dữ liệu trong cơ sở dữ liệu.
Schema	app/schemas	Định nghĩa dữ liệu request/response bằng Pydantic.
Database	app/db	Cấu hình kết nối, session và định nghĩa bảng ORM.
AI module	app/ai	Chứa phân tích hợp Gemini, prompt, function calling và xử lý giọng nói.

Ví dụ, khi frontend gọi API lấy danh sách địa điểm, request được tiếp nhận tại router locations. Router gọi location_service để xử lý nghiệp vụ, service sử dụng location_repo để lấy dữ liệu từ cơ sở dữ liệu, sau đó dữ liệu được chuẩn hóa theo schema trước khi trả về frontend. Cách triển khai này giúp từng lớp có trách nhiệm rõ ràng và dễ thay đổi độc lập hơn.

4.2.3 Xây dựng API địa điểm, media và bản đồ

Nhóm API địa điểm được xây dựng để cung cấp dữ liệu cho giao diện tham quan. Endpoint /api/locations trả về danh sách các địa điểm đang hoạt động, bao gồm thông tin cơ bản, ảnh nền, câu hỏi gợi ý, nhân vật hướng dẫn 3D và các liên kết điều hướng giữa các địa điểm. Endpoint /api/locations/{slug} trả về thông tin chi tiết của một địa điểm theo slug, trong khi /api/locations/{slug}/assets trả về danh sách media như hình ảnh, video hoặc GIF liên quan đến địa điểm đó.

Đối với bản đồ khuôn viên, backend cung cấp nhóm API điều hướng. Endpoint /api/nav/graph trả về waypoint graph gồm các nút, cạnh và trạng thái hoạt động của địa điểm. Trạng thái active/inactive được đối chiếu với dữ liệu địa điểm trong cơ sở dữ liệu để frontend chỉ hiển thị và cho phép dẫn đường đến các địa điểm phù hợp. Endpoint /api/nav/path nhận điểm bắt đầu và điểm đích, sau đó gọi service tìm đường để chạy thuật toán A^* và trả về danh sách tọa độ phục vụ việc vẽ tuyến đường trên giao diện.

Các API công khai phục vụ giao diện tham quan được trình bày trong Bảng 4.5.

Bảng 4.5. Các API công khai phục vụ giao diện tham quan

Nhóm API	Endpoint tiêu biểu	Chức năng
Địa điểm	GET /api/locations	Lấy danh sách địa điểm phục vụ giao diện tour và bản đồ.
Chi tiết địa điểm	GET /api/locations/{slug}	Lấy thông tin chi tiết của một địa điểm theo slug.
Media địa điểm	GET /api/locations/{slug}/assets	Lấy hình ảnh, video hoặc GIF liên quan đến địa điểm.
Bản đồ	GET /api/nav/graph	Lấy waypoint graph và trạng thái địa điểm để hiển thị bản đồ.
Tìm đường	GET /api/nav/path	Tính tuyến đường giữa hai địa điểm bằng thuật toán A*.

Nhóm API này là nền tảng để frontend có thể tải dữ liệu ban đầu, chuyển địa điểm, hiển thị ảnh 360 độ, mở media liên quan và vẽ đường đi trên bản đồ khuôn viên.

4.2.4 Xây dựng API chat, streaming và quản trị

API chat là phần trung tâm kết nối giao diện người dùng với hệ thống AI Agent. Khi người dùng gửi câu hỏi, frontend gọi endpoint /api/chat kèm nội dung câu hỏi, thông tin phiên trò chuyện, địa điểm hiện tại và tùy chọn phản hồi. Backend sau đó chuyển request đến RAG service để truy vấn kho tri thức, tạo prompt, gọi mô hình ngôn ngữ và xử lý các hành động giao diện nếu AI Agent đề xuất.

Endpoint /api/chat hỗ trợ nhiều chế độ phản hồi. Ở chế độ mặc định cho kiosk, backend trả về kết quả dạng JSON gồm câu trả lời, thông tin nguồn, danh sách hành động giao diện và đường dẫn âm thanh nếu có TTS. Ở chế độ streaming, backend sử dụng Server-Sent Events để gửi từng phần nội dung phản hồi về frontend, phù hợp với trường hợp chỉ hiển thị văn bản. Ngoài ra, backend có endpoint tạo phiên trò chuyện /api/chat/session và endpoint TTS để tạo hoặc truy xuất âm thanh phản hồi.

Nhóm API quản trị được đặt dưới prefix /api/admin và phục vụ các thao tác cập nhật nội dung hệ thống. Các API này quản lý địa điểm, media, tài liệu RAG, danh mục tài liệu, nhân vật hướng dẫn 3D và một số cấu hình kiosk. Nhờ Admin Panel, người quản trị có thể bổ sung hình ảnh, video, tài liệu hoặc điều chỉnh nội dung tham quan mà không cần can thiệp mã nguồn.

Đối với tài liệu RAG, API nhập liệu tài liệu tiếp nhận tệp từ trang quản trị, lưu lên Cloudflare R2, tạo bản ghi tài liệu và chuyển quá trình trích xuất, chunking, embedding sang background task. Cách triển khai này giúp thao tác upload phản hồi nhanh hơn, trong khi việc xử lý nội dung tiếp tục thực hiện ở phía backend.

Các nhóm API chính của backend được tóm tắt trong Bảng 4.6.

Bảng 4.6. Các nhóm API chính của backend

Nhóm API	Endpoint / prefix	Vai trò
Chat	POST /api/chat	Gửi câu hỏi đến AI Agent, nhận câu trả lời, nguồn tham khảo và hành động giao diện.
Phiên trò chuyện	POST /api/chat/session	Tạo phiên trò chuyện mới khi người dùng bắt đầu tham quan.
TTS	POST /api/tts, GET /api/audio/tts/{filename}	Tạo và truy xuất âm thanh phản hồi.
Quản trị nội dung	/api/admin/locations, /api/admin/media	Cập nhật địa điểm, hình ảnh, video và dữ liệu hiển thị.
Quản trị RAG	/api/admin/ingest, /api/admin/documents, /api/admin/categories	Upload, xử lý, phân loại và quản lý tài liệu RAG.
Quản trị vận hành	/api/admin/config, /api/admin/cache	Cấu hình kiosk và theo dõi một số tác vụ xử lý nền.

Việc tách API công khai và API quản trị giúp backend phục vụ đồng thời hai nhóm giao diện: giao diện tham quan dành cho người dùng cuối và giao diện quản trị dành cho người vận hành hệ thống.

4.2.5 Xác thực admin bằng Supabase Auth

Các API quản trị được bảo vệ bằng Supabase Auth. Supabase Auth cung cấp sẵn cơ chế đăng nhập, quản lý phiên và access token dạng JWT, giúp giảm rủi ro so với việc tự xây dựng toàn bộ cơ chế xác thực.

Trong backend, các router quản trị sử dụng dependency `verify_supabase_token` để kiểm tra quyền truy cập. Khi nhận request, backend đọc access token từ header Authorization dạng Bearer token và xác thực thông qua Supabase trước khi cho phép xử lý thao tác quản trị. Nếu token không hợp lệ hoặc đã hết hạn, backend sẽ từ chối request thay vì chỉ dựa vào trạng thái đăng nhập ở frontend.

Ở phía frontend, Admin Panel lấy session hiện tại từ Supabase và gửi access token kèm theo mỗi request đến backend. Cách triển khai này giúp bảo vệ các chức năng như cập nhật địa điểm, upload tài liệu, quản lý media, cấu hình kiosk và quản lý dữ liệu phục vụ RAG trong suốt quá trình quản trị.

Trong phạm vi đề tài, xác thực admin tập trung vào việc kiểm tra phiên đăng nhập hợp lệ trước khi cho phép truy cập API quản trị. Các API công khai như lấy danh

sách địa điểm, hỏi đáp AI, xem bản đồ và truy vấn đường đi không yêu cầu đăng nhập, giúp người dùng có thể sử dụng giao diện tham quan trực tiếp trên kiosk.

4.3 Xây dựng hệ thống AI, RAG và giọng nói

Phần này trình bày quá trình xây dựng các thành phần AI của hệ thống, bao gồm tích hợp Gemini, pipeline nhập liệu tài liệu RAG, truy vấn vector, AI Agent với Function Calling, TTS, STT và cơ chế đồng bộ trạng thái với nhân vật hướng dẫn 3D. Đây là nhóm chức năng giúp hệ thống không chỉ hiển thị nội dung tham quan, mà còn có khả năng tiếp nhận câu hỏi tự nhiên, truy xuất tri thức và phản hồi bằng văn bản, giọng nói hoặc hành động giao diện.

4.3.1 Tích hợp Gemini Client và cấu hình mô hình

Backend tích hợp Gemini thông qua thư viện google-genai. Việc khởi tạo client được tách trong module `core_client`, sử dụng cơ chế khởi tạo khi cần để chỉ tạo client khi có yêu cầu và tái sử dụng trong các lần gọi tiếp theo. Khóa API và tên các mô hình được nạp từ biến môi trường, giúp hệ thống có thể thay đổi mô hình chat, embedding hoặc TTS mà không cần sửa trực tiếp mã nguồn.

Trong cấu hình backend, hệ thống sử dụng Gemini cho ba nhóm tác vụ chính: sinh câu trả lời, tạo embedding và tạo giọng nói. Mô hình chat được dùng để sinh phản hồi cho người dùng và xử lý Function Calling. Mô hình embedding được dùng để chuyển câu hỏi và đoạn tài liệu thành vector 768 chiều, còn mô hình TTS được dùng để tạo âm thanh cho câu trả lời của nhân vật hướng dẫn 3D.

Khi gọi mô hình chat, backend xây dựng system prompt động dựa trên vị trí hiện tại, phong cách giọng nói, prompt nhân vật, danh sách địa điểm hợp lệ và các đoạn tài liệu liên quan được truy xuất từ vector store. Cách này giúp câu trả lời của AI Agent phù hợp hơn với ngữ cảnh tham quan, đồng thời hạn chế việc mô hình điều hướng đến những địa điểm không tồn tại trong hệ thống.

Ngoài ra, phần gọi Gemini được xử lý bất đồng bộ bằng cách chuyển các lời gọi đồng bộ của SDK sang worker thread. Backend cũng áp dụng cơ chế thử lại khi gặp lỗi giới hạn tài nguyên hoặc lỗi 429, giúp giảm khả năng thất bại tạm thời khi gọi mô hình.

4.3.2 Xây dựng pipeline nhập liệu tài liệu RAG

Pipeline nhập liệu tài liệu được triển khai để người quản trị có thể tải lên các tài liệu giới thiệu trường, ngành đào tạo hoặc thông tin tuyển sinh vào hệ thống một cách

thuận tiện. Các tệp được hỗ trợ gồm PDF, DOCX và Markdown, đồng thời kích thước tệp được giới hạn để tránh làm quá tải quá trình xử lý.

Quá trình nhập liệu được chia thành hai giai đoạn nhằm tách phần tải lên và phần xử lý nội dung. Ở giai đoạn đầu, API quản trị kiểm tra loại tệp, lưu tệp gốc lên Cloudflare R2 và tạo bản ghi document trong cơ sở dữ liệu với trạng thái pending. Sau đó, các tác vụ xử lý nặng được chuyển sang tác vụ nền để frontend có thể nhận phản hồi sớm thay vì phải chờ toàn bộ quá trình xử lý tài liệu hoàn tất.

Ở giai đoạn nền, backend trích xuất văn bản từ tệp, chia nhỏ nội dung thành các đoạn, tạo embedding cho từng đoạn và lưu vào bảng document_chunk. Trạng thái tài liệu được cập nhật lần lượt từ pending sang processing, sau đó là ready nếu xử lý thành công hoặc error nếu xảy ra lỗi. Cách tổ chức này giúp Admin Panel có thể theo dõi trạng thái xử lý của tài liệu sau khi tải lên.

Các bước chính trong pipeline nhập liệu tài liệu được trình bày trong Bảng 4.7.

Bảng 4.7. Pipeline nhập liệu tài liệu RAG

Bước xử lý	Thành phần thực hiện	Kết quả
Kiểm tra tệp	ingest_service	Xác nhận định dạng PDF, DOCX, Markdown và kích thước hợp lệ.
Lưu tệp gốc	storage_service	Upload tài liệu lên Cloudflare R2 theo nhóm danh mục.
Tạo bản ghi tài liệu	document_repo	Lưu thông tin tài liệu với trạng thái ban đầu pending.
Trích xuất văn bản	TextExtractor	Chuyển nội dung PDF, DOCX hoặc Markdown thành văn bản.
Chia nhỏ nội dung	MarkdownSemanticChunker	Tạo các đoạn tài liệu có kích thước phù hợp cho truy vấn RAG.
Tạo embedding	embedding_engine	Chuyển từng đoạn tài liệu thành vector 768 chiều.
Lưu vector	document_repo	Lưu nội dung đoạn và embedding vào bảng document_chunk.
Cập nhật trạng thái	document_repo	Đánh dấu tài liệu ready hoặc error tùy theo kết quả xử lý.

Trong quá trình chia nhỏ tài liệu, hệ thống ưu tiên giữ cấu trúc nội dung theo tiêu đề của tài liệu Markdown hoặc DOCX. Các đoạn quá ngắn có thể được gộp lại, trong khi các đoạn quá dài được tách theo ranh giới đoạn văn, danh sách hoặc bảng. Cách xử lý này giúp mỗi chunk vẫn giữ được ngữ cảnh tương đối đầy đủ, thuận lợi hơn cho quá trình truy xuất và sinh câu trả lời.

4.3.3 Truy vấn vector bằng pgvector và sinh câu trả lời

Khi người dùng đặt câu hỏi, backend tạo embedding cho câu hỏi bằng mô hình embedding của Gemini với task type phù hợp cho truy vấn hỏi đáp. Vector truy vấn sau đó được dùng để tìm các đoạn tài liệu liên quan trong bảng document_chunk thông qua pgvector. Hệ thống sử dụng toán tử cosine distance của pgvector để sắp xếp các đoạn theo độ tương đồng và lấy ra các kết quả phù hợp nhất.

Trong phiên bản hiện tại, kho tài liệu RAG được triển khai theo hướng dùng chung cho toàn hệ thống. Điều này phù hợp với phạm vi đề tài vì các tài liệu chính như thông tin trường, ngành đào tạo, học phí, học bổng hoặc quy chế tuyển sinh thường không chỉ gắn với một địa điểm cụ thể. Các đoạn tài liệu truy xuất được đưa vào prompt làm ngữ cảnh cho Gemini trước khi sinh câu trả lời.

Luồng xử lý câu hỏi gồm các bước chính: nhận câu hỏi từ frontend, tạo embedding, tìm kiếm vector, xây dựng ngữ cảnh, gọi Gemini, lưu lịch sử hội thoại và trả kết quả về giao diện. Kết quả trả về không chỉ có câu trả lời, mà còn có danh sách nguồn rút gọn, thời gian phản hồi và các hành động giao diện nếu AI Agent đề xuất.

Để tăng hiệu quả phản hồi, backend sử dụng cache cho embedding truy vấn và kết quả tìm kiếm vector. Với những câu hỏi hoặc truy vấn tương tự đã được xử lý gần đây, hệ thống có thể giảm số lần gọi mô hình embedding hoặc truy vấn lại vector store, từ đó cải thiện tốc độ phản hồi của hệ thống.

4.3.4 Xây dựng Function Calling và các công cụ của AI Agent

AI Agent của hệ thống được xây dựng dựa trên cơ chế Function Calling. Thay vì chỉ sinh câu trả lời văn bản, mô hình có thể đề xuất các hành động có cấu trúc để frontend thực thi, chẳng hạn như điều hướng đến địa điểm, mở media hoặc hiển thị bản đồ. Các công cụ này được khai báo trong module tools và được truyền vào cấu hình gọi Gemini.

Các công cụ chính của AI Agent được trình bày trong Bảng 4.8.

Bảng 4.8. Các công cụ của AI Agent

Công cụ	Nhóm	Vai trò
search_documents	Data tool	Tìm kiếm thông tin trong kho tài liệu RAG khi ngữ cảnh ban đầu chưa đủ.
navigate_to	UI tool	Điều hướng người dùng đến một địa điểm khác trong khuôn viên.
show_media	UI tool	Mở hình ảnh hoặc video liên quan đến địa điểm đang tham quan.
toggle_map	UI tool	Mở hoặc đóng bản đồ khuôn viên.

Đối với công cụ dữ liệu như search_documents, backend thực thi thêm một lượt truy vấn vector bằng câu truy vấn do AI đề xuất, sau đó gọi lại Gemini để sinh câu trả lời cuối cùng dựa trên ngữ cảnh bổ sung. Đối với các công cụ giao diện như navigate_to, show_media và toggle_map, backend không trực tiếp thay đổi giao diện mà trả danh sách tool_actions về frontend để thực thi.

Đối với chế độ streaming, hệ thống xử lý theo hướng thu thập phản hồi trước rồi mới quyết định hành động cần thực hiện. Backend nhận phản hồi đầy đủ từ mô hình ở lượt đầu để xác định có tool call hay không. Nếu phản hồi có tool call dùng để tìm kiếm tài liệu, backend thực hiện truy vấn bổ sung rồi mới stream câu trả lời cuối cùng. Nếu chỉ có công cụ giao diện, backend gửi sự kiện tool_call để frontend đưa vào hàng đợi xử lý. Cách này giúp tránh việc đang truyền từng phần câu trả lời nhưng phải thay đổi quyết định khi mô hình sinh ra tool call.

Ở phía frontend, các hành động giao diện được đưa vào hàng đợi. Nếu AI Agent vừa nói vừa yêu cầu điều hướng hoặc mở media, frontend ưu tiên phát xong phần âm thanh rồi mới thực thi một số hành động trực quan để trải nghiệm không bị ngắt đột ngột. Với trường hợp điều hướng sang địa điểm khác, thao tác navigate_to được xử lý trước, sau đó các hành động như mở media sẽ được áp dụng sau khi chuyển địa điểm hoàn tất theo đúng thứ tự xử lý.

4.3.5 Tích hợp TTS, STT và đồng bộ trạng thái nhân vật 3D

Hệ thống hỗ trợ tương tác giọng nói theo hai chiều. Ở chiều người dùng nói, frontend sử dụng Web Speech API để chuyển giọng nói tiếng Việt thành văn bản. Hook useSpeechRecognition cấu hình ngôn ngữ vi-VN, bật chế độ nhận dạng liên tục và hiển

thị transcript tạm thời trong lúc người dùng đang nói. Khi người dùng dừng ghi âm, nội dung transcript được gửi vào luồng chat giống như một câu hỏi văn bản.

Ở chiều AI phản hồi, backend sử dụng Gemini TTS để tạo âm thanh từ câu trả lời. Trước khi tạo giọng nói, hệ thống viết lại một số từ viết tắt như TVU hoặc CNTT thành cụm tiếng Việt đầy đủ để TTS đọc tự nhiên hơn khi phát cho người dùng. Nếu Gemini TTS gặp lỗi hoặc bị giới hạn, backend có cơ chế fallback sang Edge TTS. Âm thanh được trả về dưới dạng URL runtime để frontend phát ngay, đồng thời có thể được đồng bộ lên Cloudflare R2 để tái sử dụng trong các lần phản hồi sau.

Frontend quản lý việc phát âm thanh bằng audio manager trong chat store. Khi có câu hỏi mới hoặc người dùng chuyển địa điểm, âm thanh cũ được dừng để tránh chồng lấn. Trạng thái chat được đồng bộ với nhân vật hướng dẫn 3D thông qua avatarState, trong đó các trạng thái idle, thinking và speaking được ánh xạ sang các animation như Idle, Thinking và Talking. Ngoài ra, Avatar3D còn sử dụng morph target để tạo chuyển động miệng đơn giản khi nhân vật đang nói.

Để tránh hiện tượng microphone thu lại giọng nói của nhân vật, frontend tự động dừng nhận giọng nói khi nhân vật bắt đầu phát âm thanh. Ngược lại, nếu người dùng bấm microphone trong lúc nhân vật đang nói, hệ thống sẽ dừng audio hiện tại để ưu tiên thao tác của người dùng. Cơ chế này giúp giảm lỗi echo và làm cho quá trình hỏi đáp bằng giọng nói phù hợp hơn với môi trường kiosk.

4.4 Xây dựng giao diện Kiosk Tour

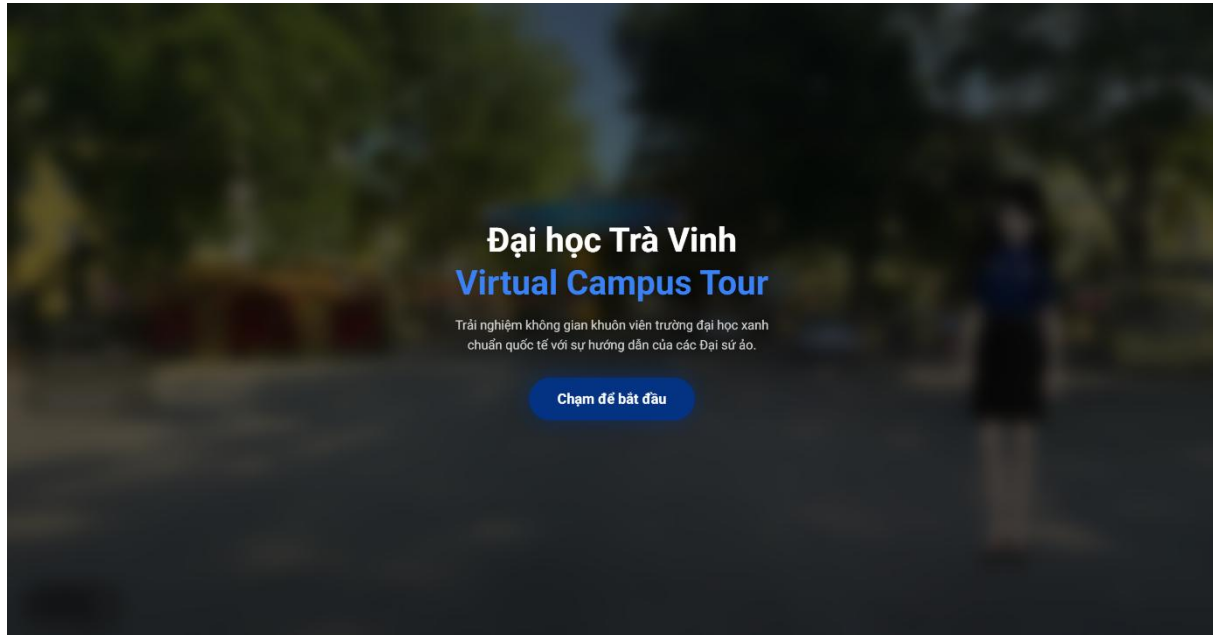
Giao diện Kiosk Tour là phần người dùng trực tiếp tương tác khi tham quan hệ thống. Giao diện này được xây dựng theo hướng một trang chính, trong đó ảnh 360 độ, nhân vật hướng dẫn 3D, khung chat, bản đồ khuôn viên và panel media được phối hợp trên cùng một màn hình. Frontend sử dụng Next.js, React, TypeScript và Zustand để quản lý trạng thái tham quan, giúp các thành phần giao diện cùng phản hồi theo địa điểm hiện tại và các hành động của AI Agent.

4.4.1 Trang tour chính, Start Overlay và tải tài nguyên

Trang tour chính được triển khai trong tệp frontend/src/app/page.tsx, đóng vai trò tổng hợp các lớp giao diện như PanoramaViewer, Avatar3D, Minimap, InfoPanel và ChatOverlay. Khi trang được tải, frontend gọi API để lấy danh sách địa điểm, dữ liệu đồ thị điều hướng và media của địa điểm bắt đầu. Do giao diện sử dụng ảnh 360 độ và

mô hình 3D, hệ thống chỉ bật trạng thái isAppReady khi các tài nguyên chính đã sẵn sàng hoặc khi hết thời gian chờ an toàn.

Start Overlay được sử dụng như màn hình mở đầu của kiosk. Người dùng cần chạm vào nút bắt đầu trước khi hệ thống phát âm thanh hoặc hiển thị đầy đủ các thành phần tương tác. Cách triển khai này tạo điểm bắt đầu rõ ràng cho phiên tham quan, đồng thời phù hợp với chính sách autoplay audio của trình duyệt. Giao diện Start Overlay trước khi bắt đầu phiên tham quan được minh họa ở Hình 4.2.



Hình 4.2. Giao diện Start Overlay trước khi bắt đầu tham quan

Sau khi người dùng bấm bắt đầu, giao diện được hiển thị theo từng lớp: ảnh 360 độ làm nền chính, nhân vật hướng dẫn 3D ở bên phải, bản đồ và panel thông tin ở hai bên, còn Chat Overlay ở phía dưới để thuận tiện nhập câu hỏi hoặc sử dụng giọng nói.

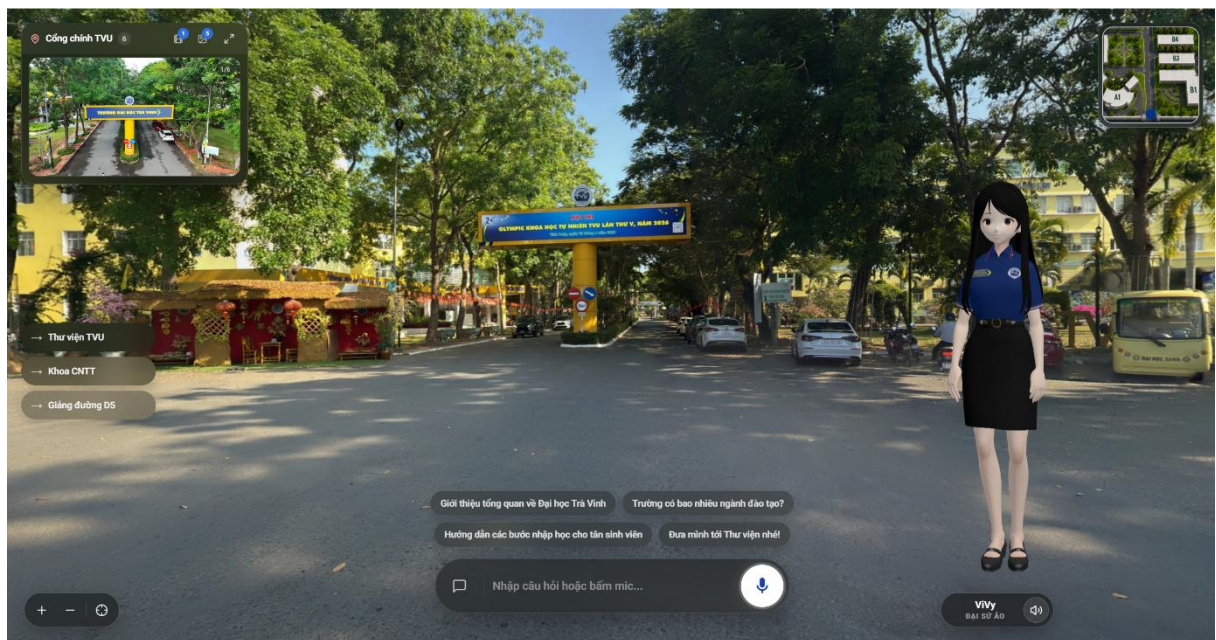
4.4.2 Hiển thị ảnh 360 độ và nhân vật hướng dẫn 3D

Ảnh 360 độ được hiển thị thông qua component PanoramaViewer. Component này bọc thư viện Pannellum để hiển thị ảnh equirectangular dưới dạng không gian quan sát 360 độ. Khi người dùng chuyển sang địa điểm khác, đường dẫn ảnh panorama được cập nhật và viewer sẽ tải lại cảnh tương ứng. Trong quá trình tải hoặc khi xảy ra lỗi ảnh, component hiển thị trạng thái riêng để tránh màn hình bị trống.

Nhân vật hướng dẫn 3D được triển khai bằng Avatar3D, sử dụng React Three Fiber và Drei để tải mô hình GLB. Component này được render ổn định trong trang chính thay vì unmount mỗi khi chuyển địa điểm, giúp hạn chế lỗi mất WebGL context

và giảm độ giật khi thay đổi trạng thái giao diện. Mô hình mặc định được đặt trong thư mục public, đồng thời dữ liệu từng địa điểm vẫn có thể khai báo model khác nếu cần.

Trạng thái hoạt ảnh của nhân vật được quản lý thông qua hook điều khiển hoạt ảnh useAvatarAnimationController. Khi người dùng bắt đầu tham quan, nhân vật phát hoạt ảnh chào. Trong quá trình AI xử lý câu hỏi, nhân vật chuyển sang trạng thái suy nghĩ, còn khi có âm thanh phản hồi, nhân vật chuyển sang trạng thái nói. Trong lúc nói, component sử dụng morph target của mô hình 3D để tạo chuyển động miệng đơn giản, giúp phản hồi bằng giọng nói tự nhiên hơn. Giao diện chính Kiosk Tour với ảnh 360 độ và nhân vật hướng dẫn 3D được minh họa ở Hình 4.3.



Hình 4.3. Giao diện chính Kiosk Tour với ảnh 360 độ và nhân vật hướng dẫn 3D

Việc kết hợp ảnh 360 độ và nhân vật hướng dẫn 3D giúp giao diện tham quan không chỉ dừng lại ở việc xem hình ảnh, mà còn tạo cảm giác có một trợ lý hướng dẫn đồng hành cùng người dùng trong quá trình trải nghiệm.

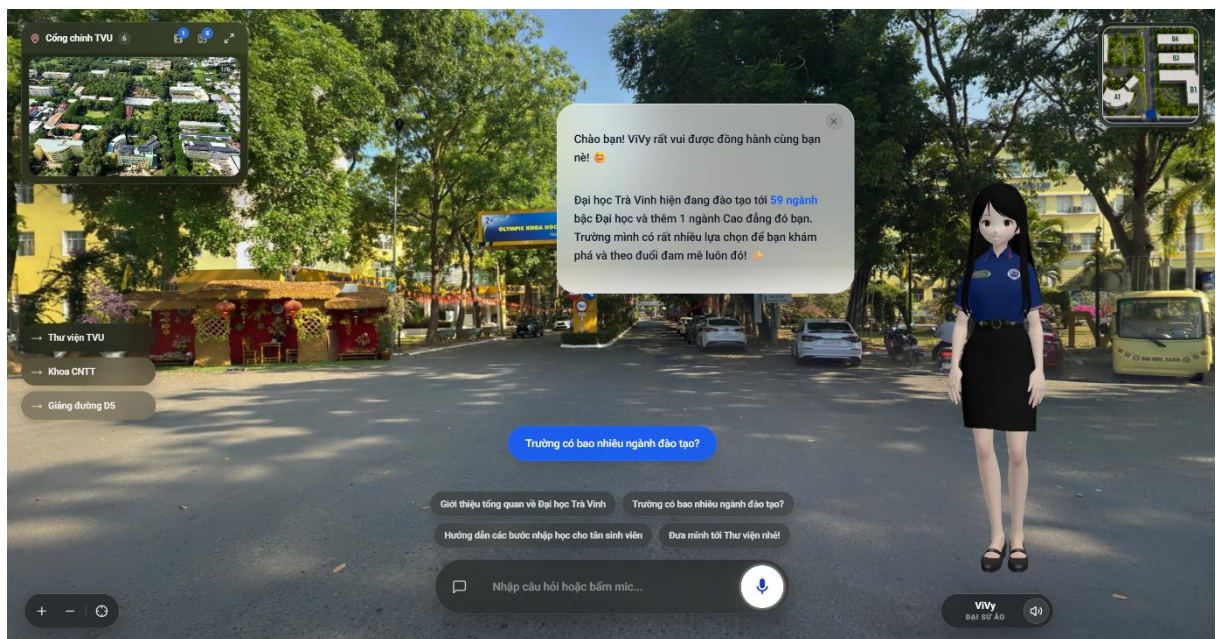
4.4.3 Chat Overlay, giọng nói và phản hồi của AI Agent

Chat Overlay là thành phần giúp người dùng đặt câu hỏi cho AI Agent bằng văn bản hoặc giọng nói. Component ChatOverlay hiển thị ô nhập câu hỏi, nút micro, các câu hỏi gợi ý theo địa điểm và bong bóng phụ đề cho câu trả lời của AI. Khi người dùng nhập câu hỏi, frontend gửi nội dung đến backend thông qua chat store, sau đó nhận lại câu trả lời, audio và các hành động giao diện nếu có.

Với chế độ giọng nói, frontend sử dụng Web Speech API để chuyển lời nói tiếng Việt thành văn bản. Chức năng nhận dạng giọng nói được triển khai thông qua một hook

riêng trong frontend, thay vì xử lý trực tiếp trong component giao diện. Nếu trình duyệt không hỗ trợ Web Speech API, giao diện vẫn giữ chức năng nhập văn bản để người dùng có thể tiếp tục tương tác.

Khi TTS được bật, frontend gửi câu hỏi đến backend với tùy chọn tạo âm thanh. Backend trả về câu trả lời, đường dẫn audio và danh sách hành động giao diện nếu AI Agent cần điều hướng, mở media hoặc hiển thị bản đồ. Trong thời gian audio phát, frontend chuyển trạng thái nhân vật sang speaking. Nếu người dùng bấm micro khi nhân vật đang nói, hệ thống sẽ dừng âm thanh hiện tại để tránh tiếng nói của hệ thống bị thu ngược lại vào micro trong quá trình tương tác. Giao diện Chat Overlay và phản hồi của AI Agent được minh họa ở Hình 4.4.



Hình 4.4. Chat Overlay và phản hồi của AI Agent trong giao diện tham quan

Các hành động giao diện do AI Agent đề xuất được đưa vào hàng đợi và xử lý theo thứ tự phù hợp. Ví dụ, nếu AI vừa trả lời vừa yêu cầu điều hướng đến địa điểm khác, hệ thống có thể phát xong câu hướng dẫn trước, sau đó mới mở bản đồ và bắt đầu tuyến đường. Nhờ đó, phản hồi đáp và phản điều khiển giao diện được liên kết với nhau nhưng vẫn giữ được luồng trải nghiệm dễ theo dõi.

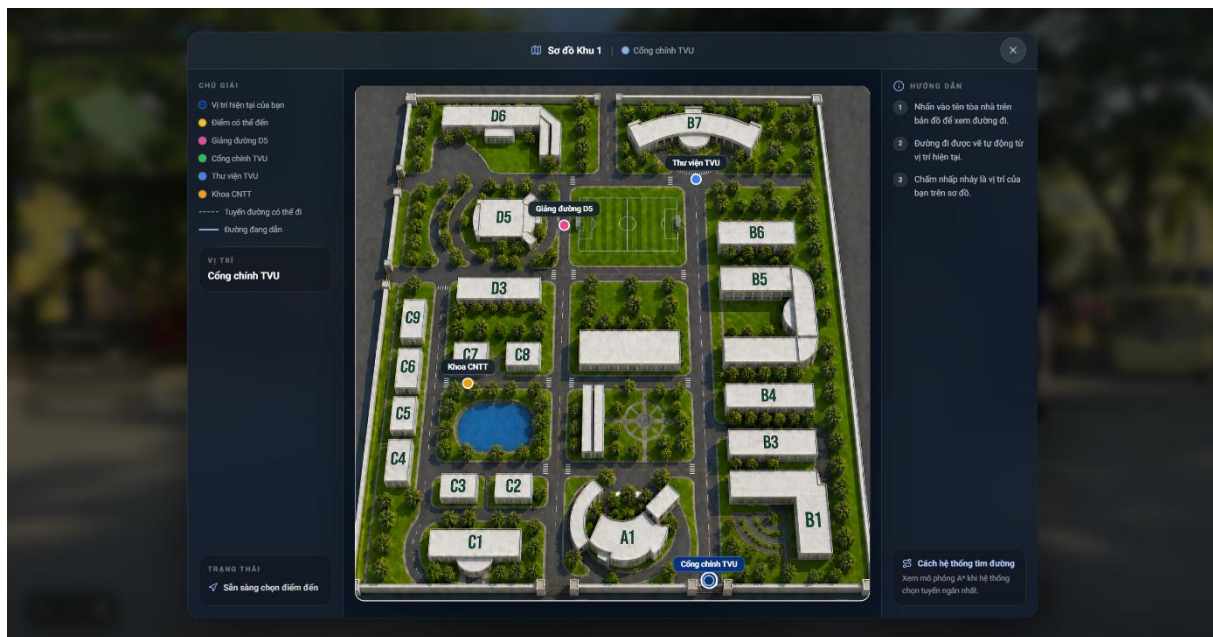
4.4.4 Bản đồ khuôn viên, minimap và dẫn đường bằng SVG route animation

Bản đồ khuôn viên được triển khai qua hai thành phần chính là Minimap và CampusMap. Ở trạng thái thu gọn, Minimap hiển thị bản đồ nhỏ để người dùng biết vị trí hiện tại, còn khi mở rộng, bản đồ hiển thị các địa điểm có thể đến, chú giải, trạng thái hiện tại và tuyến đường đang được dẫn.

CampusMap sử dụng hình ảnh bản đồ khuôn viên làm nền, sau đó phủ một lớp SVG lên trên để vẽ waypoint, marker và tuyến đường. Hệ tọa độ của SVG được chuẩn hóa theo viewBox="0 0 100 100", giúp các điểm trên bản đồ có thể lưu dưới dạng phần trăm và vẫn bám đúng vị trí khi giao diện thay đổi kích thước. Cách triển khai này phù hợp với bản đồ campus vì dữ liệu đường đi cần được tính toán bằng backend nhưng vẫn phải hiển thị trực quan trên ảnh bản đồ ở frontend.

Khi người dùng chọn một địa điểm hoặc khi AI Agent gọi công cụ điều hướng, frontend gọi API tìm đường để lấy kết quả A^* . Kết quả gồm trạng thái tìm thấy đường, danh sách nút, tọa độ đường đi và một số thông tin bổ sung phục vụ hiển thị tuyến đường. Các tọa độ này được lưu vào cache trong store để tránh gọi lại API nhiều lần cho cùng một cặp điểm bắt đầu - điểm đích.

Tuyến đường được vẽ bằng SVG polyline. Để tạo hiệu ứng dẫn đường, component tính độ dài tuyến đường và thay đổi strokeDasharray theo tiến độ animation. Khi animation hoàn tất, store chuyển địa điểm hiện tại sang địa điểm đích, tải lại ảnh 360 độ và tự động lấy media của địa điểm mới. Bản đồ khuôn viên và tuyến đường dẫn đến địa điểm được minh họa ở Hình 4.5.



Hình 4.5. Bản đồ khuôn viên và tuyến đường dẫn đến địa điểm

Cách triển khai này giúp bản đồ không chỉ là hình ảnh minh họa, mà trở thành một thành phần tương tác. Người dùng có thể tự chọn nơi muốn đến, còn AI Agent cũng có thể điều khiển bản đồ khi nội dung trả lời liên quan đến một địa điểm cụ thể.

4.4.5 Info Panel hiển thị hình ảnh, video và thông tin liên quan

Info Panel được xây dựng để hiển thị media của địa điểm hiện tại. Khi địa điểm thay đổi, frontend gọi API lấy danh sách media tương ứng và lưu vào store. Dữ liệu được phân loại thành hình ảnh, video hoặc GIF để giao diện hiển thị bằng thành phần phù hợp. Ở trạng thái thu gọn, panel hiển thị slideshow nhỏ, còn khi mở rộng, người dùng có thể xem video, thư viện ảnh và mô tả chi tiết của địa điểm.

Trong quá trình tương tác với AI Agent, công cụ show_media có thể yêu cầu frontend mở media liên quan đến câu trả lời. Khi nhận tool call này, frontend cập nhật focusedMediaId và tab ưu tiên, sau đó mở Info Panel. Nếu media là video, panel chuyển sang tab video, còn nếu media là ảnh, panel chuyển sang tab thông tin và thư viện ảnh. Cơ chế này giúp câu trả lời của AI không chỉ dừng ở văn bản, mà có thể dẫn người dùng đến hình ảnh hoặc video minh họa phù hợp. Info Panel hiển thị media liên quan đến địa điểm được minh họa ở Hình 4.6.



Hình 4.6. Info Panel hiển thị media liên quan đến địa điểm

Nhờ tách riêng Info Panel khỏi ảnh 360 độ, hệ thống có thể bổ sung nhiều loại nội dung khác nhau mà không làm thay đổi cấu trúc chính của trang tham quan.

4.4.6 Xử lý trạng thái giao diện: loading, chuyển cảnh và lỗi kết nối

Do Kiosk Tour phụ thuộc vào nhiều tài nguyên như dữ liệu địa điểm, ảnh 360 độ, mô hình 3D, media và kết quả từ backend, frontend cần xử lý rõ các trạng thái trung gian. Ở lần tải đầu tiên, người dùng chỉ có thể bắt đầu khi dữ liệu địa điểm, ảnh panorama

và nhân vật 3D đã sẵn sàng. Trong thời gian chờ, hệ thống hiển thị trạng thái tải để thông báo giao diện đang chuẩn bị không gian tham quan.

Khi chuyển địa điểm, store đặt trạng thái “isTransitioning” để hiển thị hiệu ứng chuyển cảnh và hạn chế thao tác chồng chéo. Âm thanh đang phát cũng được dừng trước khi đổi địa điểm để tránh tiếp tục phát trên cảnh mới. Nếu backend hoặc tài nguyên giao diện gặp lỗi, hệ thống sẽ thử lại và hiển thị thông báo phù hợp thay vì để màn hình trống. Các cơ chế nâng cao cho môi trường kiosk như tự reset khi không hoạt động và phục hồi kết nối sẽ được trình bày ở mục 4.6.

4.5 Xây dựng giao diện Admin Panel

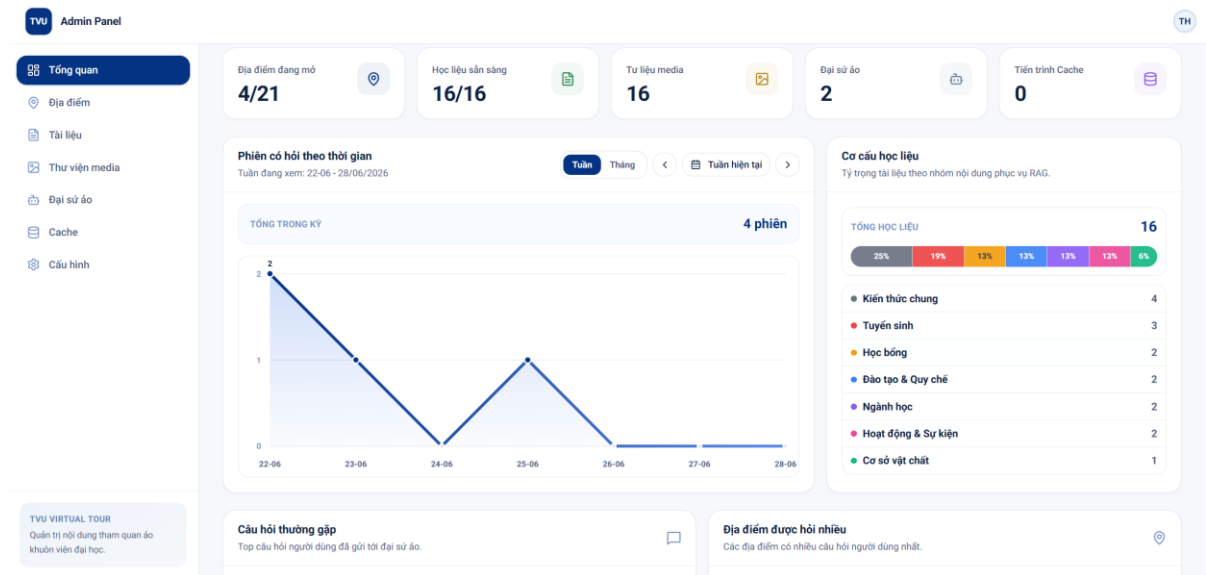
Admin Panel được xây dựng để hỗ trợ người quản trị cập nhật dữ liệu cho hệ thống tham quan mà không cần thao tác trực tiếp với cơ sở dữ liệu. Khác với Kiosk Tour là giao diện dành cho người dùng tham quan, Admin Panel tập trung vào các nghiệp vụ quản trị nội dung như quản lý địa điểm, ảnh 360 độ, media, tài liệu RAG, nhân vật hướng dẫn 3D, cache và một số cấu hình vận hành. Giao diện quản trị được triển khai trong nhóm đường dẫn /admin của frontend và giao tiếp với backend thông qua các API quản trị đã được bảo vệ bằng xác thực.

4.5.1 Bố cục và bảo vệ trang quản trị

Admin Panel được tách riêng trong nhóm đường dẫn /admin và chỉ dành cho người quản trị. Trang đăng nhập sử dụng Supabase Auth với email và mật khẩu để kiểm soát quyền truy cập. Nếu chưa có phiên đăng nhập hợp lệ, người dùng sẽ được chuyển về trang đăng nhập, còn nếu đã xác thực, hệ thống cho phép truy cập khu vực quản trị.

Sau khi đăng nhập, Admin Panel sử dụng bố cục chung gồm thanh điều hướng bên trái, thanh trên cùng và vùng nội dung chính. Các nhóm chức năng được tổ chức thành các mục như Tổng quan, Địa điểm, Tài liệu, Thư viện media, Đại sứ ảo, Cache và Cấu hình. Trang tổng quan giúp người quản trị theo dõi nhanh số lượng địa điểm, tài liệu, media, nhân vật hướng dẫn 3D và một số thống kê tương tác của hệ thống.

Các API quản trị được gọi thông qua module admin-api ở frontend. Module này lấy access token từ phiên đăng nhập Supabase và gửi kèm trong header Authorization khi gọi backend. Ở phía backend, các router quản trị kiểm tra token trước khi cho phép thực hiện thao tác thêm, sửa, xóa hoặc cập nhật dữ liệu. Giao diện Dashboard và bố cục chính của Admin Panel được minh họa ở Hình 4.7.



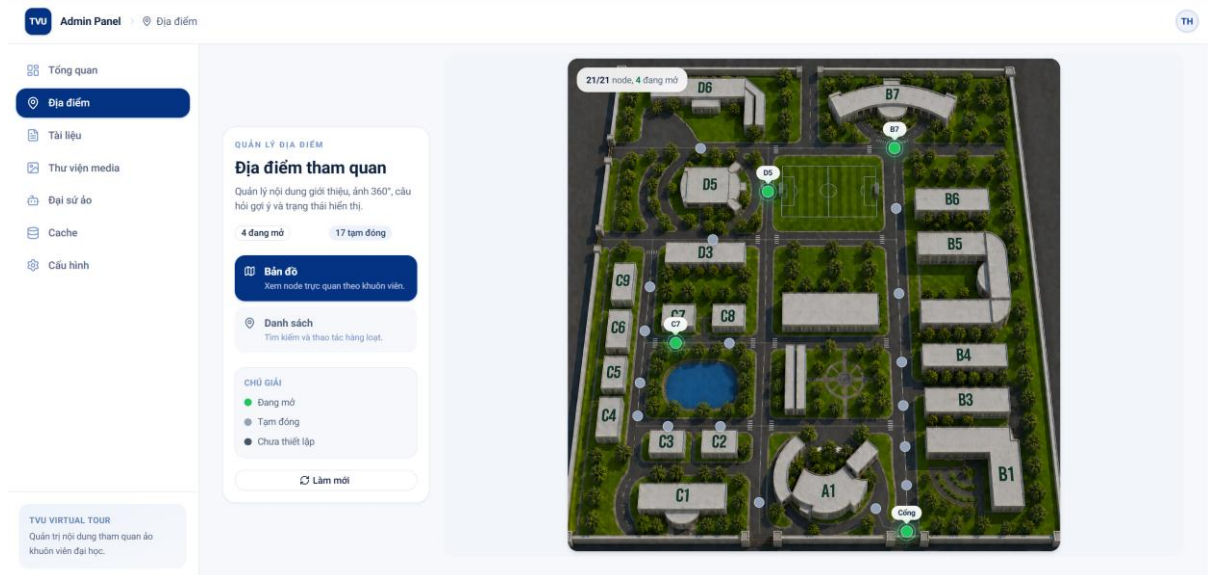
Hình 4.7. Giao diện Dashboard và bố cục chính của Admin Panel

4.5.2 Quản lý nội dung tham quan: địa điểm, bản đồ, media và nhân vật 3D

Nhóm chức năng quản lý nội dung tham quan cho phép người quản trị cập nhật dữ liệu hiển thị trên Kiosk Tour. Ở trang quản lý địa điểm, hệ thống hỗ trợ theo dõi các điểm tham quan trên bản đồ, trạng thái đang mở hoặc tạm đóng, số lượng node đã thiết lập và các điểm có thể đưa vào luồng dẫn đường. Giao diện bản đồ giúp người quản trị kiểm tra nhanh địa điểm nào đã sẵn sàng để hiển thị trong tour, thay vì chỉ nhìn danh sách dữ liệu dạng bảng, đồng thời hỗ trợ việc rà soát nội dung trực quan hơn.

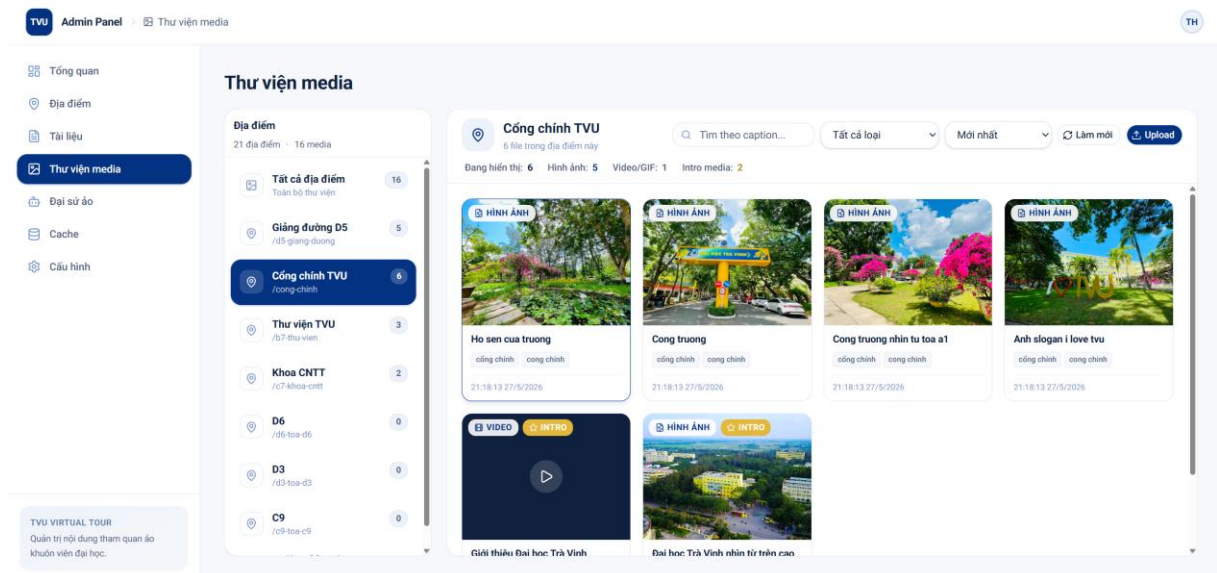
Ngoài chế độ bản đồ, trang địa điểm còn phục vụ việc chỉnh sửa thông tin chi tiết như tên địa điểm, slug, nội dung giới thiệu, ảnh 360 độ, câu hỏi gợi ý, nhân vật hướng dẫn 3D phụ trách địa điểm và trạng thái active/inactive. Khi cập nhật ảnh 360 độ, frontend gửi tệp lên backend bằng FormData, còn backend upload tệp lên Cloudflare R2 và cập nhật đường dẫn ảnh nền trong cơ sở dữ liệu. Khi người dùng quay lại Kiosk Tour, frontend sẽ lấy dữ liệu mới thông qua API và hiển thị ảnh 360 độ đã cập nhật.

Đối với bản đồ, Admin Panel có công cụ hỗ trợ kiểm tra trạng thái địa điểm và tạo lại dữ liệu đường đi khi cần. Khi một địa điểm bị tắt trạng thái hoạt động, giao diện tham quan sẽ không xem địa điểm đó là một điểm đến hợp lệ. Cách làm này giúp dữ liệu bản đồ và dữ liệu địa điểm được đồng bộ hơn, tránh trường hợp người dùng được dẫn đến một địa điểm chưa đủ nội dung. Giao diện quản lý địa điểm và trạng thái bản đồ trong Admin Panel được minh họa ở Hình 4.8.



Hình 4.8. Giao diện quản lý địa điểm và trạng thái bản đồ trong Admin Panel

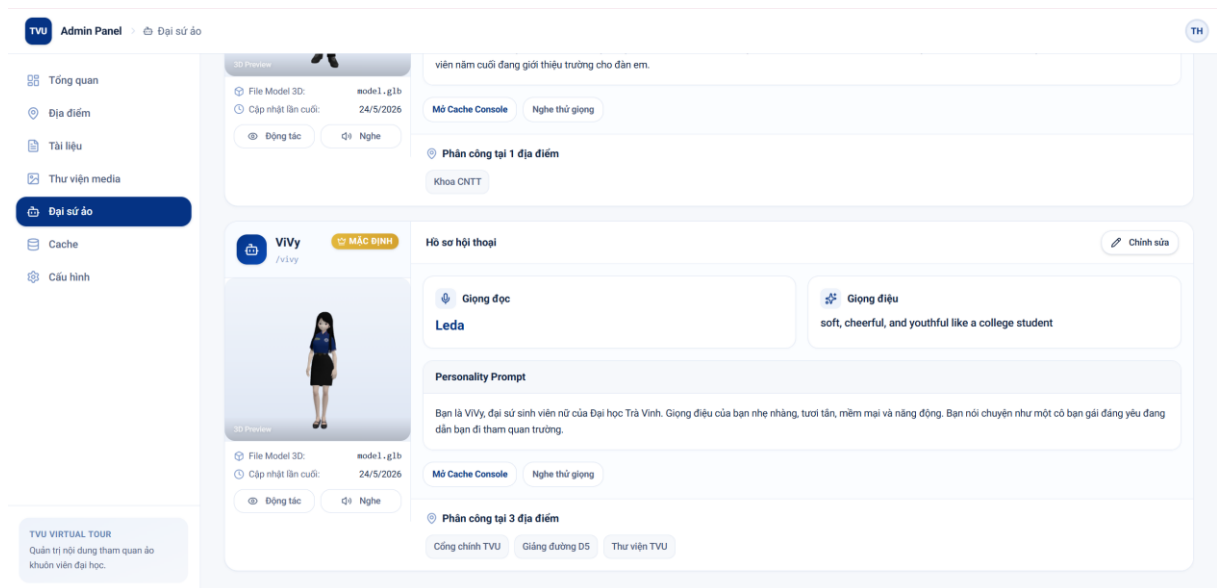
Trang thư viện media được dùng để quản lý hình ảnh, video hoặc GIF liên quan đến từng địa điểm. Người quản trị có thể tải media lên, gắn media với địa điểm, thêm chú thích, từ khóa và đánh dấu media giới thiệu. Dữ liệu này được sử dụng bởi Info Panel trong Kiosk Tour và cũng có thể được AI Agent mở ra thông qua công cụ show_media khi câu trả lời cần minh họa trực quan. Giao diện quản lý media trong Admin Panel được minh họa ở Hình 4.9.



Hình 4.9. Giao diện quản lý media trong Admin Panel

Ngoài địa điểm và media, Admin Panel còn có trang quản lý nhân vật hướng dẫn 3D. Tại đây, người quản trị có thể xem danh sách nhân vật, xem trước mô hình 3D, cấu hình giọng đọc, giọng điệu, personality prompt và các địa điểm đang được phân công. Các cấu hình này ảnh hưởng trực tiếp đến cách nhân vật xuất hiện, phát âm thanh và

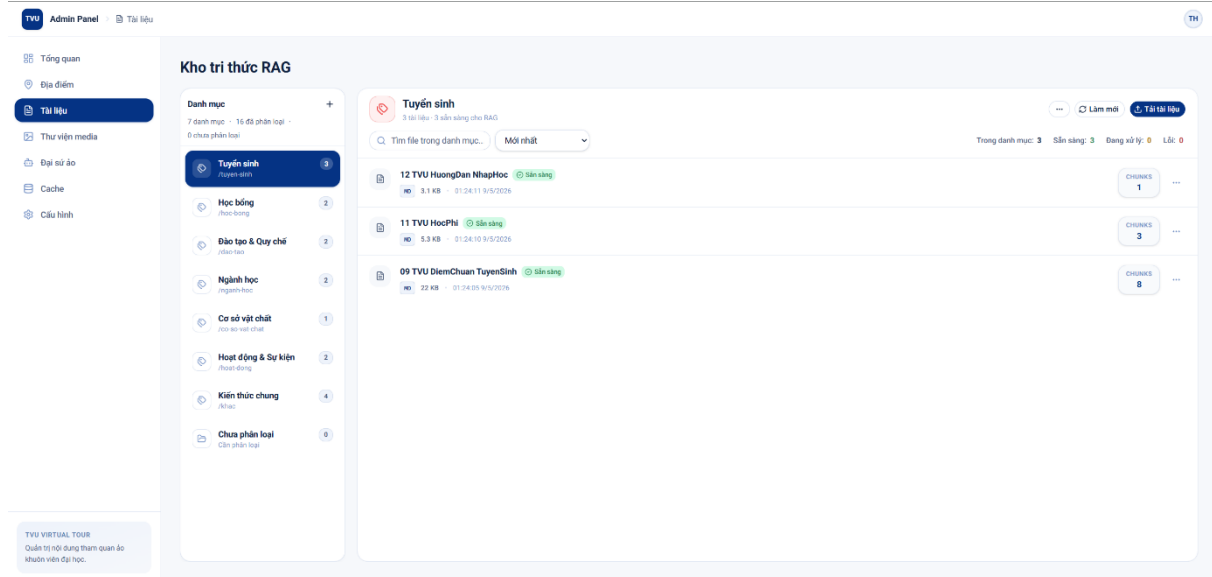
phản hồi trong giao diện tham quan. Hệ thống cũng hỗ trợ nghe thử giọng nói để người quản trị kiểm tra trước khi lưu cấu hình. Giao diện quản lý nhân vật hướng dẫn 3D trong Admin Panel được minh họa ở Hình 4.10.



Hình 4.10. Giao diện quản lý nhân vật hướng dẫn 3D trong Admin Panel

4.5.3 Quản lý tài liệu RAG và cấu hình hỗ trợ

Trang quản lý tài liệu RAG cho phép người quản trị tải lên tài liệu phục vụ kho tri thức của AI Agent. Các tài liệu có thể được phân loại theo danh mục để dễ quản lý, ví dụ thông tin chung, tuyển sinh, ngành đào tạo hoặc học phí. Khi tải tài liệu lên, frontend gửi tệp đến API quản trị, còn backend lưu tệp gốc lên Cloudflare R2, tạo bản ghi tài liệu và đưa quá trình trích xuất, chia nhỏ nội dung, tạo embedding vào tác vụ nền. Trạng thái xử lý của tài liệu được hiển thị trên giao diện để người quản trị biết tài liệu đã sẵn sàng hay gặp lỗi, từ đó thuận tiện hơn trong việc theo dõi và cập nhật kho tri thức. Giao diện quản lý tài liệu RAG và trạng thái xử lý tài liệu được minh họa ở Hình 4.11.



Hình 4.11. Giao diện quản lý tài liệu RAG và trạng thái xử lý tài liệu

Ngoài quản lý tài liệu, Admin Panel còn có trang Cache và Cấu hình để hỗ trợ vận hành hệ thống. Trang Cache giúp kiểm tra hoặc tạo lại một số dữ liệu sinh sẵn như câu trả lời, âm thanh giới thiệu và giọng nói của nhân vật, trong khi trang Cấu hình cho phép điều chỉnh các tham số kiosk như địa điểm bắt đầu, thời gian không hoạt động trước khi reset, thời lượng cảnh báo và trạng thái âm thanh mặc định.

Nhìn chung, Admin Panel đóng vai trò là công cụ quản trị nội dung và hỗ trợ vận hành cho toàn bộ hệ thống. Nhờ có giao diện này, dữ liệu phục vụ Kiosk Tour, kho tri thức RAG, media và cấu hình nhân vật hướng dẫn 3D có thể được cập nhật thuận tiện hơn mà không cần can thiệp trực tiếp vào mã nguồn hoặc cơ sở dữ liệu. Điều này giúp quá trình bổ sung nội dung, điều chỉnh thông tin, duy trì hoạt động sau khi triển khai và quản lý hệ thống về lâu dài trở nên linh hoạt hơn.

4.6 Ổn định hóa hệ thống và triển khai hệ thống

Sau khi các chức năng chính của Kiosk Tour và Admin Panel được xây dựng, hệ thống cần có thêm một số cơ chế hỗ trợ vận hành để phù hợp hơn với môi trường sử dụng thực tế. Trong phạm vi đề tài, phần ổn định hóa tập trung vào việc kiểm soát phiên tham quan, hạn chế thao tác ngoài luồng ở mức giao diện, xử lý lỗi tải dữ liệu và chuẩn bị các cấu hình cần thiết khi triển khai frontend, backend, database và storage.

4.6.1 Xử lý phiên kiosk, Start Overlay và tự reset khi không hoạt động

Giao diện Kiosk Tour được thiết kế theo dạng toàn màn hình, trong đó người dùng bắt đầu phiên tham quan thông qua Start Overlay. Màn hình này xuất hiện trước khi các thành phần tương tác chính được hiển thị đầy đủ. Người dùng cần chạm vào nút

bắt đầu đổ vào hệ thống, đồng thời cách triển khai này giúp việc phát âm thanh phù hợp hơn với chính sách autoplay của trình duyệt.

Trong quá trình khởi động, frontend tải danh sách địa điểm, dữ liệu bản đồ, ảnh 360 độ và mô hình nhân vật hướng dẫn 3D. Hệ thống chỉ mở trải nghiệm chính khi các tài nguyên quan trọng đã sẵn sàng hoặc khi hết thời gian chờ an toàn đã được thiết lập. Nhờ đó, người dùng hạn chế gặp tình trạng vào màn hình chính nhưng ảnh 360 độ hoặc nhân vật 3D chưa được chuẩn bị đầy đủ.

Để phù hợp với môi trường kiosk công cộng, frontend có cơ chế theo dõi trạng thái không hoạt động của người dùng trong suốt phiên tham quan. Hook `useKioskIdleWatcher` lắng nghe các thao tác như chạm màn hình, click chuột, bàn phím và cuộn trang. Nếu người dùng không tương tác trong một khoảng thời gian, hệ thống hiển thị cảnh báo đếm ngược trước khi reset. Khi hết thời gian cảnh báo, trang được tải lại để đưa kiosk về trạng thái ban đầu cho phiên tham quan tiếp theo.

Cơ chế idle reset được tạm dừng khi nhân vật hướng dẫn 3D đang phát giọng nói. Điều này tránh trường hợp hệ thống hiểu nhầm người dùng không hoạt động trong lúc AI Agent đang trả lời. Nếu người dùng chạm vào màn hình trong thời gian cảnh báo, cảnh báo được đóng và bộ đếm thời gian được khởi động lại.

4.6.2 Hạn chế thao tác ngoài luồng và xử lý lỗi giao diện

Khi bật biến môi trường `NEXT_PUBLIC_KIOSK_MODE`, frontend áp dụng một số xử lý nhằm hạn chế thao tác không mong muốn trong quá trình chạy kiosk. Các thao tác như mở menu chuột phải, làm mới trang bằng phím tắt, đóng tab bằng tổ hợp phím hoặc vuốt từ mép màn hình được chặn ở mức giao diện. Đây là lớp hỗ trợ phía frontend, giúp giảm các thao tác ngoài luồng trong lúc tham quan, nhưng không thay thế cho các cấu hình kiosk ở cấp hệ điều hành hoặc trình duyệt.

Hệ thống cũng xử lý một số tình huống lỗi thường gặp trong quá trình tải giao diện. Khi frontend không lấy được danh sách địa điểm từ backend, store sẽ chuyển sang trạng thái lỗi mạng, hiển thị màn hình thông báo và tự thử lại theo chu kỳ. Nếu số lần thử lại vượt quá giới hạn, giao diện hiển thị lỗi kết nối và cung cấp nút thử lại để người vận hành có thể khởi động lại quá trình tải dữ liệu.

Đối với các thành phần WebGL như ảnh 360 độ và nhân vật 3D, hệ thống có cơ chế xử lý lại khi xảy ra lỗi hiển thị hoặc mất context trong chế độ kiosk. Trong một số trường hợp, trang có thể được tải lại sau một khoảng trễ ngắn để đưa giao diện về trạng

thái ổn định hơn trước khi tiếp tục sử dụng. Cơ chế này có giới hạn thời gian chờ nhằm tránh vòng lặp tải lại liên tục khi lỗi lặp lại nhiều lần.

Ngoài ra, trong quá trình chuyển địa điểm, frontend đặt trạng thái chuyển cảnh và dừng âm thanh đang phát trước khi thay đổi dữ liệu địa điểm. Cách làm này giúp hạn chế tình trạng âm thanh của địa điểm cũ tiếp tục phát sau khi người dùng đã chuyển sang khu vực khác, đồng thời giúp quá trình chuyển cảnh diễn ra rõ ràng hơn.

4.6.3 Cấu hình triển khai frontend, backend, database và storage

Hệ thống được triển khai theo kiến trúc tách frontend và backend. Frontend Next.js chịu trách nhiệm hiển thị giao diện Kiosk Tour và Admin Panel, trong khi backend FastAPI cung cấp API cho địa điểm, media, chat, RAG, TTS, bản đồ và quản trị. Các biến môi trường được sử dụng để cấu hình địa chỉ API, khóa kết nối Supabase, Cloudflare R2, Gemini API và chế độ kiosk.

Database của hệ thống sử dụng Supabase PostgreSQL để lưu trữ dữ liệu địa điểm, media, tài liệu RAG, chunk văn bản, phiên trò chuyện và cấu hình vận hành. Cloudflare R2 được dùng để lưu các tệp có dung lượng lớn như ảnh 360 độ, media, tài liệu gốc và một số tệp âm thanh được sinh sẵn. Cách tách riêng dữ liệu quan hệ và tệp media giúp hệ thống dễ quản lý hơn khi số lượng nội dung tăng lên.

Sau khi triển khai, hệ thống được kiểm tra theo các chức năng chính. Kiosk Tour được kiểm tra khả năng tải đúng địa điểm khởi đầu, ảnh 360 độ và nhân vật 3D. AI Agent được kiểm tra khả năng trả lời câu hỏi dựa trên kho tri thức RAG. Bản đồ được kiểm tra khả năng hiển thị tuyến đường giữa các địa điểm. Admin Panel được kiểm tra chức năng đăng nhập và cập nhật dữ liệu. Các tệp tải lên R2 được kiểm tra khả năng truy xuất từ frontend. Việc kiểm tra nhằm bảo đảm toàn bộ hệ thống hoạt động ổn định và các thành phần frontend, backend, database, storage và AI được tích hợp chính xác.

Nhìn chung, các cơ chế ổn định hóa trong đề tài được triển khai ở mức phục vụ vận hành cơ bản cho giao diện kiosk. Hệ thống đã có Start Overlay, tự reset khi không hoạt động, xử lý lỗi kết nối, phục hồi một số lỗi WebGL và cấu hình triển khai cần thiết. Các cấu hình kiosk chuyên sâu hơn ở cấp thiết bị hoặc hệ điều hành có thể tiếp tục được bổ sung trong các bước phát triển sau.

CHƯƠNG 5: KẾT QUẢ VÀ ĐÁNH GIÁ

Chương này trình bày kết quả triển khai và đánh giá hệ thống AI Agent giới thiệu Đại học Trà Vinh và các ngành đào tạo sau quá trình xây dựng. Nội dung chương tập trung vào quy mô dữ liệu đã triển khai, kết quả vận hành các chức năng chính, thời gian phản hồi của một số API công khai, kết quả kiểm thử dẫn đường bằng thuật toán A^* và khả năng trả lời của AI Agent dựa trên kho tri thức RAG. Trên cơ sở các kết quả thu được từ môi trường production, chương đưa ra nhận xét về mức độ đáp ứng mục tiêu đề tài, đồng thời chỉ ra một số hạn chế và hướng có thể tiếp tục cải thiện.

5.1 Phạm vi và phương pháp đánh giá

Việc đánh giá hệ thống được thực hiện theo hướng thực nghiệm, dựa trên các chức năng chính đã xây dựng và dữ liệu đã triển khai trên môi trường production. Nội dung đánh giá bao gồm quy mô dữ liệu hệ thống, khả năng vận hành giao diện tham quan, khả năng quản trị nội dung, thời gian phản hồi của một số API công khai, kết quả dẫn đường bằng thuật toán A^* và khả năng trả lời của AI Agent dựa trên kho tri thức RAG trong điều kiện sử dụng thực tế.

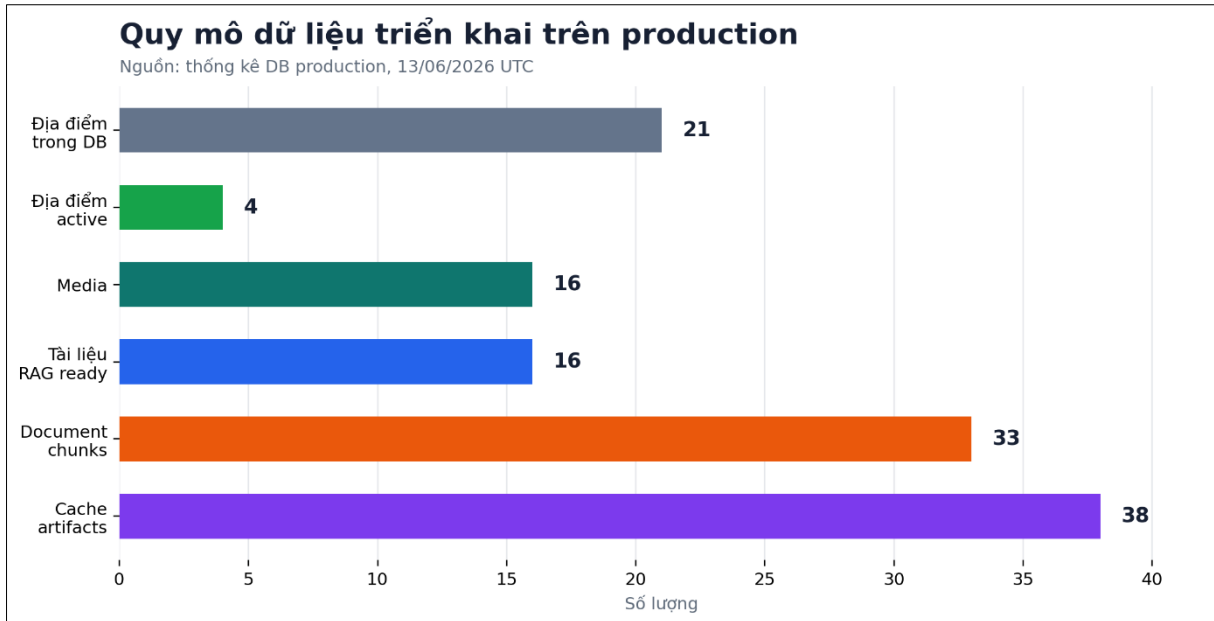
Trong phạm vi đề tài, việc đánh giá tập trung vào mức độ đáp ứng chức năng và khả năng vận hành cơ bản, chưa thực hiện kiểm thử tải lớn với nhiều người dùng đồng thời. Đối với AI Agent và RAG, hệ thống được đánh giá bằng bộ câu hỏi theo kịch bản sử dụng, bao gồm câu hỏi thông tin chung, ngành đào tạo, học phí, học bổng, cơ sở vật chất và các yêu cầu kích hoạt hành động giao diện. Phạm vi đề tài chưa thực hiện đánh giá định lượng chuyên sâu bằng các chỉ số như precision, recall hoặc faithfulness.

Các số liệu trong chương này được thu thập từ môi trường triển khai thực tế của hệ thống, thông qua thống kê cơ sở dữ liệu, gọi API công khai và kiểm thử các cặp đường đi giữa các địa điểm đang hoạt động. Việc đo thời gian phản hồi API được thực hiện bằng cách gọi mỗi API 10 lần qua tên miền công khai của hệ thống.

5.2 Kết quả triển khai hệ thống

Sau quá trình xây dựng, hệ thống đã triển khai các thành phần chính phục vụ tham quan ảo và hỏi đáp thông tin. Giao diện Kiosk Tour cho phép người dùng tham quan ảnh 360 độ, tương tác với nhân vật hướng dẫn 3D, trao đổi với AI Agent, xem bản đồ và truy cập media liên quan. Admin Panel hỗ trợ quản lý địa điểm, media, tài liệu RAG, danh mục tài liệu, nhân vật hướng dẫn và các cấu hình kiosk.

Trên môi trường production, hệ thống hiện có 21 địa điểm, trong đó 4 địa điểm đang hoạt động để phục vụ trải nghiệm tham quan. Dữ liệu gồm 16 media, 7 danh mục tài liệu RAG, 16 tài liệu sẵn sàng và 33 đoạn tài liệu đã được xử lý phục vụ truy xuất ngữ nghĩa. Ngoài ra, hệ thống có 16 câu hỏi gợi ý, 2 nhân vật hướng dẫn và 38 dữ liệu cache hỗ trợ tối ưu trải nghiệm sử dụng. Quy mô dữ liệu triển khai trên production được minh họa ở Hình 5.1.

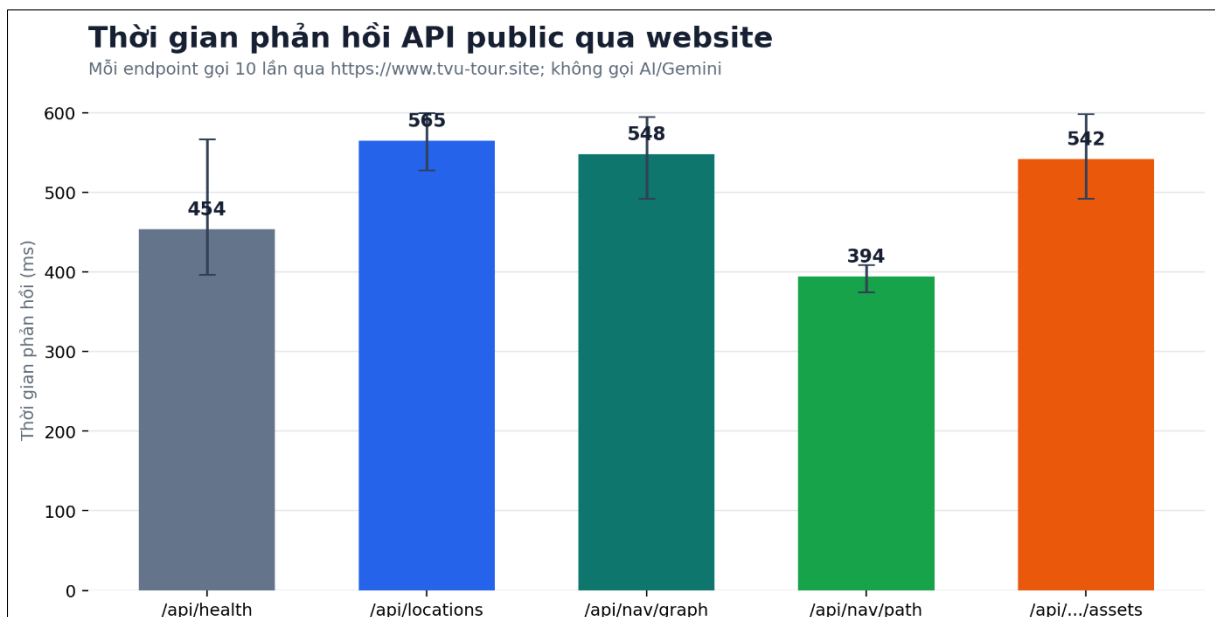


Hình 5.1. Quy mô dữ liệu triển khai trên production

Các số liệu trên cho thấy hệ thống đã có dữ liệu vận hành thực tế cho các chức năng cốt lõi như tham quan địa điểm, hiển thị media, hỏi đáp RAG, câu hỏi gợi ý, nhân vật hướng dẫn và cache. Tuy nhiên, số lượng địa điểm hiện mới tập trung ở một nhóm tiêu biểu, phù hợp với phạm vi đề tài thay vì bao phủ toàn bộ khuôn viên trường.

5.3 Kết quả kiểm thử trên môi trường production

Để kiểm tra và đánh giá khả năng vận hành thực tế của hệ thống, một số API công khai đã được gọi trực tiếp thông qua tên miền production. Các API được lựa chọn để kiểm thử bao gồm API kiểm tra trạng thái hệ thống, API lấy danh sách địa điểm, API lấy dữ liệu bản đồ, API tính toán đường đi và API lấy media của địa điểm. Mỗi API được gọi lặp lại 10 lần nhằm đảm bảo tính ổn định của kết quả, đồng thời ghi nhận các chỉ số thời gian phản hồi trung bình, nhỏ nhất và lớn nhất. Kết quả đo thời gian phản hồi của các API công khai được tổng hợp và minh họa chi tiết ở Hình 5.2.



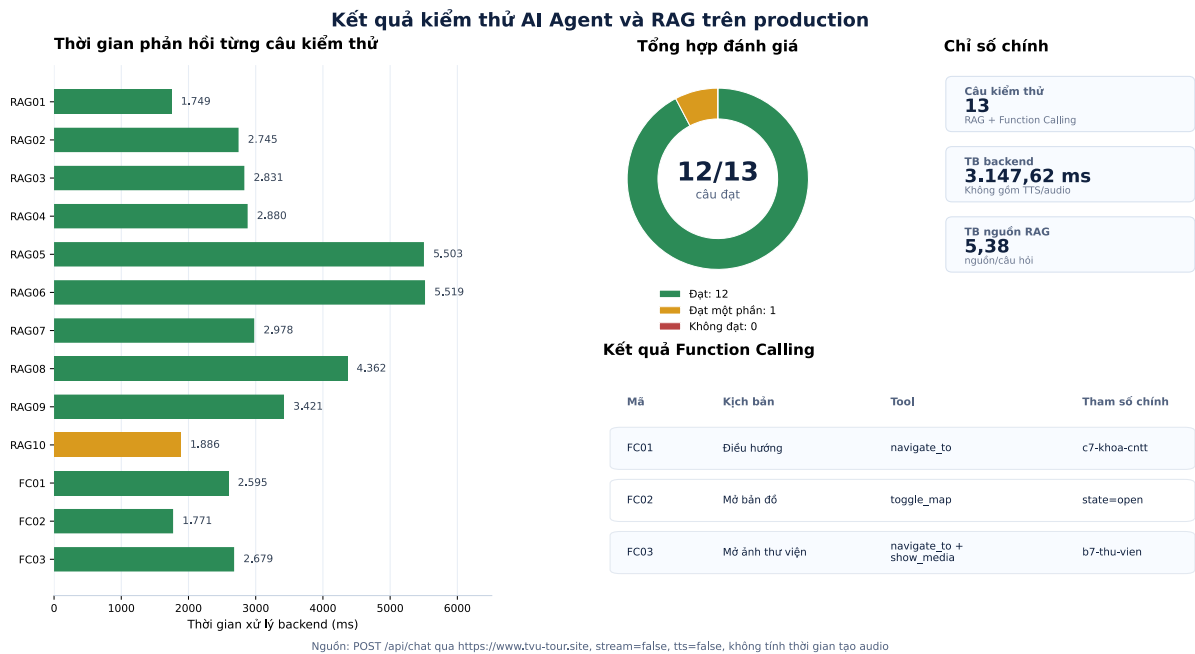
Hình 5.2. Thời gian phản hồi API public qua website

Kết quả cho thấy các API công khai đều phản hồi thành công với mã trạng thái 200 trong các lần đo. API tính đường đi có thời gian phản hồi trung bình khoảng 394,08 ms, trong khi các API có truy vấn cơ sở dữ liệu như danh sách địa điểm, dữ liệu bản đồ và media dao động trong khoảng 541-565 ms. Các kết quả này cho thấy hệ thống có thể đáp ứng các thao tác cơ bản của giao diện tham quan trong môi trường production. Tuy nhiên, đây chỉ là kết quả đo đơn lẻ trên một số API, chưa phải kiểm thử tải với nhiều người dùng đồng thời.

Cần lưu ý rằng kết quả đo trên được thực hiện qua môi trường công khai nên bao gồm độ trễ mạng, tên miền và hạ tầng triển khai. Nếu triển khai cho kiosk nội bộ, các thành phần có thể được đặt trong cùng mạng LAN để giảm thời gian phản hồi. Tuy nhiên, các chức năng gọi dịch vụ AI bên ngoài như sinh câu trả lời, tạo embedding hoặc TTS vẫn phụ thuộc vào kết nối Internet và cơ chế cache của hệ thống.

Đối với AI Agent và RAG, hệ thống được kiểm thử bằng 13 câu hỏi trên API chat của môi trường production. Khi kiểm thử, chế độ streaming và TTS được tắt để tập trung vào phần truy xuất tài liệu, sinh câu trả lời và Function Calling. Bộ câu hỏi bao gồm 10 câu hỏi thông tin dựa trên kho tri thức và 3 câu hỏi yêu cầu hành động giao diện như dẫn đường, mở bản đồ và mở hình ảnh địa điểm. Kết quả cho thấy 12/13 câu đạt yêu cầu, 1/13 câu đạt một phần và không có câu nào thất bại hoàn toàn. Thời gian xử lý backend trung bình của nhóm câu hỏi này là khoảng 3.147,62 ms, mỗi câu trả về trung bình 5,38 nguồn RAG liên quan. Câu đạt một phần là câu hỏi về số lượng sinh viên trong năm hiện tại: hệ thống trả lời được số liệu gần đúng theo tài liệu đã nhập, nhưng

chưa nhấn mạnh đủ rằng đây là thông tin có thể thay đổi theo từng năm. Kết quả kiểm thử AI Agent và RAG được minh họa ở Hình 5.3.



Hình 5.3. Kết quả kiểm thử AI Agent và RAG trên production

Chức năng bản đồ và dẫn đường được kiểm thử bằng cách gọi API tìm đường giữa tất cả các cặp địa điểm đang hoạt động. Với 4 địa điểm hoạt động, hệ thống có 12 cặp điểm xuất phát - điểm đích khác nhau. Kết quả kiểm thử cho thấy cả 12/12 cặp đều tìm được đường, không có trường hợp thất bại. Thời gian phản hồi API trung bình là 410,70 ms, trong khi thời gian tính toán A^* nội bộ trung bình chỉ khoảng 0,108 ms. Điều này cho thấy chi phí tính đường của thuật toán là nhỏ, thời gian người dùng cảm nhận chủ yếu đến từ độ trễ mạng và việc gọi API qua môi trường công khai. Kết quả kiểm thử dẫn đường bằng thuật toán A^* được minh họa ở Hình 5.4.



Hình 5.4. Kết quả kiểm thử dẫn đường A^* trên production

Ngoài các kiểm thử trên môi trường production, mã nguồn backend cũng được kiểm tra bằng bộ kiểm thử tự động hiện có. Kết quả chạy test cho thấy 46 test backend đều đạt. Frontend cũng build thành công ở chế độ production với Next.js. Các kết quả này giúp xác nhận các thành phần kỹ thuật chính có thể biên dịch và đóng gói được trước khi triển khai.

Bảng 5.1 sau đây tổng hợp kết quả kiểm thử chính của hệ thống.

Bảng 5.1. Tổng hợp kết quả kiểm thử hệ thống

Nội dung kiểm thử	Kết quả	Nhận xét
Thống kê dữ liệu production	Đạt	Có dữ liệu địa điểm, media, tài liệu RAG, chunk, câu hỏi gợi ý và dữ liệu cache.
API công khai	Đạt	Các API kiểm thử đều trả về thành công trong 10 lần gọi.
Dẫn đường A*	Đạt	12/12 cặp địa điểm hoạt động tìm được đường.
Kiểm thử backend	Đạt	46 test backend đều đạt.
Build frontend production	Đạt	Ứng dụng frontend build thành công.
AI/RAG	Đạt một phần	Trả lời được trong phạm vi tài liệu đã nhập, nhưng chất lượng phụ thuộc vào độ đầy đủ và cập nhật của kho tri thức.
Vận hành kiosk	Đạt một phần	Có Start Overlay, idle reset và xử lý lỗi cơ bản, kiosk mode cấp hệ điều hành chưa nằm trong phạm vi chính.

5.4 Nhận xét chung

Kết quả triển khai và kiểm thử cho thấy hệ thống đã đáp ứng được các mục tiêu trong phạm vi đề tài. Về phía người dùng, hệ thống cung cấp được trải nghiệm tham quan trực quan thông qua ảnh 360 độ, media, bản đồ, nhân vật hướng dẫn 3D và hỏi đáp bằng AI Agent. Về phía quản trị, hệ thống có Admin Panel để cập nhật nội dung, tài liệu RAG, media và cấu hình vận hành mà không cần thao tác trực tiếp trên cơ sở dữ liệu.

Điểm mạnh của hệ thống là kết hợp được nhiều thành phần trong cùng một trải nghiệm: tham quan ảo, hỏi đáp RAG, Function Calling, dẫn đường A* và quản trị nội dung. Các số liệu production cho thấy hệ thống đã có dữ liệu vận hành thực tế và các API chính hoạt động ổn định trong các kịch bản kiểm thử cơ bản.

Bên cạnh đó, hệ thống vẫn còn một số hạn chế cần tiếp tục cải thiện. Thứ nhất, số lượng địa điểm hoạt động mới dừng ở mức các địa điểm tiêu biểu, chưa bao phủ toàn

bộ khuôn viên. Thứ hai, chất lượng trả lời của AI Agent phụ thuộc vào kho tài liệu RAG, nếu tài liệu thiếu hoặc chưa cập nhật, câu trả lời có thể chưa đầy đủ hoặc chưa phản ánh đúng thông tin mới nhất. Thứ ba, kết quả đo API hiện tại mới là kiểm thử đơn lẻ, chưa phản ánh khả năng chịu tải khi có nhiều người dùng truy cập đồng thời. Cuối cùng, cơ chế kiosk hiện được hỗ trợ chủ yếu ở mức giao diện web, còn các cấu hình kiosk chuyên sâu ở cấp thiết bị hoặc hệ điều hành có thể tiếp tục được bổ sung trong tương lai.

Nhìn chung, hệ thống đã hoàn thành các chức năng trọng tâm của đề tài và có thể dùng làm nền tảng để mở rộng thêm dữ liệu, tăng số địa điểm tham quan, bổ sung bộ đánh giá AI/RAG chuyên sâu và hoàn thiện khả năng vận hành kiosk trong môi trường sử dụng thực tế. Những kết quả này cho thấy hướng tiếp cận kết hợp tham quan trực quan với AI Agent có tính khả thi và có thể tiếp tục phát triển trong các phiên bản sau.

KẾT LUẬN

Đề tài “Xây dựng AI Agent giới thiệu Đại học Trà Vinh và các ngành đào tạo” đã xây dựng một hệ thống tham quan ảo thông minh nhằm hỗ trợ người dùng tìm hiểu nhà trường theo cách trực quan và tương tác hơn. Thay vì chỉ trình bày thông tin ở dạng tĩnh, hệ thống kết hợp ảnh 360 độ, nhân vật hướng dẫn 3D, bản đồ khuôn viên, media minh họa và cơ chế hỏi đáp dựa trên kho tri thức RAG. Qua đó, đề tài cho thấy tính khả thi của việc ứng dụng AI Agent vào hoạt động giới thiệu trường học, hỗ trợ truyền thông, tuyển sinh và tra cứu thông tin giáo dục.

Kết quả đạt được

Trong quá trình thực hiện, đề tài đã xây dựng được các thành phần chính gồm giao diện Kiosk Tour, backend API, kho tri thức RAG, AI Agent, bản đồ dẫn đường A* và Admin Panel. Giao diện Kiosk Tour hỗ trợ người dùng bắt đầu phiên tham quan, quan sát không gian 360 độ, đặt câu hỏi bằng văn bản hoặc giọng nói, nghe phản hồi từ hệ thống và mở các nội dung minh họa theo từng địa điểm. AI Agent được tích hợp với RAG và Function Calling, nhờ đó hệ thống không chỉ trả lời dựa trên tài liệu đã chuẩn bị mà còn có thể kích hoạt một số hành động giao diện như mở bản đồ, mở media hoặc điều hướng đến địa điểm phù hợp.

Bên cạnh giao diện dành cho người dùng, hệ thống cũng xây dựng được Admin Panel để hỗ trợ người quản trị cập nhật địa điểm, ảnh 360 độ, media, tài liệu RAG, danh mục tài liệu, nhân vật hướng dẫn và một số cấu hình kiosk. Kết quả triển khai trên môi trường production cho thấy hệ thống đã có dữ liệu vận hành thực tế, các API công khai phản hồi thành công trong các lần kiểm thử, chức năng dẫn đường tìm được đường cho các cặp địa điểm đang hoạt động và AI Agent trả lời đạt yêu cầu trong phần lớn câu hỏi kiểm thử. Các kết quả này cho thấy hệ thống đã đáp ứng các chức năng trọng tâm trong phạm vi đề tài và có thể làm nền tảng để tiếp tục hoàn thiện ở các bước sau.

Hạn chế

Mặc dù đã hoàn thành các chức năng trọng tâm, hệ thống vẫn còn một số hạn chế cần tiếp tục được cải thiện. Trước hết, phạm vi dữ liệu hiện tại mới tập trung vào một số địa điểm tiêu biểu, chưa bao phủ toàn bộ khuôn viên trường. Điều này làm cho trải nghiệm tham quan vẫn còn giới hạn ở một số khu vực chính, chưa thể hiện đầy đủ toàn bộ không gian của nhà trường.

Bên cạnh đó, chất lượng trả lời của AI Agent còn phụ thuộc vào mức độ đầy đủ, chính xác và cập nhật của kho tài liệu RAG. Nếu tài liệu thiếu, chưa được chuẩn hóa hoặc thông tin thay đổi theo thời gian, câu trả lời có thể chưa phản ánh đầy đủ ngữ cảnh thực tế. Ngoài ra, việc đánh giá hệ thống chủ yếu dựa trên kiểm thử chức năng và đo thời gian phản hồi ở một số API chính, chưa thực hiện kiểm thử tải với nhiều người dùng đồng thời hoặc đánh giá định lượng chuyên sâu cho chất lượng trả lời của AI.

Cuối cùng, cơ chế kiosk hiện được hỗ trợ chủ yếu ở mức giao diện web, chưa đi sâu vào các cấu hình ở cấp thiết bị, trình duyệt hoặc hệ điều hành. Do đó, khi triển khai trong môi trường sử dụng thực tế, hệ thống vẫn cần được bổ sung thêm các cấu hình vận hành và kiểm thử ổn định trong thời gian dài.

Hướng phát triển

Trong thời gian tới, hệ thống có thể tiếp tục được hoàn thiện theo hướng thực tế hơn, tập trung vào việc mở rộng dữ liệu địa điểm, ảnh 360 độ và media để tăng mức độ bao phủ khuôn viên trường. Kho tài liệu RAG cần được chuẩn hóa, cập nhật định kỳ và bổ sung metadata rõ ràng hơn nhằm cải thiện chất lượng truy xuất thông tin, đặc biệt với các nội dung dễ thay đổi như tuyển sinh, học phí, học bổng và số liệu thống kê.

Bên cạnh đó, AI Agent có thể được bổ sung thêm một số công cụ hỗ trợ giao diện và tra cứu dữ liệu, chẳng hạn như gợi ý lộ trình tham quan theo địa điểm, tìm kiếm nhanh tài liệu liên quan, mở đúng nhóm media theo ngữ cảnh câu hỏi hoặc ghi nhận phản hồi của người dùng sau mỗi câu trả lời. Các công cụ này giúp AI Agent không chỉ trả lời thông tin, mà còn hỗ trợ người dùng thao tác với hệ thống hiệu quả hơn.

Ngoài ra, hệ thống cần được kiểm thử thêm trong các điều kiện vận hành thực tế hơn, như chạy kiosk trong thời gian dài, kiểm thử với nhiều người dùng liên tiếp và đo hiệu năng khi triển khai trong mạng nội bộ. Các cấu hình kiosk ở cấp thiết bị, trình duyệt hoặc hệ điều hành cũng có thể được bổ sung để tăng độ ổn định khi sử dụng ngoài môi trường thật. Tóm lại, đề tài đã xây dựng được một nền tảng tham quan ảo thông minh, làm cơ sở để tiếp tục hoàn thiện hệ thống theo hướng ổn định hơn, dễ cập nhật hơn và hỗ trợ tốt hơn cho hoạt động giới thiệu, truyền thông và tuyển sinh của nhà trường.

PHỤ LỤC

PHỤ LỤC A. BỘ CÂU HỎI KIỂM THỬ AI AGENT VÀ RAG

Phụ lục này trình bày bộ câu hỏi dùng để kiểm thử khả năng trả lời của AI Agent, khả năng truy xuất thông tin từ kho tri thức RAG và khả năng kích hoạt một số hành động giao diện thông qua Function Calling. Việc kiểm thử được thực hiện trên API chat của môi trường production với chế độ stream=false, tts=false và không truyền session_id. Cách cấu hình này giúp tập trung vào phần truy xuất tài liệu, sinh câu trả lời và gọi công cụ giao diện, không tính thời gian tạo giọng nói.

Bảng A.1. Bộ câu hỏi kiểm thử AI Agent và RAG

Mã kiểm thử	Câu hỏi kiểm thử	Nhóm nội dung	Nguồn RAG	Tool action	Thời gian xử lý
RAG01	Giới thiệu tổng quan về Đại học Trà Vinh.	Thông tin chung	5	-	1.749 ms
RAG02	Đại học Trà Vinh được thành lập khi nào?	Lịch sử phát triển	5	-	2.745 ms
RAG03	Trường có những nhóm ngành đào tạo nào?	Ngành đào tạo	5	-	2.831 ms
RAG04	Ngành Công nghệ thông tin học những gì?	Công nghệ thông tin	5	-	2.880 ms
RAG05	Trường có những chính sách học bổng nào?	Học bổng	10	-	5.503 ms
RAG06	Thông tin tuyển sinh của trường gồm những nội dung nào?	Tuyển sinh	5	-	5.519 ms
RAG07	Học phí mới nhất của trường là bao nhiêu?	Dữ liệu dễ thay đổi	5	-	2.978 ms
RAG08	Khuôn viên và cơ sở vật chất của TVU có điểm gì nổi bật?	Cơ sở vật chất	5	-	4.362 ms
RAG09	Thư viện của trường có gì nổi bật?	Địa điểm	5	-	3.421 ms
RAG10	Trường có bao nhiêu sinh viên trong năm hiện tại?	Dữ liệu dễ thay đổi	5	-	1.886 ms
FC01	Dẫn tôi đến Khoa Công nghệ thông tin.	Function Calling điều hướng	5	navigate_to	2.595 ms

Mã kiểm thử	Câu hỏi kiểm thử	Nhóm nội dung	Nguồn RAG	Tool action	Thời gian xử lý
FC02	Mở bản đồ khuôn viên cho tôi xem.	Function Calling bản đồ	5	toggle_map	1.771 ms
FC03	Cho tôi xem hình ảnh thư viện.	Function Calling media	5	navigate_to, show_media	2.679 ms

Kết quả kiểm thử cho thấy 12/13 câu đạt yêu cầu và 1/13 câu đạt một phần. Thời gian xử lý backend trung bình là 3.147,62 ms, dao động từ 1.749 ms đến 5.519 ms, với trung bình 5,38 nguồn RAG được truy xuất cho mỗi câu hỏi. Câu đạt một phần liên quan đến số lượng sinh viên hiện tại do hệ thống chưa nhấn mạnh đầy đủ tính thay đổi theo từng năm của thông tin này. Nhìn chung, kết quả cho thấy AI Agent đã có khả năng khai thác kho tri thức, trả lời câu hỏi và phối hợp với giao diện thông qua các chức năng như điều hướng, mở bản đồ và hiển thị media trong phạm vi đề tài.

PHỤ LỤC B. KẾT QUẢ ĐO API TRÊN MÔI TRƯỜNG PRODUCTION

Phụ lục này trình bày kết quả đo thời gian phản hồi của một số API công khai trên môi trường production. Mỗi API được gọi 10 lần thông qua tên miền <https://www.tvu-tour.site>. Các kết quả này chỉ phản ánh kiểm thử chức năng và thời gian phản hồi cơ bản, chưa phải kiểm thử tải với nhiều người dùng đồng thời.

Bảng B.1. Kết quả đo thời gian phản hồi API public trên môi trường production

API kiểm thử	Số lần	Thời gian TB	Nhỏ nhất	Lớn nhất	Ghi chú
/api/health	10	453,76 ms	396,54 ms	567,10 ms	Kiểm tra trạng thái backend
/api/locations	10	564,76 ms	528,09 ms	599,39 ms	Lấy danh sách địa điểm đang hoạt động
/api/nav/graph	10	548,32 ms	492,34 ms	594,71 ms	Lấy dữ liệu waypoint graph và trạng thái địa điểm
/api/nav/path	10	394,08 ms	375,01 ms	408,93 ms	Tính tuyến đường giữa hai địa điểm
/api/locations/cong-chinh/assets	10	541,69 ms	492,29 ms	598,24 ms	Lấy media của địa điểm Cổng chính

Kết quả cho thấy các API công khai đều phản hồi thành công trong các lần đo. API tính đường có thời gian phản hồi thấp hơn các API cần truy vấn dữ liệu địa điểm, bản đồ hoặc media. Do phép đo được thực hiện qua môi trường public, thời gian phản hồi bao gồm cả độ trễ mạng, tên miền và hạ tầng triển khai.

PHỤ LỤC C. KẾT QUẢ KIỂM THỬ DẪN ĐƯỜNG A^*

Chức năng dẫn đường được kiểm thử bằng cách tìm đường giữa mọi cặp địa điểm đang hoạt động. Tại thời điểm kiểm thử, hệ thống có 4 địa điểm active, tạo thành 12 cặp điểm xuất phát - điểm đích khác nhau.

Bảng C.1. Kết quả tổng hợp kiểm thử dẫn đường A^*

Chỉ số kiểm thử	Kết quả
Số địa điểm active	4
Số cặp điều hướng được kiểm thử	12
Số cặp tìm được đường	12
Số cặp không tìm được đường	0
Thời gian phản hồi API trung bình	410,70 ms
Thời gian tính A^* nội bộ trung bình	0,108 ms
Số điểm trung bình trên một tuyến	7,17

Kết quả kiểm thử cho thấy thuật toán A^* tìm được đường cho toàn bộ các cặp địa điểm đang hoạt động. Thời gian tính toán A^* nội bộ rất nhỏ, cho thấy chi phí xử lý thuật toán không phải là nguyên nhân chính gây độ trễ. Thời gian phản hồi mà người dùng cảm nhận chủ yếu đến từ quá trình gọi API qua môi trường công khai và truy xuất dữ liệu liên quan.

DANH MỤC TÀI LIỆU THAM KHẢO

- [1] R. T. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” PhD Thesis, University of California, Irvine, 2000. [Online]. Available: <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- [2] MDN Web Docs, “Using server-sent events.” 2025. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events/Using_server-sent_events
- [3] Meta Open Source, “React Quick Start.” 2026. [Online]. Available: <https://react.dev/learn>
- [4] Vercel, “Next.js Docs: App Router.” 2026. [Online]. Available: <https://nextjs.org/docs/app>
- [5] Microsoft, “The TypeScript Handbook.” 2026. [Online]. Available: <https://www.typescriptlang.org/docs/handbook/intro.html>
- [6] pmndrs, “Zustand Documentation: Introduction.” 2026. [Online]. Available: <https://zustand.docs.pmnd.rs/getting-started/introduction>
- [7] Motion, “Get started with Motion for React.” 2026. [Online]. Available: <https://motion.dev/docs/react>
- [8] Pannellum, “Pannellum Overview.” 2026. [Online]. Available: <https://pannellum.org/documentation/overview/>
- [9] three.js, “three.js manual.” 2026. [Online]. Available: <https://threejs.org/manual/>
- [10] pmndrs, “React Three Fiber: Introduction.” 2026. [Online]. Available: <https://r3f.docs.pmnd.rs/getting-started/introduction>
- [11] FastAPI, “FastAPI Documentation.” 2026. [Online]. Available: <https://fastapi.tiangolo.com/>
- [12] SQLAlchemy, “SQLAlchemy ORM Documentation.” 2026. [Online]. Available: <https://docs.sqlalchemy.org/en/20/orm/>
- [13] Supabase, “Supabase Database Documentation.” 2026. [Online]. Available: <https://supabase.com/docs/guides/database/overview>
- [14] pgvector, “pgvector: Open-source vector similarity search for Postgres.” 2026. [Online]. Available: <https://github.com/pgvector/pgvector>
- [15] Cloudflare, “Cloudflare R2 Documentation.” 2026. [Online]. Available: <https://developers.cloudflare.com/r2/>
- [16] Supabase, “Auth | Supabase Docs.” Accessed: Jun. 05, 2026. [Online]. Available: <https://supabase.com/docs/guides/auth/>
- [17] Google AI for Developers, “Gemini API Documentation.” 2026. [Online]. Available: <https://ai.google.dev/gemini-api/docs>
- [18] P. Lewis *et al.*, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” Apr. 12, 2021, *arXiv*: arXiv:2005.11401. doi: 10.48550/arXiv.2005.11401.
- [19] Google AI for Developers, “Function calling with the Gemini API.” 2026. [Online]. Available: <https://ai.google.dev/gemini-api/docs/function-calling>

- [20] L. Wang *et al.*, “A Survey on Large Language Model based Autonomous Agents,” *Front. Comput. Sci.*, vol. 18, no. 6, p. 186345, Dec. 2024, doi: 10.1007/s11704-024-40231-1.
- [21] MDN Web Docs, “Web Speech API.” 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API
- [22] Google AI for Developers, “Text-to-speech generation with the Gemini API.” 2026. [Online]. Available: <https://ai.google.dev/gemini-api/docs/speech-generation>
- [23] P. E. Hart, N. J. Nilsson, and B. Raphael, “A Formal Basis for the Heuristic Determination of Minimum Cost Paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968, doi: 10.1109/TSSC.1968.300136.